

## RESUMEN

El presente Trabajo Fin de Máster consiste en el diseño de un sistema que permite la extracción y consulta conversacional de información técnica contenida en informes CAP, documentos empleados en el diseño y validación de nodos en redes móviles. El sistema, desarrollado íntegramente en Python, combina técnicas de reconocimiento óptico de caracteres (OCR), procesamiento de lenguaje natural (NLP), y modelos de lenguaje (LLM) de código abierto, con el fin de transformar automáticamente los datos de los CAP en estructuras JSON. Posteriormente, esta información es indexada mediante técnicas de vectorización y puesta a disposición del usuario mediante un chatbot, lo que facilita su consulta tanto por el equipo de diseño radio como por el cliente. El proyecto se ha desarrollado en colaboración con Accenture, en el marco de la mención dual del Máster en Ingeniería de Telecomunicación, atendiendo a restricciones reales de confidencialidad y rendimiento computacional, con el objetivo de ofrecer una solución práctica, escalable y alineada con las necesidades operativas del sector y de la empresa.

## RESUM

Aquest Treball de Fi de Màster presenta el disseny d'un sistema que permet l'extracció i consulta conversacional d'informació tècnica continguda en informes CAP, documents utilitzats en el disseny i la validació de nodes en xarxes mòbils. El sistema, desenvolupat íntegrament en Python, combina tècniques com el reconeixement òptic de caràcters (OCR), el processament del llenguatge natural (NLP) i models de llenguatge (LLM) de codi obert, amb l'objectiu de transformar automàticament les dades dels CAP en estructures JSON. Posteriorment, aquesta informació és indexada mitjançant tècniques de vectorització i es posa a disposició de l'usuari mitjançant una interfície de xatbot, la qual cosa facilita la consulta tant per part de l'equip de disseny radio com del client. El projecte s'ha desenvolupat en col·laboració amb Accenture, en el marc de la menció dual del Màster en Enginyeria de Telecomunicació, atenent a restriccions reals de confidencialitat i rendiment computacional, amb la finalitat d'oferir una solució pràctica, escalable i alineada amb les necessitats operatives del sector i de l'empresa.

## ABSTRACT

This master's Thesis presents the design of a system that enables the extraction and conversational querying of technical information contained in CAP reports, documents used in the design and validation of mobile network nodes. The system, developed entirely in Python, combines techniques such as Optical Character Recognition (OCR), Natural Language Processing (NLP), and open-source Large Language Models (LLMs) to automatically transform the data from CAPs into structured JSON files. This information is subsequently indexed using vectorization techniques and made available to the user through a chatbot interface, facilitating access both for the radio design team and for the client. The project has been developed in collaboration with Accenture, within the dual-track framework of the master's degree in Telecommunication Engineering, considering real-world constraints regarding confidentiality and computational performance, with the aim of delivering a practical, scalable solution aligned with the operational needs of the industry and the company.



## RESUMEN EJECUTIVO

La memoria del TFM del Máster Universitario en Ingeniería de Telecomunicación debe desarrollar en el texto los siguientes conceptos, debidamente justificados y discutidos, centrados en el ámbito de la ingeniería de telecomunicación.

| CONCEPT (ABET)                                                                     | CONCEPTO (traducción)                                                                                                       | ¿Cumple?<br>(S/N) | ¿Dónde?<br>(páginas)                    |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------|
| 1. IDENTIFY:                                                                       | 1. IDENTIFICAR:                                                                                                             |                   |                                         |
| 1.1. Problem statement and opportunity                                             | 1.1. Planteamiento del problema y oportunidad                                                                               | Sí                | Cap. 1 (págs. 6–7)                      |
| 1.2. Constraints (standards, codes, needs, requirements & specifications)          | 1.2. Toma en consideración de los condicionantes (normas técnicas y regulación, necesidades, requisitos y especificaciones) | Sí                | Cap. 1.2 y Cap. 3.1 (pág. 7 y 28)       |
| 1.3. Setting of goals                                                              | 1.3. Establecimiento de objetivos                                                                                           | Sí                | Cap. 1.3 (pág. 7)                       |
| 2. FORMULATE:                                                                      | 2. FORMULAR:                                                                                                                |                   |                                         |
| 2.1. Creative solution generation (analysis)                                       | 2.1. Generación de soluciones creativas (análisis)                                                                          | Sí                | Cap. 2.2 y Cap. 4 (págs. 18-27 y 34-38) |
| 2.2. Evaluation of multiple solutions and decision-making (synthesis)              | 2.2. Evaluación de múltiples soluciones y toma de decisiones (síntesis)                                                     | Sí                | Cap. 4, 5 y 6 (págs. 34-64)             |
| 3. SOLVE:                                                                          | 3. RESOLVER:                                                                                                                |                   |                                         |
| 3.1. Fulfilment of goals                                                           | 3.1. Evaluación del cumplimiento de objetivos                                                                               | Sí                | Cap. 7.1 (pág. 65)                      |
| 3.2. Overall impact and significance (contributions and practical recommendations) | 3.2. Evaluación del impacto global y alcance (contribuciones y recomendaciones prácticas)                                   | Sí                | Cap. 6.3 y Cap. 7.2 (págs. 63-64 y 66)  |



## ÍNDICE DE CONTENIDO

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>CAPÍTULO 1. INTRODUCCIÓN</b>                                                              | <b>6</b>  |
| 1.1 PLANTEAMIENTO DEL PROBLEMA Y MOTIVACIÓN                                                  | 6         |
| 1.2 CONTEXTO Y CONDICIONANTES                                                                | 7         |
| 1.3 OBJETIVOS                                                                                | 7         |
| 1.4 ESTRUCTURA DE LA MEMORIA                                                                 | 8         |
| <b>CAPÍTULO 2. ESTADO DEL ARTE</b>                                                           | <b>9</b>  |
| 2.1 REDES MÓVILES Y DISEÑO RADIO                                                             | 9         |
| 2.1.1 Tipos de emplazamiento                                                                 | 9         |
| 2.1.2 Elementos del sistema radiante                                                         | 10        |
| 2.1.3 Orientaciones y tilts                                                                  | 13        |
| 2.1.4 Criterios de diseño y selección                                                        | 15        |
| 2.2 FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL APLICADOS EN EXTRACCIÓN Y CONSULTA DE INFORMACIÓN | 18        |
| 2.2.1 Extracción de información de documentos PDF                                            | 18        |
| 2.2.2 Procesamiento del Lenguaje Natural (NLP)                                               | 20        |
| <b>CAPÍTULO 3. FUNDAMENTOS TÉCNICOS</b>                                                      | <b>28</b> |
| 3.1 ENTORNO Y CONDICIONANTES TÉCNICOS                                                        | 28        |
| 3.2 ESTRUCTURA Y CONTENIDO DE LOS DOCUMENTOS CAP                                             | 29        |
| 3.3 TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS                                                    | 31        |
| 3.3.1 Librerías utilizadas                                                                   | 31        |
| 3.3.1 Plataforma y modelos LLM                                                               | 32        |
| <b>CAPÍTULO 4. DISEÑO DEL SISTEMA</b>                                                        | <b>34</b> |
| 4.1 ARQUITECTURA GENERAL DEL SISTEMA                                                         | 34        |
| 4.2 MÓDULO DE EXTRACCIÓN                                                                     | 35        |
| 4.3 MÓDULO DE VECTORIZACIÓN                                                                  | 36        |
| 4.4 MÓDULO RAG                                                                               | 37        |
| 4.5 INTERFAZ DE USUARIO                                                                      | 37        |
| <b>CAPÍTULO 5. DESARROLLO E IMPLEMENTACIÓN DEL SISTEMA</b>                                   | <b>39</b> |
| 5.1 ORGANIZACIÓN DEL PROYECTO Y ESTRUCTURA DE CARPETAS                                       | 39        |
| 5.2 IMPLEMENTACIÓN DEL PIPELINE DE EXTRACCIÓN                                                | 40        |
| 5.2.1 Script de extracción                                                                   | 42        |
| 5.2.2 Scripts de normalización y anonimización                                               | 45        |
| 5.3 IMPLEMENTACIÓN DEL MÓDULO DE VECTORIZACIÓN                                               | 47        |
| 5.4 IMPLEMENTACIÓN DEL SISTEMA RAG                                                           | 49        |
| 5.5 INTEGRACIÓN DEL BACKEND VÍA API                                                          | 53        |
| 5.6 DESARROLLO DE LA INTERFAZ CONVERSACIONAL                                                 | 54        |
| <b>CAPÍTULO 6. EVALUACIÓN Y SIMULACIÓN DEL SISTEMA</b>                                       | <b>56</b> |
| 6.1 EVALUACIÓN DE LOS MODELOS LLM EMPLEADOS                                                  | 56        |
| 6.2 SIMULACIÓN DE USO EN ENTORNO REAL                                                        | 59        |
| 6.3 LIMITACIONES OBSERVADAS Y FIABILIDAD DEL SISTEMA                                         | 63        |
| <b>CAPÍTULO 7. CONCLUSIONES Y TRABAJO FUTURO</b>                                             | <b>65</b> |
| 7.1 CONCLUSIONES                                                                             | 65        |
| 7.2 TRABAJO FUTURO                                                                           | 66        |
| <b>REFERENCIAS</b>                                                                           | <b>67</b> |



|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>ANEXOS .....</b>                                            | <b>69</b> |
| ANEXO A. CÓDIGO COMPLEMENTARIO .....                           | 69        |
| A.1 Módulos auxiliares de extracción (utils y parsers).....    | 69        |
| A.2 Funciones complementarias del script de normalización..... | 73        |
| A.3 Lógica adicional del módulo de vectorización.....          | 76        |
| ANEXO B. MANUAL DE USUARIO .....                               | 79        |
| B.1 Introducción .....                                         | 79        |
| B.2 Preparación del entorno.....                               | 79        |
| B.3 Descripción general del sistema .....                      | 80        |
| B.4 Instrucciones de uso .....                                 | 81        |

## ÍNDICE DE FIGURAS

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Figura 1. Ejemplos de tipos de soporte .....                                                | 10 |
| Figura 2. Esquema funcional del sistema radiante .....                                      | 11 |
| Figura 3. Ancho de haz.....                                                                 | 14 |
| Figura 4. Comparativa entre MDT y EDT .....                                                 | 15 |
| Figura 5. Representación de las distancias del PLL de una antena emisora .....              | 17 |
| Figura 6. Ejemplo de incumplimiento del PLL: intersección con zona transitable.....         | 17 |
| Figura 7. Diagrama de bloques de las etapas principales de un sistema OCR típico .....      | 19 |
| Figura 8. Ejemplo del funcionamiento de embeddings semánticos en un espacio vectorial.....  | 22 |
| Figura 9. Arquitectura Transformer.....                                                     | 24 |
| Figura 10. Esquema de recuperación semántica con FAISS .....                                | 26 |
| Figura 11. Esquema Modelo A CAP .....                                                       | 30 |
| Figura 12. Esquema modelo B CAP .....                                                       | 30 |
| Figura 13. Arquitectura del sistema.....                                                    | 34 |
| Figura 14. Código fuente del pipeline ETL.....                                              | 41 |
| Figura 15. Código fuente del script de extracción.....                                      | 43 |
| Figura 16. Script de extracción de texto OCR.....                                           | 44 |
| Figura 17. Script de extracción de texto OCR por color .....                                | 45 |
| Figura 18. Script de normalización .....                                                    | 46 |
| Figura 19. Script de anonimización.....                                                     | 47 |
| Figura 20. Ejemplo de fragmentación del bloque de consumos.....                             | 48 |
| Figura 21. Main del script de vectorización .....                                           | 49 |
| Figura 22. Función detectar sección relevante .....                                         | 50 |
| Figura 23. Funciones de recuperación de fragmentos .....                                    | 51 |
| Figura 24. Función de construir prompt y generar respuesta .....                            | 52 |
| Figura 25. Función responder pregunta.....                                                  | 52 |
| Figura 26. Script principal de la API.....                                                  | 54 |
| Figura 27. Parte del script frontend que selecciona el nodo.....                            | 55 |
| Figura 28. Parte del script frontend que genera la interfaz de conversación.....            | 55 |
| Figura 29. Gráfico de barras con los resultados obtenidos por cada modelo .....             | 58 |
| Figura 30. Resultados totales ponderados .....                                              | 58 |
| Figura 31. Inicio del entorno web del sistema.....                                          | 60 |
| Figura 32. Inicio del servidor local de la API mediante Uvicorn.....                        | 60 |
| Figura 33. Interfaz web tras validar el nodo introducido .....                              | 60 |
| Figura 34. Selección automática de la sección más relevante según similitud semántica ..... | 60 |
| Figura 35. Fragmentos recuperados del índice FAISS y utilizados como contexto. ....         | 61 |
| Figura 36. Respuesta generada por el modelo LLM a partir de los fragmentos seleccionados .  | 62 |



|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Figura 37. Respuesta del sistema ante una consulta sin información disponible ..... | 63 |
| Figura 38. Script parse_identificadores.py .....                                    | 70 |
| Figura 39. Script parse_coordenadas.py .....                                        | 71 |
| Figura 40. Script coord_utils.py .....                                              | 72 |
| Figura 41. Script pdf_utils.py .....                                                | 73 |
| Figura 42. Función normalizar consumos .....                                        | 74 |
| Figura 43. Función normalizar antenas .....                                         | 75 |
| Figura 44. Función normalizar hardware .....                                        | 76 |
| Figura 45. Funciones auxiliares del script de vectorización.....                    | 78 |

## ÍNDICE DE TABLAS

|                                                      |    |
|------------------------------------------------------|----|
| Tabla 1. Modelos LLM.....                            | 32 |
| Tabla 2. Descripción de la metodología empleada..... | 57 |
| Tabla 3. Resultados por preguntas.....               | 57 |

## Capítulo 1. Introducción

El presente Trabajo Fin de Máster se ha desarrollado en colaboración con Accenture, empresa perteneciente al marco de la mención dual del Máster Universitario en Ingeniería de Telecomunicación (MUIT). Durante esta etapa profesional, trabajé en tareas de automatización de procesos con Python para los equipos de diseño e implementación radio, lo que me permitió identificar una necesidad recurrente en el entorno de trabajo relacionada con la gestión de la documentación técnica. A partir de esta experiencia, surgió la motivación para desarrollar una solución basada en inteligencia artificial que pudiera optimizar el acceso a la información en este ámbito.

### 1.1 Planteamiento del problema y motivación

Una de las principales dificultades detectadas fue la dispersión de la información técnica relativa a los nodos, repartida en múltiples ficheros y plataformas del cliente. Esta fragmentación no solo complicaba la localización de datos clave, sino que también aumentaba el riesgo de errores humanos durante la elaboración o revisión de la documentación.

Los documentos CAP (*Customer Acceptance Package*) constituyen la fuente oficial de información técnica de los nodos. Se trata de entregables formales que se presentan al operador con el objetivo de acreditar que el diseño radio y la puesta en servicio cumplen los requisitos técnicos y contractuales establecidos. Estos documentos recopilan de forma estructurada todos los datos relevantes del nodo, incluyendo configuraciones, tecnologías instaladas, antenas, orientaciones, y demás parámetros clave para su validación y aceptación.

Con la rápida evolución de la inteligencia artificial y los modelos de lenguaje, se identificó una oportunidad para incorporar estas tecnologías con el fin de facilitar el acceso a los datos técnicos, mediante el diseño de un sistema de extracción y consulta conversacional de la información contenida en los documentos CAP.

La motivación del proyecto no se limitó a resolver un problema inmediato, sino que también buscó demostrar cómo herramientas basadas en inteligencia artificial pueden integrarse progresivamente en el flujo de trabajo de los equipos, con la posibilidad de evolucionar hacia sistemas más avanzados, capaces de sugerir diseños o configuraciones automáticas que asistan al equipo de diseño en sus tareas cotidianas.

Además, este sistema no solo resulta útil para el equipo interno de diseño, sino que puede extenderse como herramienta de soporte para el cliente. Al permitir consultas rápidas y precisas sobre el estado y configuración de sus nodos, donde el cliente podrá acceder directamente a información técnica sin necesidad de recurrir a soporte técnico, reduciendo así los tiempos de respuesta.

## 1.2 Contexto y condicionantes

El desarrollo del proyecto estuvo condicionado por diversos factores técnicos, operativos y económicos que determinaron tanto la arquitectura como las herramientas seleccionadas.

En primer lugar, al tratarse de un proyecto académico sin dotación presupuestaria, fue necesario optar exclusivamente por soluciones de código abierto y gratuitas. Esta decisión implicó descartar modelos más avanzados o servicios en la nube que ofrecieran mejores prestaciones, priorizando aquellos modelos generativos que pudieran ejecutarse en entornos locales con recursos limitados.

Además, el entorno de despliegue previsto para el sistema era una estación de trabajo convencional, sin acceso a servidores con capacidades gráficas avanzadas. Esta restricción obligó a optimizar el consumo de memoria y procesamiento, tanto en la elección del modelo LLM como en la implementación de los mecanismos de búsqueda.

Por otro lado, la naturaleza técnica de los documentos tratados exigía garantizar la confidencialidad y el cumplimiento de las políticas de protección de datos de la empresa. Por ello, se estableció como requisito no hacer referencia explícita a identificadores reales de nodos ni almacenar información sensible del cliente. También fue necesario implementar procesos de extracción y normalización robustos, ya que los documentos, si bien estandarizados, presentan variaciones de formato que afectan a la automatización del procesamiento.

Estos condicionantes definieron el alcance real del proyecto, la elección de herramientas utilizadas y la metodología de desarrollo empleada.

## 1.3 Objetivos

El objetivo general de este trabajo es diseñar e implementar un sistema de consulta conversacional en Python capaz de extraer y ofrecer información técnica contenida en los documentos CAP, utilizados en el ámbito del diseño radio en redes móviles.

Para alcanzar este propósito, se plantearon los siguientes objetivos específicos:

- Integrar un sistema de extracción automática de información técnica a partir de documentos en formato PDF, garantizando su estructuración, normalización y anonimato.
- Diseñar un motor de búsqueda semántica que permita localizar información relevante en el contexto dada una pregunta o consulta del usuario.
- Implementar y evaluar el sistema con varios modelos de lenguaje de gran tamaño de código abierto en entorno local, capaces de responder a consultas conversacionales con precisión y coherencia.
- Desarrollar una interfaz conversacional sencilla e intuitiva para facilitar el acceso a la información por parte de perfiles técnicos y no técnicos.
- Validar el sistema a través de casos de uso representativos, evaluando su precisión, rendimiento y utilidad operativa en el contexto del diseño radio.



## 1.4 Estructura de la memoria

La presente memoria se organiza en nueve capítulos que siguen una progresión lógica. El documento comienza con el actual capítulo, donde se ha planteado el problema, contextualizado el trabajo y definido los objetivos. A continuación, el Capítulo 2 recoge el estado del arte sobre los emplazamientos de redes móviles y técnicas de inteligencia artificial aplicadas, con especial énfasis en el procesamiento del lenguaje natural, los modelos LLM y la arquitectura RAG, sirviendo como base teórica para el desarrollo posterior.

En el Capítulo 3, se establecen los fundamentos técnicos, incluyendo las herramientas y tecnologías utilizadas, así como una explicación de la estructura de los documentos CAP. El Capítulo 4 describe el diseño del sistema, donde se explica la arquitectura del sistema y los módulos que la componen. El Capítulo 5 documenta el desarrollo e implementación, mientras que el Capítulo 6 presenta la evaluación funcional del sistema y de los modelos LLM empleados.

Por último, el Capítulo 7 sintetiza las conclusiones y plantea posibles mejoras, mientras que las referencias y anexos aportan soporte documental y detalles complementarios.



## Capítulo 2. Estado del arte

Este capítulo presenta una revisión del estado del arte en dos dominios fundamentales para el desarrollo del presente trabajo: las redes móviles, enfocadas en el ámbito de diseño radio, y la inteligencia artificial aplicada a entornos de ingeniería y procesamiento de información.

En primer lugar, se expone una visión general de los principios esenciales del diseño radio, es decir, los tipos de emplazamientos y los elementos que componen el sistema radiante, así como los criterios que se tienen en cuenta en el diseño. Posteriormente, se comentarán las herramientas o técnicas de inteligencia artificial aplicada empleadas en el presente trabajo, tales como el procesamiento del lenguaje natural (NLP), los modelos de lenguaje de gran tamaño (LLM) o la arquitectura de recuperación aumentada (RAG).

Al finalizar el capítulo, se obtendrá una base conceptual y técnica sobre la que se sustenta el sistema de extracción y consulta propuesto.

### 2.1 Redes móviles y diseño radio

En esta sección, se explican los tipos de emplazamientos más comunes, así como los elementos que componen el sistema radiante y los criterios de diseño que se tienen en cuenta. El objetivo es proporcionar un marco técnico coherente con la documentación que será procesada y consultada por el sistema.

#### 2.1.1 Tipos de emplazamiento

Los emplazamientos de red móvil pueden clasificarse según su entorno en emplazamientos urbanos, situados en zonas altamente pobladas, y emplazamientos rurales, ubicados en zonas menos pobladas. Adicionalmente, también se pueden clasificar según la ubicación física de los equipos en *indoor* (interiores) o *outdoor* (exteriores).

Por otro lado, el tipo de soporte o estructura sobre la que se instalan los equipos también es importante a la hora de diseñar un nodo. Estas pueden ser:

- Torres de celosía: estructura metálica con acceso por el interior, diseñada para ofrecer alta resistencia al viento. Es una de las más utilizadas en emplazamientos rurales y de gran altura por su bajo coste, su poco peso y por facilidad de mantenimiento.
- Torres tubular: estructura metálica con acceso por el exterior, lo que puede reducir la percepción de seguridad durante el mantenimiento. Su principal ventaja es el menor impacto visual, por lo que suele instalarse en zonas donde la estética urbana es prioritaria.
- Troncos piramidales: estructura metálica de base ancha y gran altura. Se suelen instalar en ubicaciones elevadas con alta densidad de tráfico, donde se requiere soportar múltiples antenas y tecnologías en un mismo emplazamiento.

- Mástiles: se instalan en emplazamientos urbanos, normalmente en azoteas de edificios y su principal ventaja es que se sostienen por sí mismas sin necesidad de apoyos adicionales, lo que facilita despliegues rápidos en espacios reducidos.
- Estructuras mimetizadas: están diseñadas para integrarse visualmente en el entorno urbano, adoptan formas de elementos comunes como vallas publicitarias, chimeneas, farolas o mobiliario urbano. Se emplean en zonas con limitaciones estéticas o normativas, donde es necesario mantener la funcionalidad radio sin afectar el paisaje.

En la Fig. 1 se muestra un ejemplo de cada tipo de estructura, ordenadas de izquierda a derecha según el orden de citación en el texto.



*Figura 1. Ejemplos de tipos de soporte*

Este tipo de estructuras o soportes son la base donde se instalan los equipos que dan lugar al sistema radiante, el cual se explicará con más detalle en el siguiente apartado.

### 2.1.2 Elementos del sistema radiante

El sistema radiante de una estación base está constituido por el conjunto de elementos responsables de la transmisión y recepción de señales de radiofrecuencia entre la red y los dispositivos móviles. Su diseño y correcta implementación resultan fundamentales para garantizar los niveles de cobertura, capacidad y calidad de servicio definidos en la planificación radio.

En la Fig. 2 se presenta un esquema representativo del sistema radiante, donde se ilustran los principales bloques funcionales y su disposición relativa dentro de una estación base.

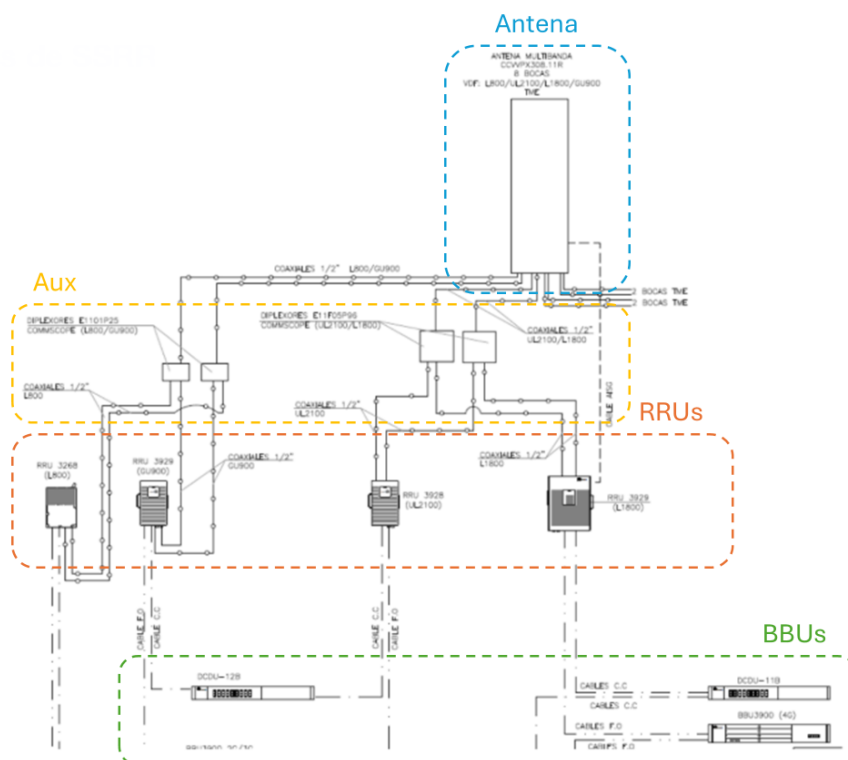


Figura 2. Esquema funcional del sistema radiante

Los elementos que integran el sistema radiante se clasifican en componentes activos, como las unidades de radio, y pasivos, como las antenas, diplexores, combinadores, cableado y otros elementos auxiliares. Todos ellos están interconectados de forma jerárquica, formando un sistema coherente que transforma la señal digital en una electromagnética.

En los siguientes subapartados se detallan los elementos más relevantes que conforman el sistema radiante, sus funciones técnicas y las implicaciones de diseño asociadas a cada uno.

### 2.1.2.1 Antenas

Las antenas constituyen el componente pasivo fundamental del sistema radiante, cuya función es emitir y recibir ondas electromagnéticas. Están diseñadas para soportar uno o varios rangos de frecuencia, lo que posibilita su uso en configuraciones multibanda y su integración en despliegues de tecnologías móviles coexistentes.

Cada antena se estructura en pares de bocas, clasificadas según la banda de frecuencia: LB (*Low Band*) para frecuencias bajas y HB (*High Band*) para frecuencias altas.

Entre los tipos más comunes empleados en el diseño de redes móviles destacan:

- **Antenas de panel:** ofrecen cobertura direccional y permiten la sectorización de la celda. Son ampliamente utilizadas en estaciones urbanas y suburbanas.

- **Antenas omnidireccionales:** proporcionan una cobertura uniforme en 360°, adecuadas para entornos rurales o de difícil acceso donde se requiere cubrir múltiples direcciones desde un único punto central.

La selección del tipo de antena, su ubicación y sus características eléctricas (como el ancho de haz o el tilt) son aspectos críticos que influyen directamente en la cobertura y en el comportamiento del enlace radio en cada sector.

### 2.1.2.2 Unidades de radio y banda base

Las unidades de radio son los elementos activos del sistema radiante encargados de generar la señal de radiofrecuencia que será transmitida por las antenas. Se distinguen dos variantes: las RRU (*Remote Radio Unit*), que se instalan en exteriores (*outdoor*), y las RU (*Radio Unit*), utilizadas en configuraciones de interior (*indoor*).

Cada unidad de radio está generalmente dedicada a una única frecuencia, aunque también existen modelos multifrecuencia que integran, en un solo equipo, soporte para bandas combinadas como 800/900 MHz o 1800/2100 MHz. Estas unidades disponen habitualmente de dos o cuatro bocas de conexión para cada frecuencia, permitiendo así configurar sistemas de transmisión MIMO.

Por otro lado, la BBU (*Baseband Unit*) o la unidad de banda base, se encarga del procesamiento digital de la señal (modulación, demodulación, corrección de errores, etc.), se suele alojar en interior (*indoor*) y se enlaza con las RRU mediante fibra óptica, minimizando las pérdidas, mientras que las RRU se conectan a las antenas mediante cable coaxial, de forma directa o a través de dispositivos pasivos.

### 2.1.2.3 Elementos pasivos auxiliares

Además de las antenas y unidades de radio y banda base, el sistema radiante se compone de varios elementos pasivos auxiliares que optimizan la transmisión de señal y aseguran un funcionamiento robusto del sistema. A continuación, se describen los más relevantes:

- **Diplexores y triplexores:** permiten combinar señales de diferentes bandas de frecuencia para ser transmitidas por una única línea coaxial hacia la antena. Esto resulta especialmente útil cuando el número de bocas disponibles en la antena es limitado o se busca reducir el cableado. Estos dispositivos deben ser compatibles con todo el ancho de banda de las señales implicadas, y algunos modelos requieren el uso de dos unidades para completar una conexión.
- **Combinadores:** se utilizan para combinar señales que operan en la misma banda de frecuencia, normalmente de distintos operadores. Existen combinadores de cavidad, que no provocan intermodulación pasiva (PIM) pero son más costosos, y combinadores híbridos, más económicos pero susceptibles de generar PIM, especialmente en bandas bajas como la de 900 MHz. En estos últimos, se instala una carga en la cuarta boca para evitar emisiones no deseadas.
- **Cargas:** son resistencias colocadas en bocas libres de antenas, combinadores o RRU con el fin de evitar la emisión de señales no deseadas. También se emplean como herramienta

de diagnóstico para identificar fallos o interferencias provocadas por elementos defectuosos del sistema radiante.

- **Splitters:** dispositivos que permiten dividir la señal procedente de una RRU entre varias antenas, facilitando así la distribución de cobertura en múltiples sectores cuando no se dispone de transmisores independientes para cada uno.
- **Atenuadores:** reducen el nivel de potencia de la señal de radiofrecuencia, siendo especialmente útiles en instalaciones interiores como hospitales, donde se requiere limitar la potencia radiada por motivos de seguridad o normativas específicas.
- **Amplificadores:** incrementan la potencia de la señal RF en el enlace ascendente, mejorando así la relación señal-ruido en trayectos largos o con pérdidas significativas. Se instalan cerca de la antena para maximizar su efectividad.

#### 2.1.2.4 Cableado

Por último, el cableado constituye un elemento esencial en el sistema radiante, ya que permite la interconexión entre las unidades activas y pasivas del sistema. Se emplean dos tipos de cableado principales:

- **Cable coaxial:** se utiliza para el transporte de señales RF entre elementos cercanos del sistema, como la conexión entre la RRU y la antena o entre la RRU y los elementos pasivos auxiliares. Se distinguen dos categorías según su longitud:
  - **Jumpers:** empleados para distancias cortas (hasta 5 metros), habitualmente entre elementos montados en la misma estructura.
  - **Feeders:** utilizados para mayores distancias, con dos calibres estándar: media pulgada, para tiradas entre 5 y 20 metros, y 7/8 de pulgada para tiradas superiores a 20 metros.
- **Fibra óptica:** se encarga de conectar las RRU con las BBU, permitiendo el transporte eficiente de la señal digital con mínimas pérdidas y mayor inmunidad frente a interferencias electromagnéticas.

La correcta selección del tipo de cableado y su instalación adecuada son fundamentales para asegurar el rendimiento del sistema radiante y minimizar pérdidas, reflexiones o degradaciones de señal.

#### 2.1.3 Orientaciones y tilts

Una vez diseñado el sistema radiante, hay que tener en cuenta dos parámetros fundamentales: la orientación y el tilt. Ambos determinan cómo se distribuye espacialmente la señal radiada, influyendo directamente en la cobertura y la calidad de la señal. Una configuración adecuada de estos ajustes permite optimizar el aprovechamiento del espectro y adaptar la red a las condiciones geográficas y de demanda específicas de cada emplazamiento.

### 2.1.3.1 Orientación

La orientación de una antena es la dirección angular hacia la que se dirige el máximo de su haz horizontal. Se expresa en grados sexagesimales, tomando como referencia el norte geográfico ( $0^\circ$ ) y siguiendo el sentido de las agujas del reloj hasta  $360^\circ$ . Esta orientación es esencial en el diseño radio, ya que determina la zona geográfica que será cubierta por el sector correspondiente y permite coordinar adecuadamente el reparto de cobertura entre estaciones vecinas.

El ancho de haz horizontal (*Horizontal Beamwidth, HBW*) representa el ángulo comprendido entre los puntos donde la ganancia de la antena disminuye 3 dB respecto a su valor máximo (véase Fig. 3). Este parámetro define la zona de radiación de la antena donde la captación de la señal es la adecuada, entre el 70% y el 100% de su ganancia máxima y, junto con la orientación, determina la zona efectiva de cobertura.

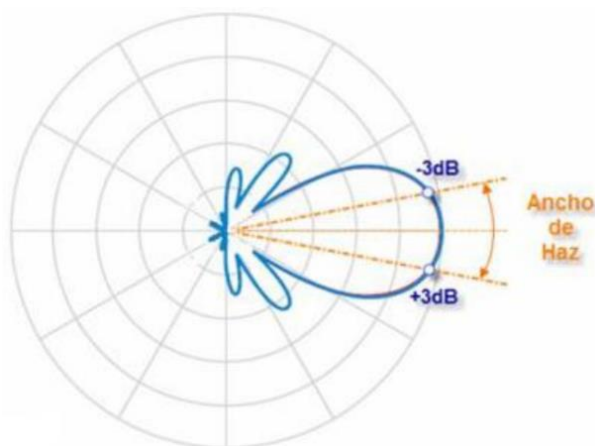


Figura 3. Ancho de haz

Por defecto, las antenas de panel empleadas en redes móviles suelen tener un HBW de  $60^\circ$ , lo que permite implementar configuraciones estándar de tres sectores con orientaciones espaciadas a  $0^\circ$ ,  $120^\circ$  y  $240^\circ$ , garantizando así una cobertura uniforme y sin solapamientos.

En función del entorno, el diseño de orientaciones varía. En zonas urbanas y suburbanas predominan las antenas de panel orientadas hacia sectores bien definidos, mientras que en entornos rurales se utilizan antenas omnidireccionales, tal como se ha comentado en el anterior apartado.

Es importante destacar que, en la medida de lo posible, hay que evitar el solapamiento de haces entre antenas adyacentes. En caso de que haya más de 3 sectores, tiene que estar justificado y la separación mínima entre haces horizontales tiene que ser mínimo de  $10^\circ$ .

### 2.1.3.2 Tilts

Por otro lado, el tilt de una antena es la inclinación vertical de su haz principal de radiación respecto al plano horizontal. Esta inclinación se utiliza para controlar la distancia a la que se proyecta la señal, optimizar la cobertura sobre el área objetivo y limitar el solapamiento no deseado con sectores o estaciones cercanas.

Existen dos tipos de tilt:

- **Tilt mecánico (*Mechanical Downtilt, MDT*):** consiste en inclinar físicamente la antena hacia abajo mediante el ajuste del soporte de sujeción. Es una solución sencilla, pero de aplicación estática, ya que requiere intervención manual.
- **Tilt eléctrico (*Electrical Downtilt, EDT*):** se implementa internamente mediante la variación del desfase entre los elementos radiantes de la antena. Permite modificar la dirección del haz vertical sin mover físicamente la antena, y puede ser fijo o variable. Hoy en día, para evitar desplazamientos al emplazamiento, el EDT es gestionado de forma remota a través del gestor y recibe el nombre de RET (*Remote Electrical Tilt*).

La diferencia entre ambos tipos de tilt puede observarse en la Fig. 4, donde a la izquierda se representa el MDT y a la derecha el EDT.

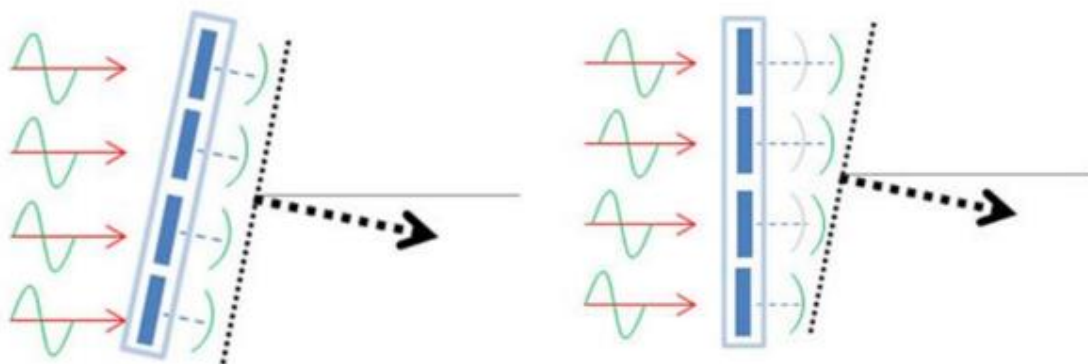


Figura 4. Comparativa entre MDT y EDT

Como se ha comentado, tanto la orientación como el tilt son parámetros fundamentales en la planificación radio, ya que determinan cómo se distribuye la potencia del sistema radiante en el entorno. Una orientación incorrecta puede provocar zonas de sombra o solapamientos innecesarios, mientras que un tilt mal ajustado afecta directamente a la distancia de cobertura efectiva y al nivel de interferencia entre celdas adyacentes. Ambos influyen en el perfil de propagación, las pérdidas de trayecto y, por tanto, en la calidad del enlace radio y la eficiencia espectral del sistema.

## 2.1.4 Criterios de diseño y selección

Por último, hay que comentar que la elección del emplazamiento adecuado y su estructura asociada responde a una combinación de criterios técnicos, normativos y operativos. Estos factores determinan la viabilidad del nodo, su impacto radioeléctrico y su integración con la infraestructura de red existente. A continuación, se describen los principales aspectos a considerar.

### 2.1.3.1 Estudios de carga (CVE)

En primer lugar, los estudios de carga determinan si una torre o mástil existente puede soportar la instalación de nuevos elementos radio. Para ello, se evalúan parámetros como la altura total de la



estructura, el tipo de torre, el peso de los equipos actuales y futuros, y su superficie proyectada al viento. Este análisis garantiza que la estructura no supere su capacidad máxima de carga, preservando la seguridad del emplazamiento.

### 2.1.3.2 Paralelepípedos (PLL)

El segundo criterio para tener en cuenta son los paralelepípedos (PLL). Estos hacen referencia al volumen que permite verificar el cumplimiento normativo en relación con la exposición a campos electromagnéticos, según lo establecido en el Real Decreto 1066/2001 (BOE, 2001) [2]. Se define alrededor del sistema radiante y representa la zona en la que podrían superarse los niveles de referencia de exposición para la población general o el personal técnico, por lo que este volumen no debería cubrir ninguna superficie transitable. El PLL se calcula en función de las frecuencias utilizadas, la potencia radiada y sigue la siguiente fórmula:

$$D_{max} = \sqrt{\frac{M \cdot PIRE}{4\pi \cdot S_{max}}} \quad (1)$$

Dónde M es el coeficiente de reflexión,  $S_{max}$  la densidad de potencia máxima permitida para las frecuencias de trabajo y  $D_{max}$  la distancia al punto donde la densidad de potencia es  $S_{max}$ .

A partir de  $D_{max}$  se calculan las dimensiones del paralelepípedo según las proyecciones de los ángulos y las ganancias de la antena en los distintos planos. Estas distancias definen los límites del volumen en el que la densidad de potencia puede superar  $S_{max}$ :

$$L_{m1} = D_{max} \quad (2)$$

$$L_{m2} = D_{max} * \sqrt{G(\theta_A)} * \cos(\theta_A) \quad (3)$$

$$L_{v1} = D_{max} * \sqrt{G(\theta_{v1})} * \sin(\theta_{v1}) \quad (4)$$

$$L_{v2} = D_{max} * \sqrt{G(\theta_{v2})} * \sin(\theta_{v2}) \quad (5)$$

Donde  $L_{m1}$  es la distancia lateral mínima en el eje perpendicular principal del haz,  $L_{m2}$  la distancia mínima en la parte trasera de la antena, y  $L_{v1}$  y  $L_{v2}$  representan las dimensiones verticales del volumen de exposición por encima y por debajo del eje principal del haz, respectivamente. Estas distancias se ilustran en la Fig. 5.



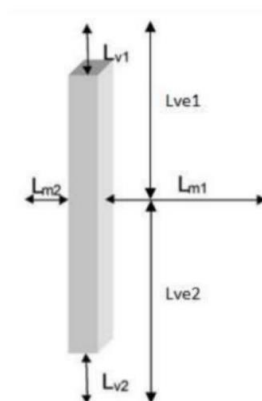


Figura 5. Representación de las distancias del PLL de una antena emisora

Por otro lado, en la Fig. 6, se representa un caso donde la instalación no cumple con el PLL, ya que el área o volumen cubre una azotea transitable. Por tanto, se debería ajustar la inclinación, reubicar equipos o limitar el acceso a determinadas áreas para garantizar la seguridad operativa y el cumplimiento legal.

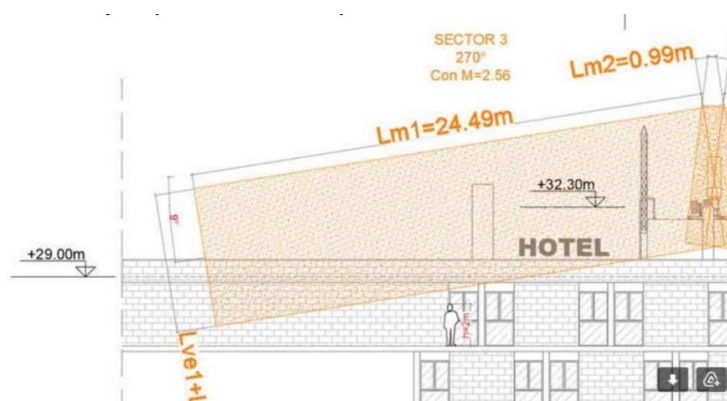


Figura 6. Ejemplo de incumplimiento del PLL: intersección con zona transitable

### 2.1.3.3 Estudios radioeléctricos (EMF)

Por último, los estudios radioeléctricos son informes que evalúan la densidad de potencia generada por los sistemas emisores en un área de 100m de radio, con el objetivo de cumplir con los niveles de cobertura y límites establecidos, especialmente si en la zona existen colegios u hospitales. Esta evaluación es obligatoria en toda instalación de sistemas radiantes, y debe realizarse conforme a la Orden CTE/23/2002 (BOE, 2002) [3].

En resumen, el diseño radio constituye el punto de partida imprescindible para garantizar la viabilidad técnica de un nodo, integrando criterios estructurales, eléctricos y radioeléctricos. No obstante, la cantidad de información generada en esta fase requieren mecanismos automatizados para su tratamiento. En este contexto, el siguiente apartado introduce los fundamentos de inteligencia artificial aplicados a la extracción y consulta eficiente de dicha información técnica.

## 2.2 Fundamentos de Inteligencia Artificial aplicados en extracción y consulta de información

En los últimos años, la inteligencia artificial ha evolucionado a niveles exponenciales, ofreciendo soluciones para automatizar, extraer y consultar información de forma mucho más rápida y efectiva. Esta evolución ha permitido integrar herramientas que transforman documentos complejos y no estructurados en formatos accesibles.

El presente apartado expone los fundamentos y disciplinas clave de la inteligencia artificial que se han aplicado en este trabajo para el desarrollo de un sistema conversacional robusto y adaptado a entornos reales. Para ello, se describe la etapa de extracción de la información y el funcionamiento de OCR, así como las técnicas de procesamiento del lenguaje natural (NLP) imprescindibles, como la arquitectura RAG y los modelos LLM.

### 2.2.1 Extracción de información de documentos PDF

La extracción de información de documentos en formato PDF (*Portable Document Format*) es la primera etapa y comienza con la identificación del tipo de archivo.

Los archivos generados digitalmente contienen texto y elementos estructurados que pueden ser accedidos directamente sin necesidad de análisis visual. Este tipo de documentos es habitual en entornos técnicos e institucionales, y permiten una recuperación precisa del contenido, esencial para su posterior procesamiento.

Para ello, en un entorno de desarrollo con Python como el del sistema propuesto, se utilizan herramientas o librerías que permiten extraer texto, tablas y metadatos. Estas operan sobre los objetos internos del formato PDF y resultan muy útiles para facilitar y automatizar esta primera etapa.

No obstante, cuando el PDF ha sido escaneado o contiene únicamente imágenes, estas herramientas no resultan efectivas, ya que no detectan la información. En esos casos, el texto no está presente como dato accesible, sino como una imagen que debe ser interpretada. Para abordar este tipo de documentos, se recurre al Reconocimiento Óptico de Caracteres (*Optical Character Recognition*, OCR), una técnica de visión por computación que convierte la imagen en texto editable y que se desarrolla con más detalle en el siguiente subapartado.

#### 2.2.1.1 Reconocimiento Óptico de Caracteres (OCR)

Esta tecnología, perteneciente a la rama de visión por computación, permite convertir imágenes de texto en datos digitales editables y analizables. Para ello, el sistema segmenta la imagen en líneas, palabras y caracteres, identifica las formas gráficas mediante técnicas de aprendizaje automático o plantillas, y finalmente las convierte en texto codificado. Es esencial para extraer contenido de documentos escaneados o fotografías que carecen de texto embebido, como planos, formularios o informes técnicos digitalizados.

Este proceso consta de múltiples etapas [4]:

1. **Preprocesamiento de imagen:** mejora la calidad de la imagen mediante técnicas como binarización, eliminación de ruido, corrección de inclinación o aumento de contraste.
2. **Análisis de diseño y segmentación:** detecta las regiones del documento que contienen texto y las divide en líneas, palabras y caracteres individuales para su posterior análisis.
3. **Extracción de características:** convierte los segmentos de imagen en descripciones numéricas o patrones visuales, útiles para la identificación posterior. Puede realizarse con algoritmos clásicos o redes neuronales convolucionales.
4. **Reconocimiento de caracteres:** asigna etiquetas a los patrones extraídos, reconociendo letras, números o símbolos mediante modelos estadísticos, SVM (*Support Vector Machine*), HMM (*Hidden Markov Model*) o redes neuronales entrenadas.
5. **Postprocesamiento y reconstrucción:** corrige errores comunes, ajusta la puntuación y recompone el formato original. A menudo incluye correctores ortográficos o lingüísticos.

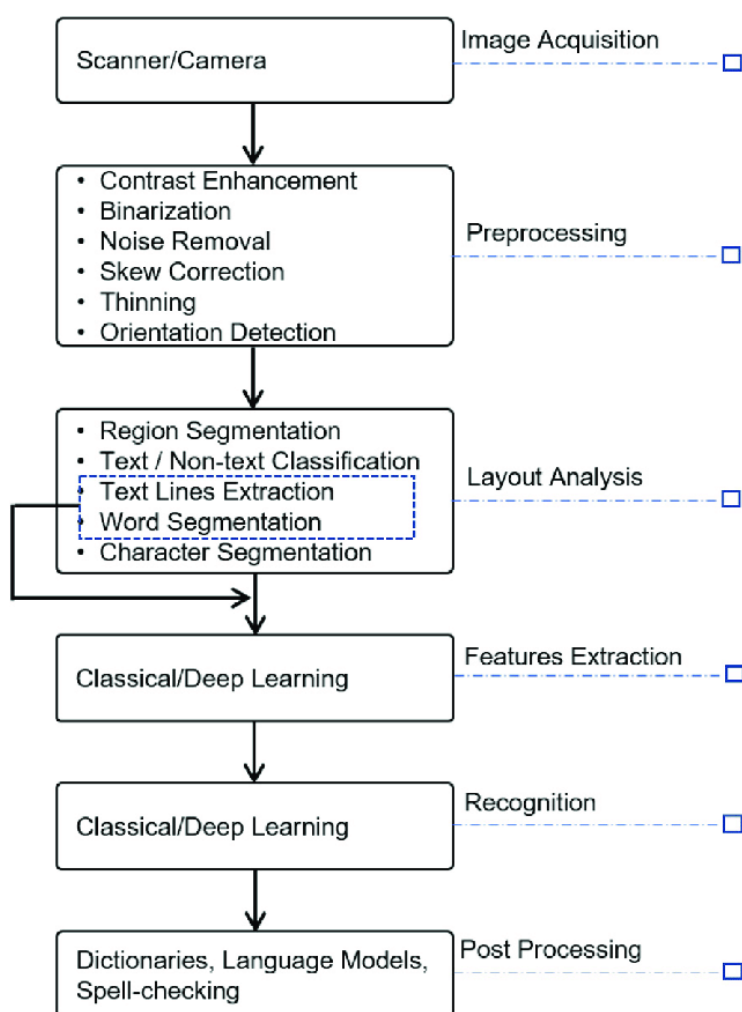


Figura 7. Diagrama de bloques de las etapas principales de un sistema OCR típico

Aunque se trata de un proceso robusto, su precisión depende de la calidad del escaneo, el tipo de fuente y la presencia de ruido visual. Para abordar estas limitaciones, Guan et al. (2025) [5] proponen el *pipeline* PreP-OCR, que introduce una fase previa de restauración de imagen y una

posterior de corrección lingüística con modelos como ByT5, logrando así mejorar notablemente la calidad del texto extraído.

La extracción de la información es la etapa previa al procesamiento del lenguaje natural (NLP, por sus siglas en inglés), que prepara los datos para que sean procesados y transformados.

### 2.2.2 Procesamiento del Lenguaje Natural (NLP)

El Procesamiento de Lenguaje Natural (*Natural Language Processing*, NLP) es la disciplina que combina técnicas de inteligencia artificial, lingüística computacional y estadística para permitir que las máquinas comprendan, generen, manipulen y respondan al lenguaje humano de manera efectiva.

Este campo ha evolucionado desde métodos basados en reglas y patrones, como los n-gramas o los modelos HMM (*Hidden Markov Models*), adecuados para tareas como clasificación de texto, pasando por modelos de aprendizaje profundo como *Word2Vec*, hasta los modelos basados en arquitecturas *Transformer*, como BERT (*Bidirectional Encoder Representations from Transformers*) o GPT (*Generative Pre-trained Transformer*), que utilizan mecanismos de atención para capturar contexto global, elevando el rendimiento y desplazando a los modelos o técnicas tradicionales gracias también a estrategias como el *fine-tuning* o *prompting*, convirtiéndose en el estándar actual en el desarrollo de sistemas NLP [7].

El NLP abarca múltiples ámbitos, como la traducción automática, en análisis del sentimiento, el reconocimiento de entidades, el etiquetado gramatical y la recuperación y clasificación de la información. El presente trabajo se centra en este último, cuyas etapas o sub-ámbitos que se han seguido en la elaboración del sistema son:

1. Normalización y tokenización del texto extraído
2. Representación semántica mediante *embeddings*
3. Uso de modelos de lenguaje de gran tamaño (LLM) para la generación de respuestas
4. Diseño de una arquitectura RAG e indexación vectorial para la recuperación de la información.

Estas etapas se desarrollarán en los siguientes subapartados.

#### 2.2.2.1 Normalización y tokenización

La normalización de texto consiste en transformar o convertir el texto extraído a una forma estándar u homogeneizada que facilite el procesamiento posterior. Dentro de esta fase, el tokenizado es el paso clave, ya que consiste en separar o segmentar la información en unidades mínimas (*tokens*) que pueden ser oraciones, palabras, o incluso caracteres individuales.

Entre las técnicas de normalización y tokenización aplicadas, destacan la conversión a minúsculas, la eliminación de caracteres no alfabéticos u espacios y la segmentación en oraciones o frases. Estas transformaciones reducen la variabilidad léxica y minimizan el ruido, mejorando así la calidad de las etapas posteriores como la vectorización semántica.

Una herramienta esencial en este proceso es el uso de expresiones regulares (*regex*), ya que su uso facilita la limpieza y la segmentación contextual. Son ampliamente utilizadas para localizar

patrones específicos dentro del texto e identificar elementos estructurados como fechas, coordenadas geográficas o códigos identificativos.

En resumen, la etapa de normalización y tokenización permite transformar el texto extraído a una forma limpia y estructurada. Esta representación se convierte en el punto de partida para la generación de *embeddings*, que constituyen la base del sistema de recuperación.

#### 2.2.2.2 Representaciones vectoriales y *embeddings*

La representación vectorial consiste en transformar unidades de texto, como palabras, oraciones o párrafos, en vectores de números reales en un espacio de alta dimensión. En este espacio, la distancia o similitud entre dos vectores refleja la relación semántica entre sus respectivos textos. Es decir, textos con significado similar, generan vectores cercanos entre sí, mientras que textos diferentes están más alejados.

Dentro de estas representaciones, los *embeddings* destacan como vectores de baja dimensión que han sido aprendidos a partir de técnicas de entrenamiento supervisado o no supervisado. A diferencia de representaciones manuales, los *embeddings* capturan patrones semánticos complejos y permiten modelar el significado contextual de las palabras de forma eficiente.

Los tipos de vectores o *embeddings* más habituales son:

- *Embeddings* estáticos: se entrenan una sola vez y generan vectores por cada palabra. Entre los más conocidos se encuentran *Word2Vec* y *GloVe*.
- *Embeddings* contextuales: el vector se calcula tras cada iteración y varía según las palabras vecinas. Estos modelos mejoran a los *embeddings* estáticos al permitir que una misma vector o palabra tenga distintos significados según el contexto dado. Entre los más conocidos se encuentran los modelos BERT y RoBERTa.

Estos modelos, conocidos como *encoders*, transforman oraciones en representaciones vectoriales. Uno de los enfoques más representativos es Sentence-BERT (SBERT), que permite codificar pares de oraciones simultáneamente y optimizar el modelo para que produzca vectores cercanos para frases similares. El resultado es una representación que puede ser comparada eficientemente utilizando métricas como la distancia coseno, ampliamente utilizada en tareas de recuperación semántica, o la distancia euclídea [9].

En la Fig. 8 se puede ver una representación visual del funcionamiento de los *embeddings* en un espacio vectorial [10]. Cada subgráfico ilustra cómo relaciones lingüísticas se reflejan como relaciones geométricas entre vectores en ese espacio.

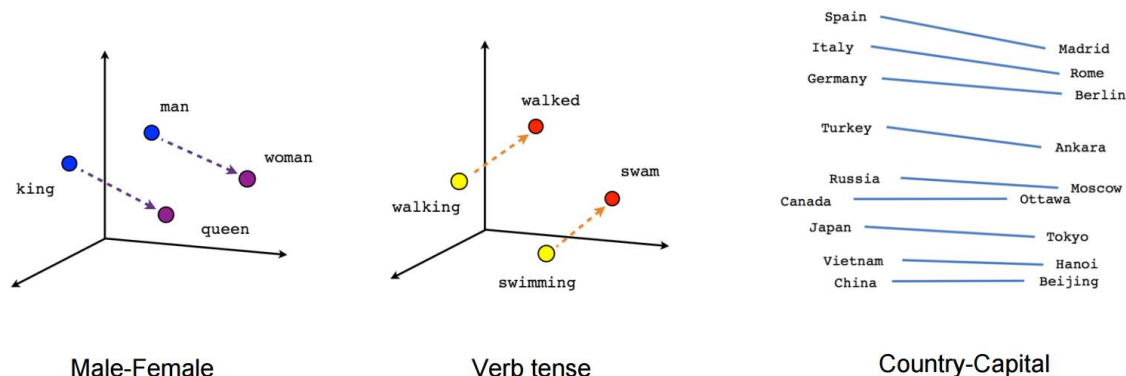


Figura 8. Ejemplo del funcionamiento de embeddings semánticos en un espacio vectorial.

Los *embeddings* se pueden utilizar en tareas como búsqueda semántica o clusterización y clasificación. Además, son muy útiles para alimentar modelos LLM proporcionando contexto relevante, especialmente en arquitecturas de tipo RAG, como la empleada en el presente trabajo.

### 2.2.2.3 Modelos de lenguaje de gran tamaño (LLM)

Los modelos de lenguaje de gran tamaño (LLMs) han supuesto un gran avance en el campo del procesamiento de lenguaje natural. Gracias a ellos, ahora es posible realizar tareas complejas como generar texto, responder preguntas o resumir documentos sin necesidad de entrenar el modelo específicamente para cada tarea. Están basados en redes neuronales muy potentes que han sido entrenadas con enormes cantidades de texto de diferentes temas e idiomas, lo que les permite entender el contexto y generar respuestas con bastante precisión.

El desarrollo de estos modelos ha sido progresivo y ha ido mejorando con el tiempo. Uno de los primeros avances fue BERT (*Bidirectional Encoder Representations from Transformers*), que introdujo representaciones bidireccionales contextuales mediante una arquitectura basada exclusivamente en *encoders* de tipo *Transformer* [11].

Más adelante, modelos como GPT-3 popularizaron el enfoque autorregresivo con *decoders* unidireccionales, demostrando capacidades excepcionales de aprendizaje en pocas muestras (*few-shot learning*) y sin necesidad de ajuste específico (*zero-shot*) [12]. Más recientemente, GPT-4 ha ampliado este paradigma incorporando entrada multimodal (texto e imágenes) y mejoras en razonamiento, alineamiento con instrucciones y seguridad conversacional, consolidando el papel de los LLM como sistemas versátiles aplicables tanto a tareas técnicas como a consultas generales.

Los LLM siguen una estrategia en dos fases para adquirir y adaptar sus conocimientos: el preentrenamiento y el ajuste fino (*fine-tuning*).

- **Preentrenamiento:** es la fase donde el modelo se entrena con enormes volúmenes de texto no etiquetado para aprender patrones estadísticos del lenguaje. A este conjunto se le conoce como parámetros y dependiendo de la arquitectura, este entrenamiento puede centrarse en: MLM (*Masked Language Modeling*), donde ciertas palabras se ocultan y el modelo debe predecirlas a partir del contexto, o CLM (*Casual Language modeling*), donde se predice la siguiente palabra dado el historial previo, permitiendo generar texto de manera coherente.



- **Ajuste fino (*fine-tuning*):** fase donde el modelo se adapta a tareas específicas, empleando conjuntos de datos anotados que guían su comportamiento hacia objetivos concretos como clasificación, resumen, traducción o respuesta a preguntas. En los modelos más recientes, esta fase se refuerza con técnicas como *instruct tuning*, RLHF (*Reinforcement Learning with Human Feedback*) o DPO (*Direct Preference Optimization*) que ajustan el modelo para alinearlo con instrucciones humanas, mejorar su utilidad y reducir respuestas erróneas o sesgadas.

#### 2.2.2.3.1 Arquitectura Transformer

La arquitectura *Transformer* constituye el núcleo de los modelos de lenguaje modernos de hoy en día, ya que permite procesar secuencias de texto de forma paralela y eficiente.

Se basa en la idea de que, para entender una secuencia de texto, no es necesario procesar palabra por palabra en orden. En lugar de eso, cada palabra puede analizar al mismo tiempo el resto de la frase y decidir cuáles son más relevantes para su significado. Este enfoque se consigue mediante un mecanismo llamado autoatención (*self-attention*), que calcula cuánto influye cada palabra en las demás dentro de una misma secuencia. Gracias a esta capacidad, el modelo puede captar relaciones contextuales complejas incluso entre términos que están muy alejados, lo que resulta especialmente útil en frases técnicas o muy largas [13].

Esta arquitectura está dividida en dos bloques principales:

- ***Encoder*:** recibe la secuencia de entrada, que puede ser una pregunta o un documento, y la transforma en vectores numéricos mediante *embeddings* (ya comentados en el anterior apartado). Estos añaden información posicional para reflejar el orden de las palabras, y la procesan a través de varias capas que combinan mecanismos de atención y redes neuronales. El resultado son representaciones del texto original, que recogen relaciones semánticas incluso entre elementos lejanos de la frase.
- ***Decoder*:** genera la respuesta palabra a palabra. Aplica en primer lugar *self-attention* para no acceder a palabras futuras aún no generadas, y luego emplea un mecanismo de atención cruzada que le permite consultar las representaciones generadas por el *encoder*. Esto le permite tener en cuenta tanto el contenido original como lo que ha generado previamente, produciendo una salida coherente y contextualizada.

Esta arquitectura modular ha superado en eficiencia y rendimiento a modelos anteriores como las redes neuronales recurrentes (*Recurrent Neural Network*, RNN), consolidándose como base de los modelos más avanzados actualmente.

La arquitectura *Transformer* explicada se representa en la Fig. 9, donde a la izquierda se representa el *encoder* y a la derecha el *decoder*.

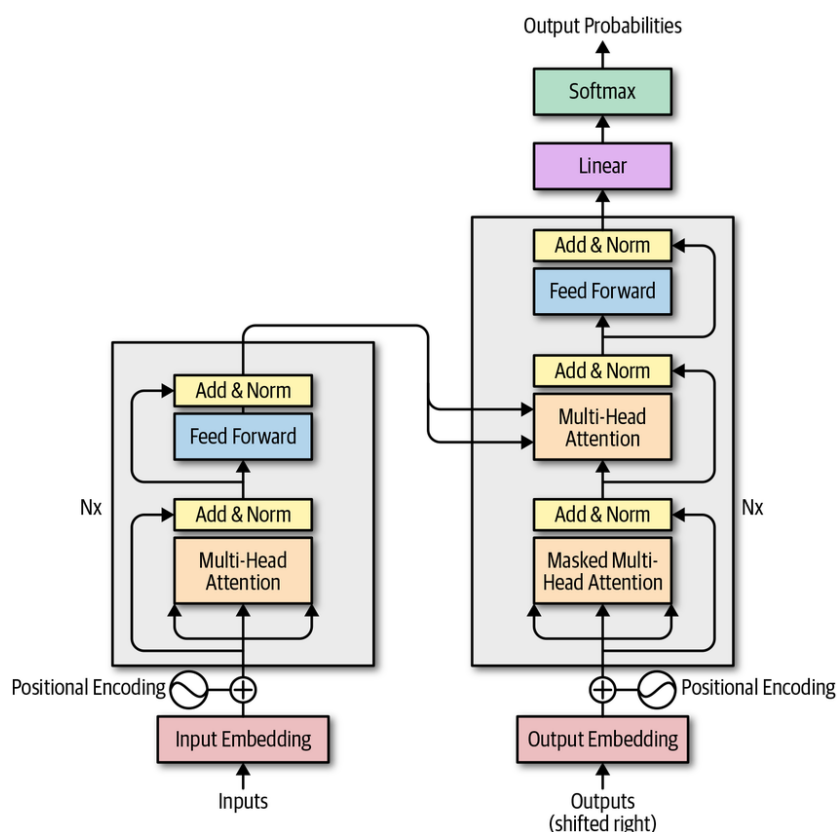


Figura 9. Arquitectura Transformer

#### 2.2.2.3.2 Tipos de modelos y cuantización

Hoy en día existen numerosos modelos de lenguaje en el mercado, aunque los más conocidos son sin duda los modelos GPT, desarrollados por OpenAI. Estos modelos, de carácter propietario, destacan por su precisión y versatilidad, pero presentan ciertas limitaciones para su uso en entornos locales, ya que requieren conexión constante a servicios en la nube y no permiten acceso completo al modelo subyacente ni a sus pesos, lo que restringe su personalización y despliegue privado.

Frente a este enfoque cerrado, ha surgido en paralelo un ecosistema de modelos *open-source*, que ofrecen un rendimiento competitivo con la ventaja de poder ser ejecutados y adaptados localmente. Estos modelos permiten acceder a su arquitectura, pesos y documentación, lo que los convierte en una opción especialmente interesante para proyectos que buscan mayor control, privacidad y eficiencia en costes de infraestructura.

En este trabajo se utilizarán algunos de los modelos *open-source* más relevantes y recientes, diseñados para funcionar en entornos locales sin depender de servicios externos, como Llama, Mistral o Phi-3, entre otros.

Un aspecto clave para poder ejecutar modelos LLM de forma eficiente en entornos locales es la cuantización. Esta técnica permite reducir significativamente el uso de memoria y el tamaño en disco de los modelos, sin que ello afecte de forma relevante a su rendimiento. Básicamente, consiste en representar los pesos del modelo, que normalmente están en formatos de alta precisión



(como float32), mediante formatos de menor precisión como int8 o float16. Esto no solo reduce el consumo de memoria, sino que también permite aprovechar instrucciones optimizadas a nivel de hardware, lo que acelera la inferencia en GPU, y, aunque puede implicar una leve pérdida de precisión, en la práctica su impacto suele ser mínimo si se utilizan técnicas como la cuantización calibrada postentrenamiento o la cuantización consciente durante el entrenamiento (*quantization-aware training*).

Una de las soluciones más utilizadas actualmente es QLoRA (*Quantized Low-Rank Adaptation*), que combina cuantización con técnicas de adaptación de bajo rango para permitir el ajuste de modelos grandes con un coste computacional muy bajo. Gracias a esta técnica, es posible afinar modelos LLM incluso en hardware de consumo, lo que ha sido clave para probar variantes optimizadas en este trabajo [14].

#### 2.2.2.4 Arquitectura RAG

La arquitectura RAG (*Retrieval Augmented Generation*) o generación aumentada por recuperación, surge como respuesta a la incapacidad para acceder a información externa en tiempo real de los modelos LLM. Estos pueden generar respuestas a priori correctas pero erróneas, conocidas como alucinaciones (*hallucinations*), especialmente en tareas que requieren conocimiento técnico preciso o actualizado.

RAG, introducida por un equipo de investigadores de IA de Facebook en 2020 [15], integra un sistema de recuperación documental que localiza fragmentos relevantes en un conjunto o corpus, los cuales se proporcionan como contexto al modelo generativo, mejorando así la exactitud y trazabilidad de las respuestas.

Revisiones recientes destacan la rápida consolidación de RAG como solución de referencia en tareas de respuesta a preguntas (QA), asistencia técnica y generación documentada, en dominios como medicina, derecho o ingeniería. Su ventaja principal se basa en combinar lo mejor de los sistemas de recuperación y la fluidez de los modelos generativos, permitiendo construir sistemas conversacionales que no solo generan lenguaje natural, sino que lo hacen basándose en evidencia verificable [16].

La arquitectura RAG se compone fundamentalmente de dos módulos interconectados: un motor de recuperación de información y un modelo generativo. En la primera etapa, el sistema transforma la consulta del usuario en un vector semántico mediante técnicas de *embedding* y lo compara con una base de datos vectorial construida a partir de una colección de documentos técnicos. Los fragmentos más relevantes se seleccionan en función de su similitud contextual y se utilizan como contexto para la etapa siguiente, donde un modelo LLM genera la respuesta final condicionada a la información recuperada.

Además, el número de tokens disponibles en el modelo LLM influye directamente en la cantidad de contexto que se puede aportar. Por ello, los documentos se suelen dividir en *chunks* de entre 100 y 500 tokens, para maximizar la cobertura y minimizar el coste computacional, especialmente en entornos locales o limitados.

#### 2.2.2.4.1 Indexación semántica

La recuperación de la información se da gracias a la indexación semántica de los *embeddings* ya creados. Tras generar estos, es necesario almacenarlos e indexarlos de forma eficiente para permitir búsquedas rápidas por similitud. Este proceso es esencial en cualquier arquitectura RAG y se realiza mediante motores de búsqueda vectorial como FAISS (*Facebook AI Similarity Search*), ampliamente utilizado en este tipo de sistemas por su capacidad de escalar a millones de vectores.

FAISS organiza los *embeddings* en estructuras optimizadas que permiten recuperar los *top-k* fragmentos más cercanos a una consulta, utilizando métricas como la distancia coseno o la distancia euclídea. Existen varios algoritmos disponibles, entre ellos:

- **Flat:** búsqueda exacta en todo el espacio vectorial.
- **IVF (*Inverted File Index*):** particiona el espacio en clústeres y busca solo en los más cercanos, acelerando la recuperación.
- **HNSW (*Hierarchical Navigable Small World*):** estructura basada en grafos eficiente para grandes escalas y consultas de baja latencia.

En el sistema desarrollado, se utiliza un índice optimizado con FAISS para recuperar los fragmentos más relevantes, que luego son ponderados en función de su similitud antes de alimentar al modelo generativo.

La Fig. 10 ilustra este proceso, donde se puede ver que el *embedding* de entrada se compara con los almacenados, posteriormente se obtienen los más similares y finalmente se asignan unos pesos que reflejan su relevancia semántica [17].

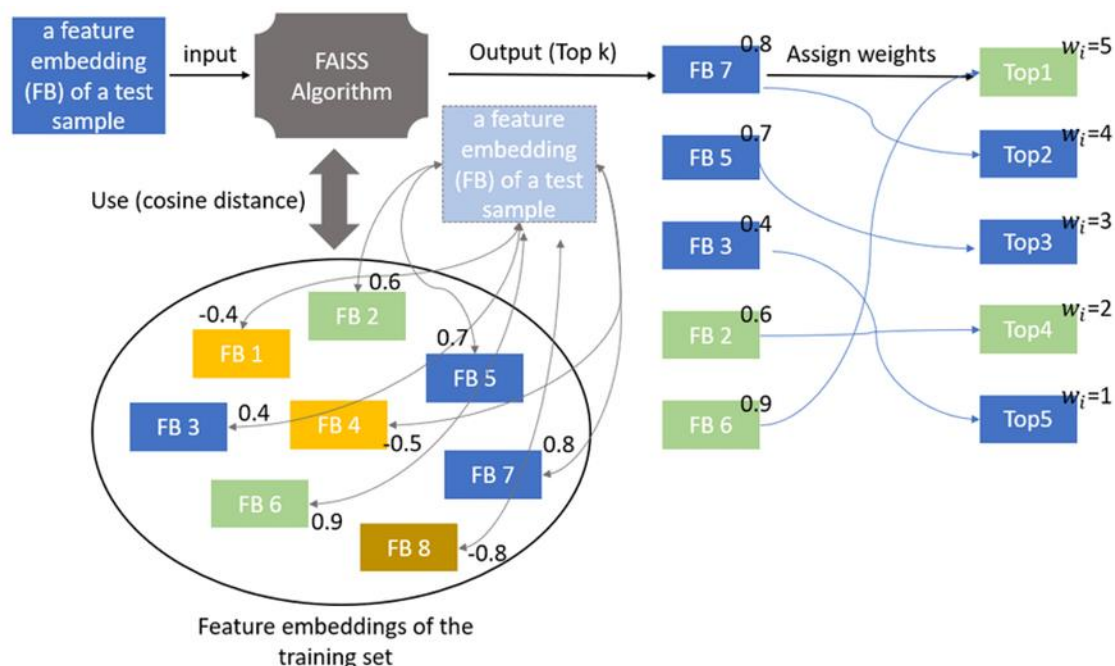


Figura 10. Esquema de recuperación semántica con FAISS

Es importante destacar que, aunque los modelos *Transformer* ya usan *encoders* internamente para permitir búsquedas eficientes y reutilizables, se necesita almacenar e indexar *embeddings* explícitos. Estos se calculan una sola vez y se indexan con FAISS, sin estos se tendría que reanalizar todos los textos cada vez que se hace una consulta, aumentando el coste computacional y reduciendo la eficiencia.

Este enfoque hace posible recuperar contenido técnicamente relevante incluso cuando no comparte términos literales con la consulta, y constituye la base funcional de toda arquitectura RAG efectiva.

#### 2.2.2.4.2 Variantes de arquitectura RAG

A lo largo de los últimos años, la arquitectura RAG ha evolucionado en múltiples direcciones para adaptarse a distintos dominios y niveles de complejidad. A continuación, se resumen las principales categorías y tendencias descritas recientemente [18]:

- **RAG Naive:** enfoque básico propuesto donde el modelo generativo consume directamente los documentos más similares recuperados por un motor de búsqueda vectorial. Es sencillo de implementar y adecuado para dominios con corpus moderados.
- **RAG modular:** introduce componentes intermedios como *re-rankers*, fusión de contexto o filtros de calidad antes de alimentar al modelo generativo. Mejora la precisión y relevancia en sistemas con corpus más extensos o respuestas críticas.
- **RAG End-To-End:** recuperación y generación se entrenan de forma conjunta, optimizando los *embeddings* y la generación para tareas específicas. Esta aproximación requiere mayor capacidad computacional, pero proporciona respuestas más coherentes y contextuales.

Por otro lado, investigaciones recientes destacan nuevas direcciones en RAG, como el uso de *feedback* del usuario, el uso de datos estructurados y fuentes multimodales, y su aplicación en tareas más allá del QA.

En conjunto, la arquitectura RAG representa una solución robusta para integrar modelos de lenguaje con documentación técnica compleja como la utilizada en el presente trabajo, permitiendo respuestas precisas, contextualizadas y trazables.

## Capítulo 3. Fundamentos técnicos

En este capítulo se analizan los condicionantes que han determinado las decisiones de diseño, tales como la naturaleza de los documentos procesados, las restricciones operativas y los requisitos legales. Seguidamente, se explica el contenido de los documentos CAP y finalmente se detallan las herramientas tecnológicas empleadas para su implementación.

### 3.1 Entorno y condicionantes técnicos

El desarrollo del sistema de extracción y consulta se ha desarrollado en el lenguaje de programación Python (versión 3.12), utilizando Visual Studio Code como entorno de trabajo. Sin embargo, el sistema enfrenta a un conjunto de condicionantes que determinan tanto su viabilidad como sus especificaciones funcionales.

En primer lugar, los documentos de entrada CAP presentan una estructura heterogénea y formatos no estandarizados, lo que requiere la utilización de técnicas avanzadas de reconocimiento óptico de caracteres (OCR) y algoritmos o librerías para extraer la información relevante.

A pesar de esta diversidad, se ha identificado que la mayoría de los documentos siguen dos formatos estructurales predominantes, y es sobre estos dos tipos sobre los que se ha desarrollado el sistema, por lo que, cualquier documento que no se ajuste a estos formatos queda fuera del alcance del sistema, constituyendo una restricción explícita en el diseño del módulo de extracción.

Por otro lado, la naturaleza sensible de la información contenida en los CAP, que incluye datos identificativos y localización de nodos, exige la implementación de mecanismos de anonimización. Este requisito deriva de la necesidad de proteger los datos sensibles del cliente y cumplir el Reglamento General de Protección de Datos (RGPD).

Para ello, el sistema incorpora técnicas de pseudonimización al sustituir elementos identificativos como nombres o dirección del nodo por códigos neutros y aleatorios, así como el uso de expresiones regulares (*regex*) para localizar y anonimizar información concreta, como el nombre del cliente o proveedores externos.

Por último, debido tanto a razones de confidencialidad como a limitaciones presupuestarias inherentes a un proyecto académico, las pruebas y desarrollo del sistema se han realizado íntegramente en un entorno local, sin acceso a servicios en la nube ni aceleración por GPU. El entorno de ejecución se ha limitado a un equipo con procesador Intel Core i5-10310U a 1.70 GHz, 16 GB de memoria RAM y una GPU integrada Intel UHD Graphics con 128 MB de memoria compartida.

Esta restricción evita la transferencia de datos técnicos fuera del dominio controlado y garantiza que toda la información sensible permanezca bajo control del usuario. A nivel técnico, esta condición ha condicionado la elección de los modelos de lenguaje y de las técnicas de recuperación semántica, descartando el uso de soluciones comerciales basadas en API externas, como la de Open AI, y optando por alternativas ligeras y de código abierto.

En concreto, se han empleado modelos LLM ejecutables localmente a través de Ollama, una herramienta que permite ejecutar versiones optimizadas de modelos directamente en CPU o GPU sin conexión a Internet. Mientras que, para la recuperación semántica se utilizan *embeddings*, que son representaciones vectoriales del contenido textual, y el motor de búsqueda FAISS, que permite realizar búsquedas eficientes de similitud entre estos vectores en grandes volúmenes de datos.

En este contexto, resulta necesario definir el contenido de los documentos CAP, los cuales son la fuente de información a consultar del sistema propuesto.

### 3.2 Estructura y contenido de los documentos CAP

El CAP es un documento técnico clave en el proceso de diseño, despliegue y aceptación de instalaciones de red móvil. Su función es formalizar la validación de una propuesta técnica por parte del operador para un emplazamiento específico. Esta puede ser una ampliación del nodo, donde se añade un nuevo sector, o la inclusión de una nueva tecnología al *site*. Este documento contiene toda la información necesaria para coordinar su ejecución y posterior integración en la red.

En la práctica los documentos CAP presentan una organización bastante regular, que suele incluir las siguientes secciones [19]:

- **Información básica del emplazamiento:** datos identificativos del nodo, cliente, dirección y coordenadas geográficas.
- **Planos de planta y alzado:** representación gráfica del estado actual y modificado de la distribución horizontal y vertical de la estructura del emplazamiento
- **Cuadro de trabajos a realizar por operador:** tareas asignadas al operador y propietario del *site*. Estas suelen representarse en cuadros de texto por colores, según el operador o propietario.
- **Detalle del sistema radiante:** características de los equipos y distribución del sistema radiante.
- **Detalle de conexiones en antena:** distribución del cableado, puertos y conexiones por antena o sector.
- **Tabla de antenas y consumos:** información sobre los modelos de antena instalados, tecnologías desplegadas, orientaciones, tilts y consumo energético.

Si bien no existe una plantilla única estandarizada, se han identificado dos modelos o plantillas predominantes entre los documentos CAP utilizados en el proyecto, cuya principal diferencia se encuentra en el bloque de información del emplazamiento.

- **Modelo A:** en esta primera plantilla, la información básica del emplazamiento está distribuida en bloques horizontales en la parte inferior del documento, tal como se puede observar en la Fig. 11.

|               |                     |                                                                                   |             |                    |              |
|---------------|---------------------|-----------------------------------------------------------------------------------|-------------|--------------------|--------------|
| COORDENADAS   |                     | PROYECTO DE INSTALACIÓN PARA ESTACIÓN DE EQUIPOS DE TELEFONÍA MÓVIL               |             |                    |              |
| DATUM ETRGR   | DIRECCIÓN           |  | OPERADOR -- | PLANO DE SITUACIÓN | 01.<br>DE 10 |
| MUSO UTM 19   | MUNICIPIO PROVINCIA |                                                                                   |             |                    |              |
| COTA<br>650 m | DISEÑADO DIBUJADO   |                                                                                   |             |                    |              |

Figura 11. Esquema Modelo A CAP

Dentro de este modelo podemos observar dos variantes:

- Modelo A1: las coordenadas están situadas en la parte inferior izquierda, alineadas horizontalmente con otros datos técnicos, como el de la Fig. 11.
- Modelo A2: las coordenadas están desplazadas hacia la parte inferior derecha, colocadas más arriba del bloque principal de información del emplazamiento.
- **Modelo B**: en este otro modelo, la principal diferencia es que la información del emplazamiento está ubicada en la parte inferior derecha.

|                                                                                     |                                                                                                            |                  |                       |           |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------|-----------------------|-----------|
|                                                                                     |                                                                                                            | AMPLIACIÓN RADIO |                       | EJECUCIÓN |
|                                                                                     |                                                                                                            | DESCRIPCIÓN      |                       | ESTADO    |
|  | ESTACIÓN BASE:                                                                                             | CODIGO           |                       |           |
|                                                                                     | DIRECCIÓN:                                                                                                 | CODIGO           |                       |           |
|                                                                                     | LOCALIDAD:                                                                                                 |                  |                       |           |
|                                                                                     | PROVINCIA:                                                                                                 |                  |                       |           |
| EL INGENIERO:                                                                       | <b>AMPLIACIÓN RADIO - PLAN 3500</b><br>PROYECTO DE INSTALACION PARA ESTACIÓN DE EQUIPOS DE TELEFONÍA MÓVIL |                  | PLANO N°<br><b>01</b> |           |
| COLEGIADO N°                                                                        | TÍTULO PLANO:<br><b>SITUACION Y EMPLAZAMIENTO</b>                                                          |                  |                       |           |
| FECHA:                                                                              | REVISIÓN:                                                                                                  | ESCALA:          | S/E                   |           |

Figura 12. Esquema modelo B CAP

Dentro de este modelo, también observamos dos variantes:

- Modelo B1: las coordenadas están bien organizadas y claramente visibles en el centro de la primera página.
- Modelo B2: las coordenadas están ubicadas más hacia la izquierda, en formato vertical y en una zona media o inferior. A diferencia del modelo B1, este suele presentar una estructura más desorganizada.

El sistema debe ser capaz de localizar, interpretar y extraer los bloques de información, tanto en formato estructurado como semiestructurado. No obstante, la presencia de variantes en el diseño de los CAP exige el uso de algoritmos de segmentación flexibles y, en muchos casos, la aplicación de OCR para convertir imágenes o zonas escaneadas en texto procesable. Este paso resulta especialmente necesario en documentos generados como imagen embebida en PDF, una situación habitual en flujos documentales no digitalizados.

Finalmente, cabe destacar que el sistema desarrollado se ha diseñado específicamente para procesar los documentos que se ajustan a dos formatos estructurales predominantes identificados durante el análisis. Cualquier documento que no se corresponda con estas variantes queda fuera del alcance funcional actual, lo que constituye una limitación explícita en el diseño del módulo de extracción.

### 3.3 Tecnologías y herramientas utilizadas

Para el desarrollo del sistema se han empleado herramientas de código abierto y librerías de Python. Estas permiten abordar todo el ciclo de procesamiento técnico, desde la extracción de información hasta la consulta mediante la interfaz conversacional.

#### 3.3.1 Librerías utilizadas

A continuación, se enumeran las principales librerías utilizadas y su función dentro del sistema:

- **pandas**: usada para el procesamiento de datos tabulares, especialmente en la normalización de las tablas de consumos y hardware extraídas.
- **numpy**: utilizada para la manipulación eficiente de arrays y el cálculo de similitudes en el motor de recuperación semántica.
- **fitz**: empleada para la lectura estructurada de documentos PDF y la extracción de zonas específicas mediante coordenadas.
- **pdfplumber**: utilizada para extraer tablas estructuradas directamente de las páginas PDF cuando el OCR no es necesario.
- **pytesseract**: motor OCR basado en Tesseract, utilizado para extraer texto desde imágenes rasterizadas obtenidas de zonas específicas de documentos PDF, resulta especialmente útil en campos con formatos no estructurados. Para su funcionamiento, ha sido necesario instalar el motor Tesseract desde el repositorio mantenido por la Universidad de Mannheim, el cual proporciona versiones optimizadas para sistemas Windows [20].
- **pdf2image**: librería empleada para convertir páginas PDF en imágenes, paso intermedio fundamental para la posterior detección OCR por zonas o colores. Ha sido esencial para identificar visualmente bloques de información relevantes, como las secciones de trabajos a realizar. Esta librería requiere la instalación previa de Poppler para Windows, disponible en el repositorio de GitHub de oschwartz10612 [21].
- **re (regex)**: utilizada intensivamente en la limpieza y extracción de texto.
- **sentence-transformers**: librería NLP utilizada para generar representaciones vectoriales semánticas de textos y realizar búsqueda contextualizada.
- **faiss**: motor de búsqueda vectorial de alta eficiencia, utilizado para indexar y recuperar fragmentos relevantes previamente almacenados.
- **Streamlit**: framework para desarrollar la interfaz gráfica conversacional de consulta, permitiendo interacción sencilla entre el usuario y el sistema.



- **FastAPI:** framework web ligero para exponer las funcionalidades de búsqueda y recuperación en forma de API RESTful.

### 3.3.1 Plataforma y modelos LLM

Por otro lado, para la ejecución local de los modelos LLM, se ha empleado la plataforma Ollama, que permite cargar y ejecutar modelos de código abierto de forma eficiente en estaciones de trabajo convencionales, facilitando su integración con sistemas Python a través de API locales.

En cuanto a los modelos LLM que se han evaluado o probado en el sistema, teniendo en cuenta las limitaciones computacionales, han sido los siguientes:

- **TinyLlama:** modelo ultraligero entrenado con 1.1 mil millones de parámetros. Admite hasta 2048 tokens de contexto y ha sido diseñado para tareas conversacionales básicas. Su reducido tamaño lo convierte en una excelente opción para pruebas rápidas y entornos con recursos limitados, aunque con menor capacidad de razonamiento técnico complejo.
- **Phi3:** modelo ligero desarrollado por Microsoft con 3.8 mil millones de parámetros. Admite hasta 4096 tokens de contexto y se centra en la calidad de instrucciones y comprensión de tareas específicas, ofreciendo un buen equilibrio entre rendimiento y eficiencia.
- **Mistral:** modelo intermedio entrenado para instrucciones con 7 mil millones de parámetros. Se ha elegido por su precisión en respuestas breves y su capacidad para mantener coherencia técnica en múltiples turnos. Presenta un rendimiento muy competitivo para tareas técnicas con contexto limitado y admite hasta 32 mil tokens.
- **Llama3:** modelo de tamaño medio-alto desarrollado por Meta con 8 mil millones de parámetros, entrenado con datos de alta calidad y ajustado para instrucciones. Admite hasta 8 mil tokens de contexto y ofrece resultados consistentes y precisos en dominios técnicos.
- **Zephyr:** modelo orientado a diálogos naturales, entrenado con técnicas de alignment como RLHF (*Reinforcement Learning with Human Feedback*) sobre la base del modelo Mistral. Su fortaleza reside en la naturalidad de las respuestas, lo que mejora la experiencia de usuario en el sistema conversacional.

En la Tabla 1 se representan las características principales de cada modelo.

| Modelo      | Tamaño | Parámetros | Contexto | Tipo     | Cuantización |
|-------------|--------|------------|----------|----------|--------------|
| tinylama    | 638 MB | 1.1B       | 2K       | Chat     | Q4           |
| phi3-mini   | 2.2 GB | 3.8B       | 4K       | Instruct | Q4           |
| mistral     | 4.1 GB | 7B         | 32K      | Instruct | Q4           |
| llama3      | 4.7 GB | 8B         | 8K       | Instruct | Q4           |
| zephyr-beta | 4.1 GB | 7B         | 32K      | Chat     | Q4           |

Tabla 1. Modelos LLM



Dónde el contexto es la cantidad máxima de tokens o unidades lingüísticas de entrada que puede recibir el modelo, y los parámetros son la cantidad de datos con los que ha sido entrenado el modelo.

Como se puede observar, todos los modelos han sido cuantizados a nivel Q4 para permitir una ejecución eficiente en CPU sin sacrificar significativamente la calidad de respuesta, asegurando así un equilibrio entre rendimiento y viabilidad técnica en entornos locales.

La precisión y rendimiento de estos modelos serán evaluados posteriormente, donde se podrán ver las diferencias entre ellos, especialmente entre los modelos más pequeños, como tinyllama y los modelos de mayor tamaño como llama3.

## Capítulo 4. Diseño del sistema

El presente capítulo explica cómo se ha diseñado el sistema de extracción y consulta conversacional. Este se ha estructurado por bloques modulares o *pipelines*, es decir, cadenas de procesos donde la salida de cada etapa sirve como entrada para la siguiente.

El sistema tiene como objetivo extraer de forma automática la información de los informes CAP en PDF, estructurarla en formato normalizado y anonimizado, para posteriormente vectorizarla y procesarla siguiendo el proceso RAG, el cual genera una respuesta a través del modelo LLM en base al contexto recuperado por indexación según la pregunta del usuario.

En primer lugar, se describe la arquitectura general del sistema y posteriormente se detallan los principales bloques funcionales: ETL, vectorización semántica, RAG e interfaz de usuario.

### 4.1 Arquitectura general del sistema

El sistema propuesto sigue una arquitectura de bloques independientes que se comunican de forma secuencial. Esta estructura permite adaptar o modificar cada módulo de forma independiente sin afectar al resto, favoreciendo la mantenibilidad y escalabilidad del sistema. La Fig. 13 muestra una visión global de esta arquitectura mediante un diagrama de bloques funcionales, donde se distinguen claramente los principales módulos del sistema.

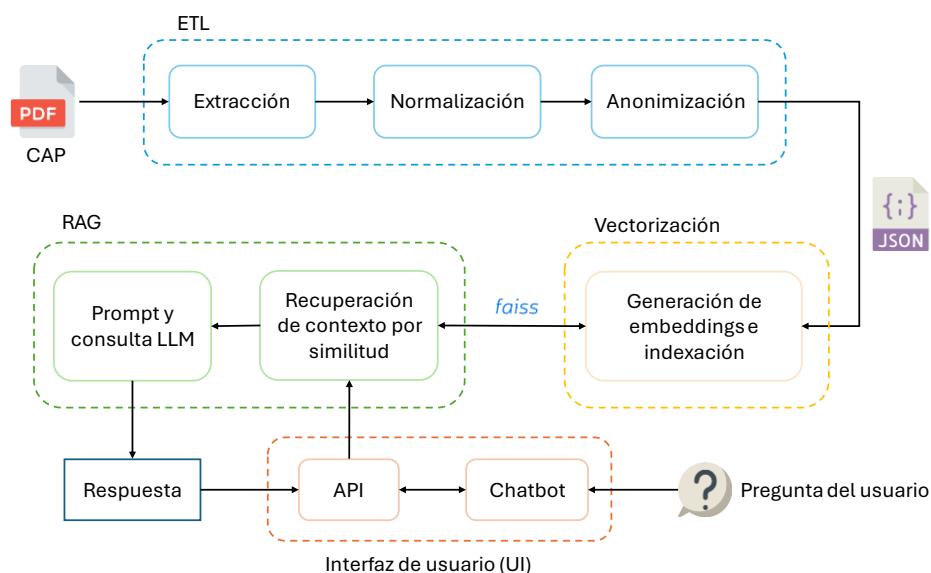


Figura 13. Arquitectura del sistema

Como se puede observar, el flujo general del sistema es progresivo. En primer lugar, el módulo de extracción o ETL (*Extract, Transform, Load*), se encarga de procesar el documento PDF y extraer de forma automática la información técnica mediante técnicas de OCR o funciones personalizadas. Esta información se normaliza y anonimiza, y se almacena en un fichero JSON, que contiene toda la información estructurada y es la base de información del siguiente bloque.

A continuación, el módulo de vectorización semántica toma ese JSON y lo fragmenta en unidades informativas, representadas como vectores de *embeddings*, cuyo proceso se ha explicado en anteriores apartados. Estas representaciones se indexan utilizando una base vectorial para permitir su recuperación por similitud en el módulo RAG.

El módulo RAG gestiona las consultas del usuario. Ante una pregunta en lenguaje natural, el sistema clasifica e interpreta la intención de la consulta, recupera los fragmentos más relevantes a partir del índice vectorial y construye un contexto que se emplea para generar la respuesta mediante un modelo LLM local.

Por último, la interfaz de usuario permite introducir preguntas y visualizar respuestas a través de una chatbot conversacional. Es sencilla y se comunica con el backend del sistema vía API.

## 4.2 Módulo de extracción

El módulo de extracción o ETL se encarga de extraer, transformar y cargar la información técnica de los documentos CAP en PDF en ficheros JSON estructurados, limpios y anonimizados. A continuación, se describen las fases clave de este bloque:

- **Identificación de la plantilla del documento:** dado que los informes CAP pueden variar en su estructura visual y distribución de contenido, el sistema comienza aplicando OCR sobre la zona del documento correspondiente a los datos de emplazamiento. A partir de la ubicación y el formato de esta información, se clasifica el documento en una de las plantillas predefinidas (A o B). Esta detección condiciona el tratamiento posterior, ya que determina qué lógica de extracción aplicar en cada sección.
- **Extracción segmentada por secciones:** Una vez identificada la plantilla, el sistema procede a extraer la información técnica mediante un enfoque híbrido. Se combinan técnicas de OCR zonal y extracción estructurada con librerías como *pdfplumber*. La extracción se realiza por secciones lógicas del documento, principalmente:
  - **Emplazamiento:** identificadores, coordenadas geográficas y ubicación.
  - **Cálculos y tablas:** consumo energético, tabla del hardware instalado y tabla con los datos de los sectores del nodo, es decir, las tecnologías desplegadas, el tipo de antenas activas, el EDT o MDT dado a cada sector, etc.
  - **Tareas del operador y del propietario:** tareas a realizar por el operador y por el propietario del nodo, representadas en cuadros de texto de colores.
  - **Prevención de riesgos laborales (PRL):** como se debe acceder al nodo y a la ubicación de los equipos radio.
- **Procesado, limpieza y normalización:** los datos extraídos pueden presentar variaciones en nombres, formatos o unidades, según la plantilla o la calidad del documento original. Esta etapa aplica reglas de limpieza y normalización que transforman los datos a una estructura común. De este modo, se garantiza que todos los documentos, independientemente de su origen, generen un JSON con un esquema homogéneo, lo cual es esencial para el procesamiento posterior en los módulos de vectorización y RAG.

- **Anonimización:** esta etapa se encarga de garantizar la privacidad y cumplir criterios de protección de datos. Se aplica un proceso de anonimización que sustituye nombres del cliente, ubicaciones reales y otros identificadores sensibles por valores ficticios consistentes. Esta transformación permite mantener la coherencia técnica del documento sin exponer datos reales.
- **Generación del JSON estructurado:** finalmente, los datos procesados se agrupan en un fichero JSON estructurado jerárquicamente por secciones. Este archivo representa la versión digital técnica del nodo y actúa como entrada principal para su indexación semántica en la siguiente etapa del sistema.

De forma resumida, este módulo recibe como entrada un documento CAP en formato PDF, a partir del cual identifica la plantilla correspondiente y procede a extraer, limpiar, normalizar y anonimizar la información técnica contenida en distintas secciones del documento. El resultado y salida del módulo es un fichero JSON anonimizado que sirve como entrada del siguiente bloque funcional, el cual se comentará a continuación.

### 4.3 Módulo de vectorización

El módulo de vectorización actúa como puente entre la información estructurada obtenida en el módulo anterior y el sistema de recuperación aumentada por generación (RAG). Su finalidad es representar el contenido de forma que pueda ser recuperado eficientemente en función de las consultas del usuario. Para ello, el sistema transforma los datos contenidos en el fichero JSON anonimizado en vectores numéricos que capturan su semántica, y los almacena en una base vectorial optimizada para búsquedas por similitud.

El proceso comienza con la fragmentación del contenido del JSON. Cada sección extraída del informe CAP se divide en pequeños fragmentos de información, que reciben el nombre de *docs* y representan unidades informativas coherentes. Además, cada uno de ellos se etiqueta con metadatos relevantes como el tipo de sección o la plantilla asociada, mejorando la trazabilidad de los datos.

A continuación, cada fragmento se transforma en una representación vectorial utilizando el modelo “all-MiniLM-L6-v2” de SentenceTransformers, que genera *embeddings* de 384 dimensiones adecuados para tareas de recuperación semántica.

Los vectores resultantes se almacenan en un índice vectorial mediante la librería FAISS. En concreto, se emplea el índice IndexFlatIP (tipo flat), que permite búsquedas eficientes por similitud de coseno en tiempo real, con alta precisión en volúmenes moderados de datos.

La salida del módulo es, por tanto, un fichero persistente en formato binario (.faiss) que guarda los índices vectoriales, acompañado de un archivo en formato Pickle (.pkl), que almacena todos los *docs* y sus metadatos.

## 4.4 Módulo RAG

El módulo RAG es el encargado de dar respuesta a las preguntas del usuario combinando mecanismos de recuperación semántica y generación de lenguaje natural. A partir del índice vectorial construido previamente, el sistema localiza los fragmentos más relevantes en función de la consulta, los integra como contexto y genera una respuesta coherente mediante un modelo LLM. Este enfoque garantiza que las respuestas estén fundamentadas en información real y actualizada, al tiempo que se expresan de forma comprensible para el usuario.

El proceso comienza cuando el usuario formula una consulta a través de la interfaz. Esta consulta es enviada al backend mediante la API y es procesada por el subsistema RAG en tres etapas principales:

1. **Recuperación de contexto por similitud:** La pregunta del usuario se transforma en un vector de *embeddings* utilizando el mismo modelo (all-MiniLM-L6-v2) empleado en la vectorización de los documentos. Este vector de consulta se compara con los vectores almacenados mediante una búsqueda por similitud de coseno. El sistema selecciona los fragmentos más relevantes (*top-k*) y los recupera como contexto textual, conservando la información estructurada por sección.
2. **Construcción del *prompt* y consulta al LLM:** Los fragmentos recuperados se concatenan junto con la pregunta original en un *prompt* predefinido que guía el comportamiento del modelo LLM. Este *prompt* incluye instrucciones específicas, así como un rol. El modelo LLM utilizado en el sistema se ejecuta de forma local a través de Ollama, tal como se ha comentado anteriormente.
3. **Generación de la respuesta:** El LLM procesa el *prompt* y genera una respuesta que se devuelve al usuario a través de la API y la interfaz. Esta respuesta está contextualizada, filtrada por relevancia y redactada de forma natural, lo que permite al usuario acceder a la información contenida en los CAP sin necesidad de revisar manualmente los documentos.

Este módulo representa la capa cognitiva del sistema, donde la información técnica estructurada y anonimizada se transforma en conocimiento consultable. El uso de RAG permite combinar la precisión de los sistemas de recuperación con la flexibilidad expresiva de los modelos generativos, habilitando un acceso conversacional eficiente y controlado, evitando alucinaciones de los modelos LLM.

## 4.5 Interfaz de usuario

La interfaz de usuario (UI) constituye el punto de entrada para la interacción entre el usuario y el sistema de consulta conversacional. Su diseño se ha orientado a facilitar el acceso a la información técnica contenida en los informes CAP mediante un canal conversacional sencillo, accesible y eficiente. El sistema no requiere que el usuario conozca la estructura interna de los documentos, ya que puede realizar preguntas en lenguaje natural a través de un chatbot.

La arquitectura de la interfaz se organiza en dos niveles funcionales:



- **Aplicación web (frontend):** desarrollada con la librería Streamlit, esta capa proporciona el entorno gráfico a través del cual el usuario interactúa con el sistema. La interfaz permite introducir de forma sencilla el identificador del nodo a consultar, así como escribir preguntas en lenguaje natural. La conversación se presenta en formato tipo chat, mostrando tanto las preguntas formuladas como las respuestas generadas por el sistema. El uso de Streamlit se ha escogido ya que permite un desarrollo ágil, con integración directa en Python y ejecución local sin necesidad de servidores web adicionales.
- **API (backend):** implementada con FastAPI, esta capa intermedia gestiona las solicitudes entre la interfaz y los módulos de procesamiento semántico del sistema. Recibe las preguntas del usuario, consulta el módulo RAG con los parámetros adecuados y devuelve la respuesta correspondiente. Además, gestiona posibles errores o excepciones y mantiene una lógica desacoplada para facilitar la escalabilidad o integración futura con otros servicios.

En conjunto, la interfaz de usuario representa la capa de accesibilidad del sistema, transformando un complejo *pipeline* de procesamiento semántico en una herramienta consultiva directa y usable por cualquier técnico, ingeniero o analista sin necesidad de formación adicional.



## Capítulo 5. Desarrollo e implementación del sistema

En este capítulo se describe en detalle el proceso de desarrollo e implementación del sistema propuesto. Para ello, se analizan los distintos módulos implementados, los scripts desarrollados y las herramientas utilizadas en cada fase, desde la estructura de carpetas hasta la lógica interna de los *pipelines* de extracción, vectorización, RAG e interfaz de usuario.

El sistema ha sido desarrollado en Python 3.12 y está diseñado para ejecutarse de forma local en un entorno virtual. Además, para garantizar la reproducibilidad y el correcto funcionamiento de todos los módulos en cualquier equipo, se ha definido un fichero 'requirements.txt' que contiene las librerías necesarias a instalar para su ejecución.

### 5.1 Organización del proyecto y estructura de carpetas

El proyecto se ha estructurado en base a la arquitectura modular explicada en el apartado 4.1. Esta organización permite separar responsabilidades, facilitar la depuración y escalar el sistema con nuevas funcionalidades en el futuro. A continuación, se presenta un resumen de la estructura del repositorio:

- **backend:** contiene la lógica principal del sistema dividida por módulos funcionales.
  - **etl:** módulo de extracción, transformación y carga de datos. En él se encuentran los scripts `pipeline_etl.py`, `cap_extractor.py`, `normalizar_json.py` y `anon_json.py`, encargados de procesar los informes CAP y generar estructuras JSON estandarizadas y anonimizadas. Además, incluye las siguientes carpetas:
    - **ocr:** scripts de reconocimiento óptico de caracteres, tanto por contenido (`ocr_text.py`) como por color de zona (`ocr_color.py`).
    - **parsers:** contiene funciones auxiliares especializadas para extraer y estructurar contenido técnico del documento, como tablas, identificadores o coordenadas.
    - **utils:** utilidades generales empleadas en el proceso ETL, como operaciones geográficas, manipulación de PDFs o transformación de coordenadas.
  - **vectorization:** contiene el script relacionado con la generación de *embeddings* y la creación del índice semántico.
  - **rag:** módulo responsable del sistema de recuperación aumentada con generación. En él se encuentra '`consultar_cap.py`', que gestiona la recuperación semántica de fragmentos a partir de una consulta en lenguaje natural y construye el *prompt* para el modelo LLM.
  - **api:** contiene los componentes del backend web desarrollados con FastAPI. Incluye '`main.py`', que define los *endpoints* de extracción y consulta, y '`cargar_datos.py`', que gestiona la lectura de los fragmentos y el acceso al índice vectorial.
- **frontend:** carpeta que implementa la interfaz conversacional, en ella se encuentra el fichero '`chatbot.py`', que representa en la interfaz web el chatbot.
- **config:** aquí se encuentra el archivo '`config.json`', donde se definen parámetros y las coordenadas OCR por plantilla de las secciones del documento
- **data:** carpeta donde se almacenan todos los datos del sistema. Se divide en:

- **raw:** documentos CAP originales en formato PDF.
- **processed:** archivos JSON estructurados sin anonimizar de los nodos procesados en el módulo ETL.
- **final:** archivos JSON estructurados anonimizados de los nodos procesados en el módulo ETL.
- **index:** ficheros de los índices vectoriales y de los fragmentos procesados en el módulo de vectorización.

Gracias a esta organización, es posible adaptar el sistema a nuevos formatos de entrada, incorporar otros modelos de lenguaje o extender la funcionalidad de la interfaz sin modificar el núcleo del procesamiento técnico.

## 5.2 Implementación del pipeline de extracción

El pipeline o módulo ETL implementa la cadena de procesamiento encargada de convertir la información de los CAP en archivos JSON estructurados y anonimizados.

Se ejecuta a través del script 'pipeline\_etl.py' y su función es buscar el documento CAP dentro de la carpeta 'raw' de un nodo dado como parámetro de entrada. Seguidamente, aplica el proceso completo de extracción, normalización y anonimización, y guarda el resultado final en formato JSON. El script está diseñado para ejecutarse desde terminal, permitiendo procesar lotes completos de informes de forma automática.

En la Fig. 14 se muestra el código fuente completo del script, el cual se estructura en una serie de funciones claramente diferenciadas:

- **cargar\_config:** abre el fichero 'config.json' que contiene las coordenadas OCR por plantilla.
- **extraer\_datos:** instancia un objeto de la clase 'CAPExtractor', que encapsula toda la lógica de extracción del documento. La función `procesar` realiza las operaciones necesarias y devuelve un diccionario con la información extraída.
- **normalizar\_datos:** invoca a la función `normalizar_y_reestructurar_json_cap`, que adapta la estructura del JSON de salida para homogeneizar los distintos formatos plantilla.
- **anonimizar\_datos:** utiliza la función `anonimizar_json`, que aplica reglas de anonimización sobre los campos sensibles.
- **guardar\_json:** se encarga de guardar los JSON normalizados y anonimizados, creando la estructura de carpetas necesarias si no existen.
- **procesar\_cap:** representa el núcleo del flujo. Recibe un identificador del nodo (`id_cap`), construye las rutas esperadas, lanza los procesos anteriores y gestiona los logs de seguimiento del proceso.
- **main:** define el punto de entrada desde la terminal. Utiliza `argparse` para definir el argumento 'id\_cap', y posteriormente llama a `procesar_cap` con la configuración cargada.

```
# pipeline_etl.py
import json
import logging
from pathlib import Path
import argparse
from backend.etl.cap_extractor import CAPExtractor
from backend.etl.normalizar_json import normalizar_y_reestructurar_json_cap
from backend.etl.anon import anonimizar_json

logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

def cargar_config(path="config/config.json"):
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def extraer_datos(pdf_path: Path, config: dict) -> dict:
    extractor = CAPExtractor(str(pdf_path), config)
    extractor.procesar()
    return extractor.resultado

def normalizar_datos(resultado: dict) -> dict:
    return normalizar_y_reestructurar_json_cap(resultado)

def anonimizar_datos(datos: dict, id_cap) -> dict:
    return anonimizar_json(datos, id_cap)

def guardar_json(data: dict, path: Path):
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def procesar_cap(id_cap: str, config: dict):
    pdf_path = Path("data/raw") / f"{id_cap}.pdf"
    json_path = Path("data/processed") / f"{id_cap}.json"
    anon_path = Path("data/final") / f"{id_cap}_anon.json"

    if not pdf_path.exists():
        logging.error(f"No se encuentra el archivo PDF: {pdf_path}")
        return

    logging.info(f"Procesando CAP: {id_cap}")

    datos = extraer_datos(pdf_path, config)
    normalizado = normalizar_datos(datos)
    guardar_json(normalizado, json_path)
    logging.info(f"Guardado JSON normalizado en: {json_path}")

    anon = anonimizar_datos(normalizado, id_cap)
    guardar_json(anon, anon_path)
    logging.info(f"Guardado JSON anonimizado en: {anon_path}")

def main():
    parser = argparse.ArgumentParser(description="Extrae y transforma un documento CAP en un JSON estructurado.")
    parser.add_argument("id_cap", help="Ej: CAP_TFMA101")
    args = parser.parse_args()

    config = cargar_config()
    procesar_cap(args.id_cap, config)

if __name__ == "__main__":
    main()
```

Figura 14. Código fuente del pipeline ETL

En los siguientes subapartados se explican los scripts auxiliares de extracción, normalización y anonimización.

### 5.2.1 Script de extracción

Este módulo implementa la lógica principal de procesamiento de los informes CAP, encapsulada en la clase 'CAPExtractor'. Se encarga de extraer, estructurar y consolidar los distintos bloques de información presentes en el documento, según su plantilla. El código completo se muestra en la Fig. 15 y puede dividirse en los siguientes componentes clave:

- **Constructor `__init__`**: inicializa el objeto cargando el archivo PDF mediante `fitz`, el archivo de configuración, y establece variables internas como el tipo de plantilla (A o B) y el resultado parcial (`self.resultado`), que se irá completando.
- **`extraer_info_general`**: detecta la plantilla del documento CAP. Llama internamente a `_intentar_extraer_plantilla` pasando primero la opción "A" y, si no se detecta, prueba con "B". Se evalúa cuál de las posibles variantes de plantilla (A1, A2, B1, B2) contiene mayor número de campos válidos, y se selecciona la más completa.
- **`intentar_extraer_plantilla`**: utiliza OCR por zonas (`extraer_texto_ocr_por_zona`) para extraer fragmentos clave del encabezado, como identificadores del nodo, coordenadas o dirección. Se comparan los resultados según la cantidad de campos válidos y se devuelve el mejor ajuste como diccionario.
- **`extraer_calculos`**: busca la sección de cálculos técnicos (consumos, hardware y antenas), localizando la página por título y extrayendo las tablas correspondientes. Cada tabla se parsea con funciones específicas del módulo `parsers` (véase Anexo A.1).
- **`extraer_tabla_prl`**: extrae información de prevención de riesgos laborales, siguiendo una lógica similar a `extraer_calculos`. Localiza la tabla correspondiente a la sección PRL y la divide entre "antenas" y "equipos", con ayuda de `procesar_tabla_prl`.
- **`extraer_trabajos`**: utiliza segmentación por color (`buscar_trabajos_por_color`) para identificar bloques asociados a tareas por empresa. Luego aplica procesamiento de texto y resumen técnico para estructurarlo como texto plano y resumen.
- **`procesar`**: método orquestador que llama a todos los métodos anteriores en orden: `extraer_info_general`, `extraer_calculos`, `extraer_tabla_prl` y `extraer_trabajos`. El resultado se guarda en el atributo `self.resultado`.

```
# cap_extractor.py
import fitz
import json
from ocr.ocr_text import extraer_texto_ocr_por_zona
from ocr.ocr_color import buscar_trabajos_por_color
from utils.pdf_utils import encontrar_pagina_por_titulo, extraer_tablas_de_pagina
from parsers.parse_coordenadas import parsear_coordenadas
from parsers.parse_identificadores import parsear_identificadores_emplazamiento
from parsers.parse_tablas import parsear_tabla_potencia, parsear_tabla_hardware, parsear_tabla_antenas, procesar_tabla_prl
from parsers.procesar_trabajos import procesar_trabajos_detallado

class CAPExtractor:
    def __init__(self, pdf_path, config):
        self.pdf_path = pdf_path
        self.config = config
        self.doc = fitz.open(pdf_path)
        self.resultado = {}
        self.tipo_cap = 'A'

    def extraer_info_general(self):
        info = self._intentar_extraer_plantilla("A")
        if not info:
            info = self._intentar_extraer_plantilla("B")
        if info:
            self.resultado["emplazamiento"] = info["datos"]
            self.tipo_cap = info["tipo"]
        else:
            print("No se pudo identificar la plantilla del CAP.")

    def _intentar_extraer_plantilla(self, tipo):
        page = self.doc[0]
        rects = self.config["rectangles"]
        info_mejor, tipo_mejor = None, None
        max_camos = 0
        for sufijo in ["1", "2"]:
            key_coords = f"coordenadas_{tipo}{sufijo}"
            if key_coords not in rects:
                continue
            texto_coords = extraer_texto_ocr_por_zona(page, tuple(rects[key_coords]))
            coords = parsear_coordenadas(texto_coords, False)
            if not coords:
                continue
            nodo_txt = extraer_texto_ocr_por_zona(page, tuple(rects.get(f"nodo_{tipo}", [])))
            dir_txt = extraer_texto_ocr_por_zona(page, tuple(rects.get(f"direccion_{tipo}", [])))
            identificadores = parsear_identificadores_emplazamiento(nodo_txt + "\n" + dir_txt)
            total_camos = len(identificadores) + len(coords)
            if total_camos > max_camos:
                info_mejor = {"**identificadores", **coords}
                tipo_mejor = f"{tipo}{sufijo}"
                max_camos = total_camos
        if info_mejor:
            print(f"Plantilla detectada: {tipo_mejor} ({max_camos} campos)", "\n", info_mejor)
            return {"datos": info_mejor, "tipo": tipo_mejor}
        return None

    def extraer_calculos(self):
        rect = self.config["rectangles"].get(f"titulo_pagina_{self.tipo_cap[0]}", [])
        idx = encontrar_pagina_por_titulo(self.doc, tuple(rect), ["CALCULOS", "ACTUACIONES"])
        if idx is not None:
            tablas = extraer_tablas_de_pagina(self.pdf_path, idx)
            self.resultado["calculos"] = {"consumos": parsear_tabla_potencia(tablas),
                                         "hardware": parsear_tabla_hardware(tablas),
                                         "antenas": parsear_tabla_antenas(tablas)}

    def extraer_tabla_prl(self):
        titulos_prl = ["PLANTA P.R.L.", "P.R.L. PLANTA", "P.R.L. PLANTA GENERAL"]
        rect = self.config["rectangles"].get(f"titulo_pagina_{self.tipo_cap[0]}", [])
        idx = encontrar_pagina_por_titulo(self.doc, tuple(rect), titulos_prl)
        if idx is not None:
            tablas = extraer_tablas_de_pagina(self.pdf_path, idx)
            for tabla in tablas:
                if len(tabla) > 15:
                    prl_antenas, prl_equipos = procesar_tabla_prl(tabla)
                    self.resultado["prl"] = {"antenas": prl_antenas, "equipos": prl_equipos}

    def extraer_trabajos(self):
        trabajos = {}
        for empresa in self.config['colores_hsv'].keys():
            seccion = buscar_trabajos_por_color(self.pdf_path, empresa, self.config['colores_hsv'])
            if seccion:
                texto = seccion['texto']
                resumen = procesar_trabajos_detallado(texto)
                trabajos[empresa] = {"texto": texto, "resumen": resumen}
        self.resultado['tarefas'] = trabajos

    def exportar_json(self, output_path):
        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(self.resultado, f, ensure_ascii=False, indent=4)

    def procesar(self):
        self.extraer_info_general()
        self.extraer_calculos()
        self.extraer_tabla_prl()
        self.extraer_trabajos()
```

Figura 15. Código fuente del script de extracción

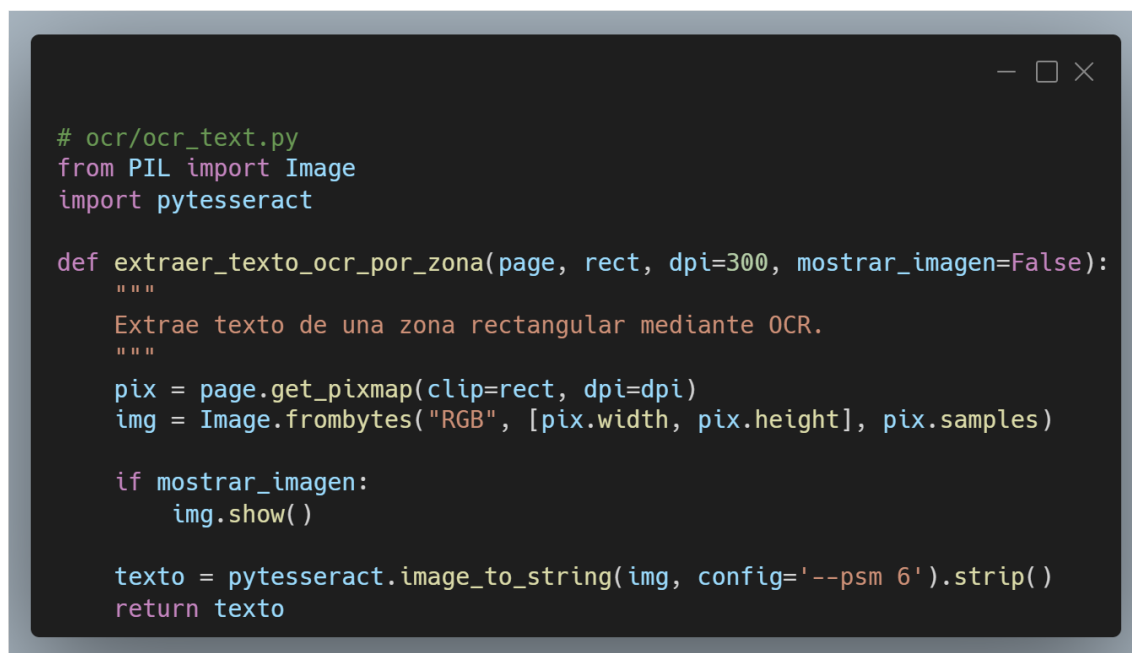
Este módulo representa el núcleo de la etapa ETL del sistema, ya que abstrae las particularidades del documento CAP y unifica la salida bajo un mismo esquema estructurado. Su diseño modular, basado en funciones específicas por sección, permite su mantenimiento y mejora de forma sencilla.

#### 5.2.1.1 Funciones de extracción OCR

Dos funciones clave del script de extracción que permiten extraer la información a través de OCR son:

- extraer texto ocr por zona: función que permite leer texto de una zona concreta del documento, indicada por unas coordenadas. Se utiliza cuando se sabe exactamente en qué parte de la página se encuentra una información importante, como el nombre del nodo o las coordenadas. La función convierte esa zona en una imagen y extrae el texto que contiene.
- buscar trabajos por color: se encarga de localizar bloques de color que aparecen en los CAP para señalar tareas asignadas al operador o propietario. La función detecta estos bloques y extrae el texto que contienen, devolviendo solo aquellos que incluyen indicaciones.

Dichas funciones se muestran en la Fig. 16 y Fig. 17 respectivamente.



```
# ocr/ocr_text.py
from PIL import Image
import pytesseract

def extraer_texto_ocr_por_zona(page, rect, dpi=300, mostrar_imagen=False):
    """
    Extrae texto de una zona rectangular mediante OCR.
    """
    pix = page.get_pixmap(clip=rect, dpi=dpi)
    img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)

    if mostrar_imagen:
        img.show()

    texto = pytesseract.image_to_string(img, config='--psm 6').strip()
    return texto
```

Figura 16. Script de extracción de texto OCR



```
# ocr/ocr_color.py
import numpy as np
import cv2
import pytesseract
from pdf2image import convert_from_path

def buscar_trabajos_por_color(pdf_path, empresa, rangos_hsv, dpi=300):
    """
    Busca bloques de color en el PDF asociados a trabajos por empresa.
    Retorna texto OCR si encuentra la sección.
    """
    empresa = empresa.lower()
    imagenes = convert_from_path(pdf_path, dpi=dpi)
    if not imagenes:
        return None

    if empresa not in rangos_hsv:
        raise ValueError("Empresa no reconocida")

    lower = np.array(rangos_hsv[empresa]['lower'])
    upper = np.array(rangos_hsv[empresa]['upper'])

    for i, img_pil in enumerate(imagenes):
        img = np.array(img_pil.convert('RGB'))
        hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
        mascara = cv2.inRange(hsv, lower, upper)
        contornos, _ = cv2.findContours(mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        for cnt in contornos:
            x, y, w, h = cv2.boundingRect(cnt)
            if w > 50 and h > 15:
                zona = img[y:y+h, x:x+w]
                texto = pytesseract
                if any(kw in texto.upper() for kw in ["TRABAJOS A REALIZAR", "TAREAS A REALIZAR"]) and len(texto) > 50:
                    return {"pagina": i, "texto": texto}

    return None
```

Figura 17. Script de extracción de texto OCR por color

El resto de las funciones auxiliares y módulos que forman parte del script de extracción se detallan en el Anexo A.1.

### 5.2.2 Scripts de normalización y anonimización

Una vez extraído el contenido técnico del informe CAP y almacenado en un diccionario Python, el siguiente paso consiste en estandarizar su estructura y anonimizar cualquier información sensible. Para ello se utilizan los scripts ‘normalizar\_json.py’ y ‘anon\_json.py’, cuyas funciones principales se muestran en la Fig. 18 y Fig. 19 respectivamente.



```
# normalizar_json.py
import re
import copy
from collections import defaultdict

...
...

def normalizar_y_reestructurar_json_cap(json_dict):
    data = copy.deepcopy(json_dict)

    # 1. Limpiar coordenadas
    if "emplazamiento" in data:
        for campo in ["Latitud", "Longitud"]:
            if campo in data.get("emplazamiento", {}):
                valor = data["emplazamiento"][campo]
                data["emplazamiento"][campo] = valor.replace(" ", "").replace("\n", "")

    # 2. Limpiar y normalizar calculos
    if "calculos" in data:
        # 2.1 Normalizar Potencia
        raw_consumos = data.get("calculos", {}).get("consumos", {})
        data["calculos"]["consumo"] = normalizar_consumos(raw_consumos)
        data["calculos"].pop("consumos", None)
        # 2.2 Normalizar Hardware
        data["calculos"]["hardware"] = normalizar_hardware(data["calculos"].get("hardware", {}))
        # 2.3 Normalizar Antenas y Sectores
        data["calculos"]["antenas"] = normalizar_antenas(data["calculos"].get("antenas", {}))

    # 3. Limpiar y normalizar PRL
    if "prl" in data:
        data["prl"] = normalizar_prl(data["prl"])

    # 4. Eliminar claves vacías
    def limpiar_diccionario(d):
        if isinstance(d, dict):
            return {k: limpiar_diccionario(v) for k, v in d.items() if v not in [None, "", [], {}]}
        elif isinstance(d, list):
            return [limpiar_diccionario(i) for i in d if i not in [None, "", [], {}]]
        return d

    return limpiar_diccionario(data)
```

Figura 18. Script de normalización

La función `normalizar_y_reestructurar_json_cap`, se encarga de aplicar un conjunto de transformaciones sobre el diccionario de datos. En primer lugar, limpia campos como las coordenadas de latitud y longitud, eliminando caracteres innecesarios. A continuación, reestructura los datos técnicos del apartado de cálculos, unificando el formato de los consumos, el hardware y las antenas mediante funciones auxiliares. También aplica un procedimiento similar sobre los datos de prevención y riesgos laborales (PRL), asegurando así la homogeneidad del esquema JSON. Por último, se eliminan recursivamente todas las claves vacías para obtener un resultado limpio y uniforme. Las funciones de este script se muestran más en detalle en el Anexo A.2.

```
# anon_json.py
import copy, re, random
from typing import Dict, Any

REGEX_REPL = [
    (r"\...\b", "PROPIETARIO"),
    (r"\...\b", "OPERADOR"),
]

def anonimizar_json(data: Dict[str, Any], id_cap, verbose: bool = False) -> Dict[str, Any]:
    anon = copy.deepcopy(data)

    empl = anon.get("emplazamiento", {})
    empl["Nodo"] = id_cap
    empl["Cluster"] = "MUIT2025"
    empl["Direccion"] = "Camino de Vera, 25"
    empl["Municipio"] = "ALGIROS"
    empl["Provincia"] = "VALENCIA"
    empl["Latitud"] = "39°28'51.96 N"
    empl["Longitud"] = "0°20'24.00 W"
    anon["emplazamiento"] = empl

    tareas = anon.get("tareas", {})
    for k, info in tareas.items():
        text = info.get("texto")
        if not text:
            continue
        for pattern, repl in REGEX_REPL:
            text = re.sub(pattern, repl, text, flags=re.IGNORECASE)
        info["texto"] = text

    return anon
```

Figura 19. Script de anonimización

Posteriormente, el archivo es anonimizado mediante la función `anonimizar_json`, mostrada en la Fig. 19. En este paso, se sustituyen los valores sensibles del bloque de emplazamiento, como el identificador de nodo por datos ficticios, y la dirección o las coordenadas por valores estáticos predefinidos, donde se ha optado, como se puede observar, por los datos de la ETSIT y de la UPV.

Asimismo, se aplican expresiones regulares sobre los textos de las tareas para reemplazar posibles apariciones de los nombres del operador o el propietario del emplazamiento, los cuales se sustituyen por operador o propietario, protegiendo así la confidencialidad de la información.

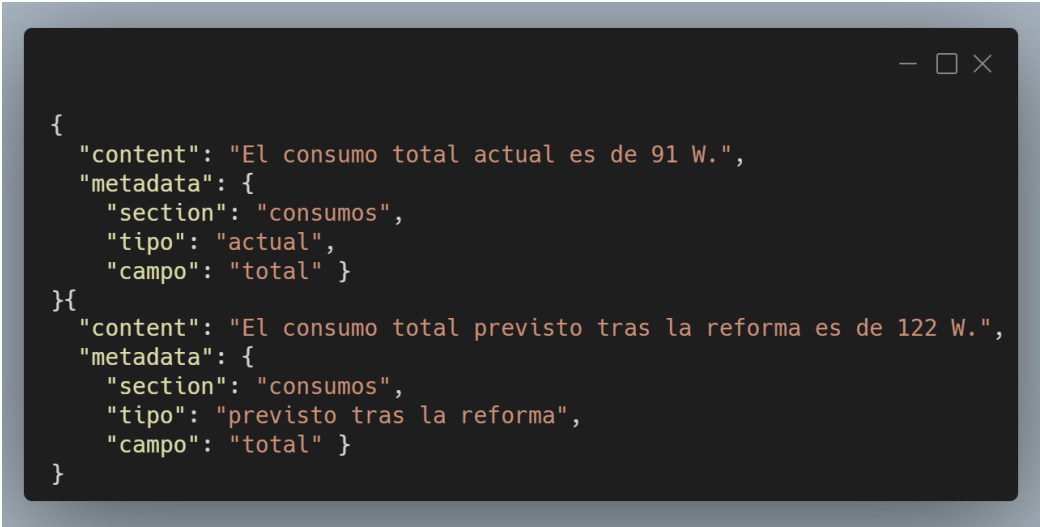
Gracias a esta etapa final, el sistema genera como salida un archivo JSON listo para ser vectorizado o consultado, con una estructura estandarizada y libre de datos identificables.

### 5.3 Implementación del módulo de vectorización

El módulo de vectorización constituye el bloque intermedio entre la extracción de datos y el sistema RAG. Su objetivo es transformar el contenido técnico estructurado en los archivos JSON anonimizados en representaciones vectoriales que capturen su significado semántico. Estas representaciones son almacenadas en un índice optimizado que permitirá recuperar fragmentos relevantes a partir de una consulta en lenguaje natural. Todo este proceso se encuentra implementado en el script `'indexar_cap.py'`.

El flujo de ejecución se resume en los siguientes pasos:

1. **Carga del archivo JSON:** a partir del identificador de nodo, el script carga el JSON anonimizado ubicado en la carpeta data/final/.
2. **Fragmentación del contenido técnico:** a través de la función `json_a_docs`, el contenido se divide en fragmentos semánticos, agrupados por secciones como emplazamiento, cálculos, PRL o tareas. Cada fragmento contiene un campo 'content' con el texto, y un campo 'metadata' que almacena información contextual como el tipo de dato, subtipo, nodo, etc. Por ejemplo, el bloque de consumos genera fragmentos como:



```
{
  "content": "El consumo total actual es de 91 W.",
  "metadata": {
    "section": "consumos",
    "tipo": "actual",
    "campo": "total" }
}{
  "content": "El consumo total previsto tras la reforma es de 122 W.",
  "metadata": {
    "section": "consumos",
    "tipo": "previsto tras la reforma",
    "campo": "total" }
}
```

Figura 20. Ejemplo de fragmentación del bloque de consumos

3. **Generación de embeddings:** los fragmentos se codifican en vectores utilizando el modelo “paraphrase-multilingual-MiniLM-L12-v2” de la librería SentenceTransformers. Este modelo ha sido seleccionado por su capacidad multilingüe y su buen rendimiento en entornos locales.
4. **Creación del índice vectorial con FAISS:** una vez generados los *embeddings*, se crea un índice con la librería FAISS, específicamente del tipo ‘IndexFlatL2’.
5. **Almacenamiento de resultados:** finalmente, se guardan los ficheros (.faiss y .pkl) y se almacenan en la carpeta data/index/.

El main del script se muestra en la Fig. 21, donde se pueden ver los pasos comentados de ejecución, mientras que el resto del script se incluye en el Anexo A.3.

```
if __name__ == "__main__":
    nombre_archivo = "CAP_TFMA101_anon.json"
    id_cap = nombre_archivo.split("_")[1] # Extraer ID del nombre del archivo
    print(id_cap)
    # Cargar JSON
    cap_json = cargar_json("data/final/" + nombre_archivo)
    # Convertir a documentos
    docs = json_a_docs(cap_json)
    texts = [doc['content'] for doc in docs]
    # Generar embeddings con SentenceTransformer
    embedder = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
    embeddings = embedder.encode(texts, show_progress_bar=True)
    embeddings = np.array(embeddings).astype("float32")
    # Crear índice FAISS
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    # Guardar índice y documentos
    faiss.write_index(index, f"cap_index_{id_cap}.faiss")
    with open(f"cap_docs_{id_cap}.pkl", "wb") as f:
        pickle.dump(docs, f)

    print(f"Indexados {len(docs)} documentos para el nodo {id_cap}")
```

Figura 21. Main del script de vectorización

Este script puede ejecutarse independientemente desde terminal para indexar nuevos nodos CAP una vez hayan sido procesados por el pipeline ETL.

## 5.4 Implementación del sistema RAG

El sistema RAG implementado combina un motor de recuperación semántica con un modelo LLM para responder consultas a partir de los fragmentos e índices vectorizados del anterior bloque.

El flujo general del sistema se resume en los siguientes pasos:

1. **Detección de la sección relevante:** el sistema intenta identificar primero qué sección del documento se relaciona con la pregunta del usuario. Para ello, se emplea la función `detectar_seccion_relevante`, que codifica la pregunta del usuario en un *embedding* y la compara con una base de *embeddings* predefinidos correspondientes a descripciones semánticas de cada sección.

Esta comparación se realiza mediante el producto escalar sobre los vectores normalizados, empleando la función `'dot'` de Numpy, lo que equivale a calcular la similitud del coseno entre los vectores. La función devuelve la sección con mayor similitud siempre que supere un umbral mínimo. Por el contrario, si no se alcanza ese umbral, se devuelve *None*, permitiendo que el sistema realice la búsqueda sobre todos los fragmentos disponibles. La función que detecta la sección relevante, así como las descripciones por sección se muestran en la Fig. 222.

```
# Descripciones semánticas por sección
DESCRIPCIONES_SECCIONES = {
    "emplazamiento": "Ubicación del nodo, dirección, municipio, provincia, coordenadas geográficas, latitud, longitud, cota.",
    "consumos": "Consumo eléctrico actual y reformado, consumo total, consumo por componente como eq_radio, tx, bbu, rru.",
    "hardware": "Modelos y cantidades de antenas, RRUs, BBUs, conectores, tipo de hardware instalado.",
    "prl": "Prevención de riesgos laborales (PRL), accesos a las antenas, medidas y sistemas de seguridad",
    "antenas_sector": "Sectores del nodo, distribución de antenas por sector, modelos predominantes, tecnologías, orientaciones.",
    "tareas": "Tareas del operador y propietario, responsabilidades, mantenimiento, configuración, instalación."
}

def detectar_seccion_relevante(pregunta, embedder, seccion_embeddings, verbose=True):
    pregunta_emb = embedder.encode([pregunta], normalize_embeddings=True)
    similitudes = np.dot(seccion_embeddings, pregunta_emb.T).squeeze()
    secciones = list(DESCRIPCIONES_SECCIONES.keys())
    idx_max = np.argmax(similitudes)
    seccion = secciones[idx_max]

    if verbose:
        print("Similitudes por sección:")
        for sec, sim in zip(secciones, similitudes):
            print(f" {sec}: {sim:.3f}")

    return seccion if similitudes[idx_max] > 0.4 else None # Umbral ajustable
```

Figura 22. Función detectar sección relevante

El umbral de similitud escogido ha sido 0.4, ya que, tras probar con varios valores, ha sido el que mejor detectaba las secciones. Este mecanismo aporta robustez y eficiencia al sistema, ya que reduce el espacio de búsqueda y focaliza la generación de respuestas en el contexto más relevante.

2. **Recuperación de fragmentos mediante FAISS:** una vez identificada la sección, el sistema procede a recuperar los fragmentos más próximos semánticamente mediante la función `buscar_similares`. Esta función adapta su estrategia en función de si se ha detectado o no la sección relevante:

- **Búsqueda con índice FAISS:** si no se ha detectado la sección, se emplea el índice vectorial completo. La consulta se codifica en un vector, y FAISS realiza una búsqueda eficiente basada en similitud del coseno, devolviendo los *top\_k* fragmentos más cercanos.
- **Búsqueda directa sin FAISS:** si previamente se ha detectado la sección. En este caso, se recalculan los *embeddings* de los textos localmente y se comparan directamente con el *embedding* de la pregunta mediante producto escalar.

Posteriormente, se aplica un filtro que devuelve solo los *docs* cuya sección coincida con la buscada y se eliminan posibles *docs* duplicados para evitar información redundante. Las funciones empleadas en este paso se reflejan en la Fig. 23.

```
def buscar_similares(query, embedder, index, docs, top_k=3):
    query_emb = embedder.encode([query]).astype("float32")
    if isinstance(index, faiss.Index):
        # Búsqueda en índice completo
        D, I = index.search(query_emb, top_k)
        resultados = [docs[idx] for idx in I[0] if idx < len(docs)]
    else:
        # Búsqueda en subconjunto (sin FAISS)
        docs_texts = [doc['content'] for doc in docs]
        doc_embs = embedder.encode(docs_texts).astype("float32")
        sims = np.dot(doc_embs, query_emb.T).squeeze()
        top_indices = sims.argsort()[::-1][:top_k]
        resultados = [docs[i] for i in top_indices]
    return resultados

def filtrar_docs_por_seccion(docs, seccion_objetivo):
    return [doc for doc in docs if doc['metadata'].get('section', '') == seccion_objetivo]

def eliminar_fragmentos_duplicados(docs):
    vistos = set()
    unicos = []
    for doc in docs:
        content = doc["content"].strip()
        if content not in vistos:
            vistos.add(content)
            unicos.append(doc)
    return unicos
```

Figura 23. Funciones de recuperación de fragmentos

3. **Construcción del *prompt* y generación de respuesta:** los fragmentos recuperados se concatenan en un *prompt* estructurado que se utiliza como entrada del modelo LLM. Este *prompt* sigue una plantilla diseñada para simular el comportamiento de un ingeniero experto en diseño radio. Incluye numeración de los fragmentos y una instrucción clara para que el modelo responda exclusivamente con base en esa información. Además, se especifica explícitamente que, si no se encuentra la respuesta en los fragmentos, el modelo debe indicar “No se encuentra información suficiente”.

La función `generar_respuesta` se encarga de enviar este *prompt* al servidor local de Ollama, que ejecuta el modelo especificado de forma eficiente en entorno local devolviendo una respuesta en lenguaje natural.



```
def construir_prompt_rag(context_chunks, pregunta):
    contexto = "\n\n".join([f"Fragmento {i+1}: {chunk}" for i, chunk in enumerate(context_chunks)])
    prompt = """
    Eres un ingeniero de telecomunicaciones experto en diseño radio con más de 25 años de experiencia.
    Responde en español de forma clara y concisa a la siguiente pregunta SOLO usando la información de los
    fragmentos proporcionados. Si la información no está en los fragmentos, responde 'No se encuentra información
    suficiente'.\n\n"""
    f"{contexto}\n\n"
    f"Pregunta: {pregunta}\n"
    "Respuesta:"
    )
    return prompt

def generar_respuesta(prompt, model="llama3:8b-instruct-q4_0"):
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": model,
            "prompt": prompt,
            "stream": False
        }
    )
    print("Respuesta bruta del modelo:", response.json()) # Agregado para depuración

    data = response.json()
    if "response" not in data:
        raise ValueError(f"La respuesta del modelo no contiene la clave 'response': {data}")

    return data["response"]
```

Figura 24. Función de construir prompt y generar respuesta

La función `responder_pregunta` encapsula toda la lógica del pipeline RAG: detección semántica, recuperación, filtrado, construcción del contexto, *prompting* y generación final. Dicha función se representa en la Fig. 25, donde su diseño modular y secuencial permite mantener claridad y adaptarse a distintos entornos.

```
def responder_pregunta(query, embedder, seccion_embeddings, index, docs, top_k=3, model="llama3:8b-instruct-q4_0"):
    print(f"\nPregunta recibida: {query}")

    seccion_relevante = detectar_seccion_relevante(query, embedder, seccion_embeddings)

    if seccion_relevante:
        print(f"Sección detectada: {seccion_relevante}")
        docs_filtrados = filtrar_docs_por_seccion(docs, seccion_relevante)
        print(f"Documentos filtrados por sección: {len(docs_filtrados)}")

        if len(docs_filtrados) > 0:
            docs_para_contexto = buscar_similares(query, embedder, None, docs_filtrados, top_k=top_k)
        else:
            print("No se encontraron documentos en la sección. Usando búsqueda general.")
            docs_para_contexto = buscar_similares(query, embedder, index, docs, top_k=top_k)
    else:
        print("No se detectó una sección relevante con suficiente confianza.")
        docs_para_contexto = buscar_similares(query, embedder, index, docs, top_k=top_k)

    print(f"Fragmentos recuperados antes de eliminar duplicados: {len(docs_para_contexto)}")
    docs_para_contexto = eliminar_fragmentos_duplicados(docs_para_contexto)
    print(f"Fragmentos únicos tras eliminar duplicados: {len(docs_para_contexto)}")

    context_chunks = [doc['content'] for doc in docs_para_contexto]
    if not context_chunks:
        print("No se encontraron fragmentos relevantes para construir el contexto.")
    else:
        print("Fragmentos utilizados en el prompt:")
        for i, frag in enumerate(context_chunks, 1):
            print(f" Fragmento {i}: {frag[:100]}...") # Mostrar solo los primeros 100 caracteres

    prompt = construir_prompt_rag(context_chunks, query)
    print("Prompt construido correctamente. Enviando al modelo...")

    respuesta = generar_respuesta(prompt, model)
    print("Respuesta generada por el modelo.")

    return respuesta, docs_para_contexto
```

Figura 25. Función responder pregunta



En este script, además de implementar la lógica del sistema RAG, también se ha programado la evaluación de varios modelos LLM, dada una batería de preguntas predefinidas. El comportamiento y los resultados de cada modelo se detallan en el siguiente capítulo.

## 5.5 Integración del backend vía API

Con el objetivo de facilitar el uso del sistema desde entornos externos y desacoplar la lógica de backend de la interfaz de usuario, se ha desarrollado una API REST utilizando el *framework* FastAPI. Esta arquitectura permite exponer la funcionalidad del sistema RAG como un servicio accesible vía HTTP, favoreciendo su escalabilidad, reutilización y despliegue en diferentes plataformas.

La API está definida en el script ‘main.py’ (véase Fig. 26), que implementa un único *endpoint* público llamado *consultar*, de tipo POST. Este *endpoint* encapsula todo el flujo del sistema RAG explicado en el anterior apartado.

Los datos de entrada se definen mediante un esquema ‘PreguntaRequest’, basado en Pydantic, que incluye:

- pregunta: texto de la consulta en lenguaje natural.
- modelo: nombre del modelo LLM a utilizar.
- cap: identificador del nodo CAP sobre el que se desea consultar.

El flujo que sigue el endpoint es el siguiente:

1. **Carga dinámica de datos:** se llama a la función `cargar_datos_por_nodo`, definida en `carga_datos.py`, la cual carga el índice vectorial FAISS y el conjunto de fragmentos correspondientes al nodo especificado. Además, inicializa el modelo de *embeddings* y los vectores de sección.
2. **Procesamiento de la pregunta:** se ejecuta la función `responder_pregunta`, que implementa la lógica completa del sistema RAG, tal como se ha comentado en el anterior apartado.
3. **Devolución de la respuesta:** la API responde en formato JSON, incluyendo la respuesta generada y los fragmentos utilizados como contexto para dar transparencia al razonamiento del sistema.

En caso de producirse errores durante la ejecución, se captura la excepción y se devuelve un código HTTP 500 con el mensaje correspondiente.

```
# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from backend.rag.consultar_cap import responder_pregunta
from backend.carga_datos import cargar_datos_por_nodo
import traceback

app = FastAPI()

class PreguntaRequest(BaseModel):
    pregunta: str
    modelo: str = "llama3:8b-instruct-q4_0"
    cap: str = "TFMA101"

@app.post("/consultar")
def consultar_pregunta(req: PreguntaRequest):
    try:
        print(req.cap)
        index, docs, embedder, seccion_embeddings = cargar_datos_por_nodo(req.cap)
        respuesta, fragmentos = responder_pregunta(req.pregunta, embedder, seccion_embeddings, index, docs, top_k=4, model=req.modelo)
        return {
            "respuesta": respuesta,
            "fragmentos": [frag["content"] for frag in fragmentos]
        }
    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))
```

Figura 26. Script principal de la API

Para lanzar el servidor de la API en entorno local, se utiliza el comando `uvicorn backend.api.main:app --reload` en la terminal del sistema, lanzando así un ASGI (*Asynchronous Server Gateway Interface*) a través de uvicorn, accesible desde el equipo local.

Si se desea lanzar el servidor para que sea accesible desde otros equipos en la misma red, se debe utilizar el comando `uvicorn backend.api.main:app --host 0.0.0.0 --port 8000 --reload`, que escucha en todas las interfaces de red del equipo empleando el puerto 8000.

## 5.6 Desarrollo de la interfaz conversacional

Por último, la interfaz web conversacional está basada en la biblioteca Streamlit, la cual permite construir aplicaciones interactivas de forma rápida y sencilla. Esta interfaz simula el comportamiento de un chatbot, permitiendo al usuario seleccionar un nodo y plantear preguntas sobre el contenido de su CAP correspondiente.

El código principal se encuentra en el script ‘chatbot.py’ cuyo funcionamiento puede dividirse en dos etapas:

1. **Selección del nodo:** al iniciar la aplicación, se solicita al usuario que introduzca el nombre del nodo CAP que desea consultar. Esta entrada se valida contra una lista predefinida. Si el nodo existe, se almacena en el estado de la sesión y se redirige automáticamente a la pantalla de chat.

```
st.set_page_config(page_title="CAP Bot")

# Inicializar el estado de la sesión
if "nodo" not in st.session_state:
    st.session_state.nodo = None
if "chat" not in st.session_state:
    st.session_state.chat = []

# Título de la app
st.title("🤖 CAP Bot")

# Paso 1: Selección del nodo
if st.session_state.nodo is None:
    # Mensaje de bienvenida
    st.markdown("¡Bienvenido! Soy tu asistente para consultar información técnica de nodos de telecomunicaciones.")
    nodo_input = st.text_input("🔍 Introduce el nodo a consultar (por ejemplo: TFMA101):")
    if nodo_input:
        nodo_input = nodo_input.strip().upper()
        if nodo_input in NODOS_DISPONIBLES:
            st.session_state.nodo = nodo_input
            st.success(f"Nodo {nodo_input} encontrado, abriendo chat de consulta. Por favor, espere unos segundos.")
            time.sleep(1)
            st.rerun()
        else:
            st.error("Nodo no encontrado. Por favor, prueba con otro.")
```

Figura 27. Parte del script frontend que selecciona el nodo

2. **Interfaz de conversación:** una vez validado el nodo, se muestra la interfaz conversacional, donde el usuario puede introducir preguntas en lenguaje natural a través de un campo de texto. Cada pregunta se envía como una solicitud HTTP POST al endpoint 'consultar' de la API local, cuya respuesta es generada por el backend y mostrada en pantalla junto con el historial de mensajes.

```
else:
    # Mensaje de bienvenida
    st.markdown(f"Chat de consulta para el nodo {st.session_state.nodo}")
    # Entrada de nueva pregunta
    pregunta = st.chat_input("Escribe tu pregunta...")
    if pregunta:
        st.session_state.chat.append({"role": "user", "content": pregunta})
        try:
            response = requests.post(
                "http://localhost:8000/consultar",
                json={"pregunta": pregunta, "modelo": "llama3:8b-instruct-q4_0", "nodo": st.session_state.nodo}
            )
            if response.status_code == 200:
                data = response.json()
                respuesta = data["respuesta"]
                st.session_state.chat.append({"role": "assistant", "content": respuesta})
            else:
                error_msg = f"Error al consultar la API: {response.status_code}"
                st.session_state.chat.append({"role": "assistant", "content": error_msg})
        except Exception as e:
            error_msg = f"No se pudo conectar con la API: {e}"
            st.session_state.chat.append({"role": "assistant", "content": error_msg})

# Mostrar historial de conversación solo si hay preguntas
if st.session_state.chat:
    st.markdown("----")
    for entrada in st.session_state.chat:
        with st.chat_message(entrada["role"]):
            st.markdown(entrada["content"])
```

Figura 28. Parte del script frontend que genera la interfaz de conversación

La interfaz gestiona el estado de forma reactiva, ya que las variables de sesión permiten mantener el nodo y el historial de conversación entre interacciones. Además, cada mensaje se muestra en pantalla diferenciando entre el rol de usuario y asistente, y en caso de errores de red o respuesta fallida, se notifica al usuario mediante mensajes de alerta.

## Capítulo 6. Evaluación y simulación del sistema

Tras la implementación completa del sistema, este capítulo presenta la evaluación de su rendimiento mediante pruebas comparativas entre distintos modelos LLM, así como una simulación de uso a través de la interfaz conversacional. El objetivo es validar la calidad de las respuestas generadas, el comportamiento del sistema ante preguntas reales y la eficacia de su arquitectura modular. Los resultados obtenidos permiten valorar la utilidad práctica del sistema, identificar sus limitaciones y establecer posibles líneas de mejora.

### 6.1 Evaluación de los modelos LLM empleados

La evaluación abarca cinco modelos: Phi3, TinyLlama, Mistral, Llama3 y Zephyr. Con el objetivo de seleccionar el modelo más adecuado, se ha utilizado una batería de preguntas que cubren todos los bloques extraídos del documento CAP. Estas son:

1. ¿En qué municipio y provincia se encuentra ubicado el nodo, y cuál es su latitud y longitud?
2. ¿Cuál es el consumo eléctrico total previsto tras la reforma y cuál es el consumo del equipo radio en ese escenario?
3. ¿Qué modelos de antena y de RRU se van a instalar en el nodo?
4. ¿Cómo se accede a las antenas desde el punto de vista de PRL y qué sistemas de seguridad existen para ellas?
5. ¿Cuántos sectores tiene el nodo y qué modelos de antenas predominan en cada sector?
6. Resume las tareas principales que debe realizar el operador y menciona al menos una acción que corresponde al propietario.

A cada modelo se le formularon las mismas preguntas y se evaluaron sus respuestas manualmente según seis criterios: precisión, cobertura, concisión, contextualización, ausencia de alucinaciones y latencia de respuesta.

La Tabla 2 recoge las métricas empleadas para evaluar el rendimiento de cada modelo. Cada métrica responde a un aspecto clave del comportamiento esperado en un sistema conversacional técnico. Las métricas de precisión, cobertura y contexto permiten valorar la fidelidad de la respuesta respecto al contenido técnico del documento. Concisión y alucinación evalúan aspectos cualitativos del lenguaje natural generado, y latencia ofrece una medida relativa del tiempo de respuesta en el entorno actual. La puntuación global se calcula como una media ponderada, con mayor peso asignado a las métricas que impactan directamente en la utilidad técnica del sistema.

| Métrica   | Descripción                                                                                                      | Rango | Peso |
|-----------|------------------------------------------------------------------------------------------------------------------|-------|------|
| Precisión | Evalúa si la respuesta proporcionada por el modelo es correcta y se ajusta a los datos o fragmentos del contexto | 0-2   | 0.25 |
| Cobertura | Valora cuánto de la información relevante del contexto ha sido recogida en la respuesta                          | 0-2   | 0.2  |
| Contexto  | Evalúa la capacidad del modelo para utilizar correctamente los fragmentos de contexto proporcionados             | 0-2   | 0.25 |

|             |                                                                                                                                                                           |     |     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|
| Concisión   | Puntúa la capacidad del modelo para responder de manera clara, directa y sin añadir información innecesaria                                                               | 0-1 | 0.1 |
| Alucinación | Valora si el modelo inventa datos, asume información no dada o responde con elementos ficticios                                                                           | 0-1 | 0.1 |
| Latencia    | Evalúa el tiempo que tarda el modelo en generar la respuesta ante una consulta. Dicha puntuación esta normalizada según la latencia máxima y mínima total de los modelos. | 0-1 | 0.1 |

Tabla 2. Descripción de la metodología empleada

Aunque las pruebas se han realizado en un entorno local con recursos limitados, se ha incluido la latencia como criterio de evaluación. Si bien los tiempos absolutos son algo elevados para un chat en tiempo real, la comparación relativa sirve como referencia y permite extrapolar qué modelos serán más eficientes en producción, manteniendo la proporcionalidad de diferencias.

La Tabla 3 muestra las puntuaciones obtenidas por cada modelo en cada pregunta, donde se ha aplicado un filtro de color para visualizar mejor el comportamiento de cada uno. Las puntuaciones se expresan según el rango indicado en la Tabla 2, es decir, de 0 a 2 para las métricas de precisión, cobertura y contexto, y de 0 a 1 para las restantes.

| Modelo    | Pregunta | Precisión | Cobertura | Concisión | Contexto | Alucinación | Latencia  | Total    |
|-----------|----------|-----------|-----------|-----------|----------|-------------|-----------|----------|
| Phi3      | 1        | 2         | 2         | 1         | 2        | 1           | 0.9447749 | 1.694477 |
| Phi3      | 2        | 2         | 2         | 1         | 2        | 1           | 0.9351544 | 1.693515 |
| Phi3      | 3        | 1         | 1         | 1         | 1        | 0           | 0.9348424 | 0.893484 |
| Phi3      | 4        | 1         | 1         | 1         | 1        | 1           | 0.9661069 | 0.996611 |
| Phi3      | 5        | 1         | 1         | 0         | 1        | 0           | 0.8354779 | 0.783548 |
| Phi3      | 6        | 0         | 0         | 0         | 0        | 0           | 0.5750426 | 0.057504 |
| TinyLlama | 1        | 0         | 0         | 1         | 0        | 1           | 0.9911528 | 0.299115 |
| TinyLlama | 2        | 0         | 0         | 1         | 0        | 0           | 1         | 0.2      |
| TinyLlama | 3        | 0         | 0         | 1         | 0        | 1           | 0.9946901 | 0.299469 |
| TinyLlama | 4        | 1         | 1         | 0         | 1        | 0           | 0.974991  | 0.797499 |
| TinyLlama | 5        | 1         | 1         | 1         | 1        | 0           | 0.9553154 | 0.895532 |
| TinyLlama | 6        | 0         | 0         | 0         | 0        | 0           | 0.9126418 | 0.091264 |
| Mistral   | 1        | 2         | 2         | 1         | 2        | 1           | 0.9075603 | 1.690756 |
| Mistral   | 2        | 1         | 1         | 1         | 1        | 1           | 0.9270818 | 0.992708 |
| Mistral   | 3        | 1         | 1         | 1         | 1        | 1           | 0.8720508 | 0.987205 |
| Mistral   | 4        | 2         | 2         | 1         | 2        | 1           | 0.9259092 | 1.692591 |
| Mistral   | 5        | 2         | 2         | 1         | 2        | 1           | 0.7269074 | 1.672691 |
| Mistral   | 6        | 2         | 2         | 1         | 2        | 1           | 0         | 1.6      |
| Llama3    | 1        | 2         | 2         | 1         | 2        | 1           | 0.9179749 | 1.691797 |
| Llama3    | 2        | 1         | 1         | 1         | 1        | 1           | 0.8777435 | 0.987774 |
| Llama3    | 3        | 2         | 2         | 1         | 2        | 1           | 0.8682374 | 1.686824 |
| Llama3    | 4        | 2         | 2         | 1         | 2        | 1           | 0.9407927 | 1.694079 |
| Llama3    | 5        | 2         | 2         | 1         | 2        | 1           | 0.7169424 | 1.671694 |
| Llama3    | 6        | 2         | 2         | 1         | 2        | 1           | 0.4137912 | 1.641379 |
| Zephyr    | 1        | 2         | 2         | 1         | 2        | 1           | 0.8860913 | 1.688609 |
| Zephyr    | 2        | 1         | 1         | 1         | 1        | 1           | 0.8334899 | 0.983349 |
| Zephyr    | 3        | 1         | 1         | 1         | 1        | 1           | 0.79035   | 0.979035 |
| Zephyr    | 4        | 2         | 2         | 1         | 2        | 1           | 0.8370458 | 1.683705 |
| Zephyr    | 5        | 2         | 2         | 1         | 2        | 1           | 0.6783703 | 1.667837 |
| Zephyr    | 6        | 1         | 1         | 1         | 1        | 1           | 0.2864613 | 0.928646 |

Tabla 3. Resultados por preguntas

El fichero Excel, con los resultados completos por pregunta, junto con las respuestas generadas por cada modelo y los fragmentos utilizados como contexto, se incluye como archivo adjunto adicional entregado junto con la presente memoria.

Los resultados finales agregados para cada modelo, en una escala normalizada de 0 a 10, se muestran en la siguiente gráfica:

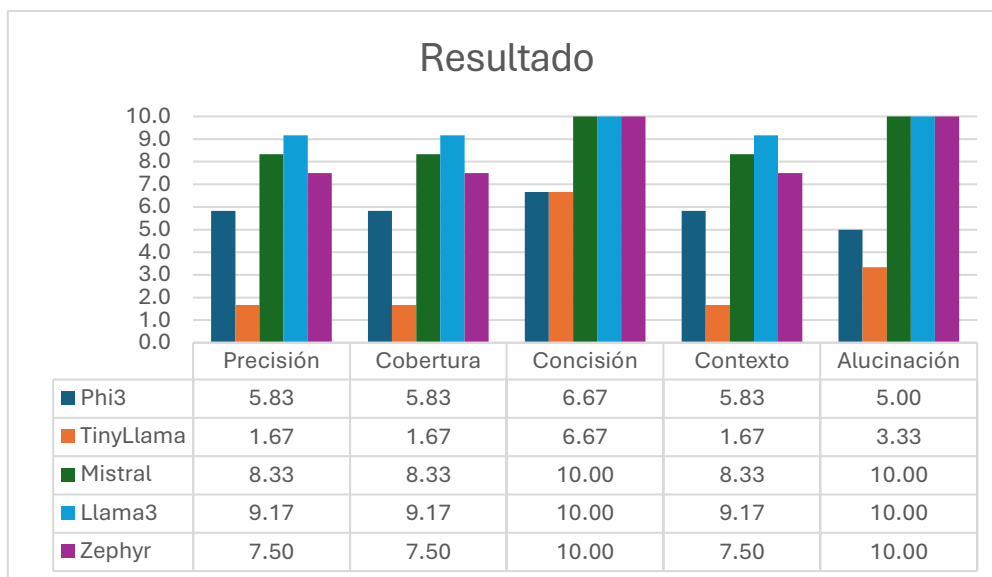


Figura 29. Gráfico de barras con los resultados obtenidos por cada modelo

Donde, puntuaciones más altas significan que el modelo se ha comportado mejor en esa métrica, destacando los modelos más potentes como Mistral, Llama3 o Zephyr.

Por otro lado, el resultado total ponderado se ilustra en la Fig. 30, cuyos resultados también están normalizados en una escala de 0 a 10.

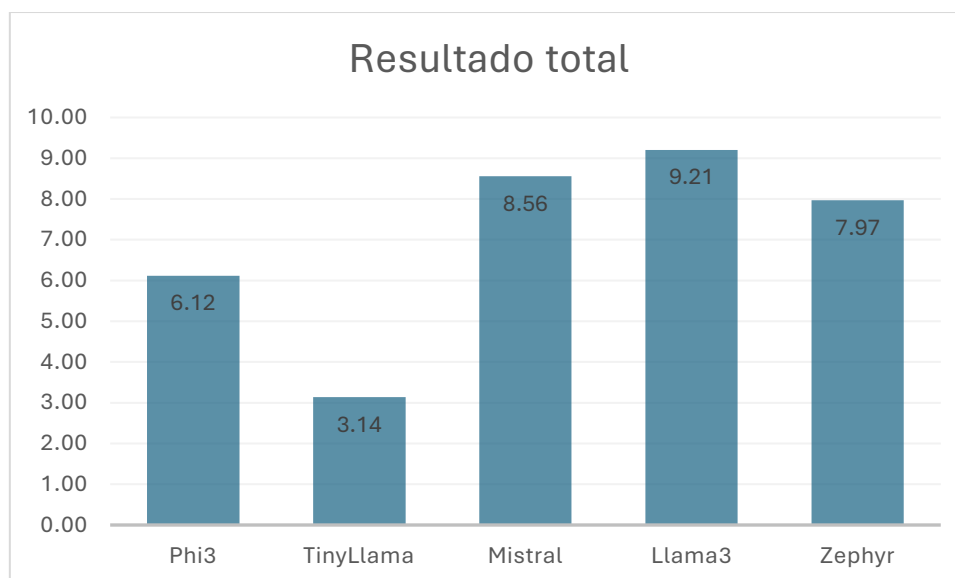


Figura 30. Resultados totales ponderados

Los resultados obtenidos muestran una diferencia clara de rendimiento entre los modelos evaluados. Llama3 lidera la comparativa con una puntuación global de 9.21 sobre 10, destacando por su equilibrio entre precisión, cobertura y uso contextual del documento, sin incurrir en alucinaciones. Le siguen Mistral y Zephyr, ofreciendo también respuestas fiables, aunque con menor rendimiento en algunos aspectos. A continuación, tenemos el modelo Phi3 con una puntuación de 6.12 que muestra dificultades en preguntas más complejas. Finalmente, tenemos el modelo más pequeño, TinyLlama con un 3.14 que, a pesar de ser el más rápido, es el que exhibe más limitaciones en todos los criterios.

Ante estos resultados, también se pueden observar las diferencias técnicas entre los modelos evaluados. Los modelos más pequeños como TinyLlama y Phi3 muestran un rendimiento notablemente inferior. Aunque ocupan menos espacio y son más rápidos, presentan limitaciones claras a la hora de interpretar contexto amplio en tareas complejas. En particular, TinyLlama, con solo 1.1 mil millones de parámetros y orientación tipo chat general, no está diseñado para consultas técnicas precisas, y Phi3, pese a ser tipo *instruct*, también se ve limitado por su menor número de parámetros.

Por el contrario, modelos como Mistral y Zephyr, y sobre todo Llama3, están mejor preparados para seguir instrucciones, gestionar múltiples fragmentos de información y ofrecer respuestas más completas, precisas y sin alucinaciones.

Por tanto, se concluye que los modelos Llama3 y Mistral son los más adecuados para el despliegue en el sistema propuesto, aunque, por su fiabilidad y rendimiento, Llama3 ha sido elegido modelo por defecto.

Por último, cabe destacar que los modelos evaluados han sido probados en un entorno local con recursos limitados, lo que condiciona tanto la latencia como el rendimiento. En un entorno de producción, sería posible emplear versiones más potentes o no cuantizadas de estos mismos modelos, o incluso modelos de mayor tamaño, lo que permitiría mejorar tanto la velocidad de respuesta como la precisión en tareas más complejas. La arquitectura del sistema es compatible con esta escalabilidad, por lo que se contempla la posibilidad de adaptar el modelo al entorno hardware disponible en cada caso.

## 6.2 Simulación de uso en entorno real

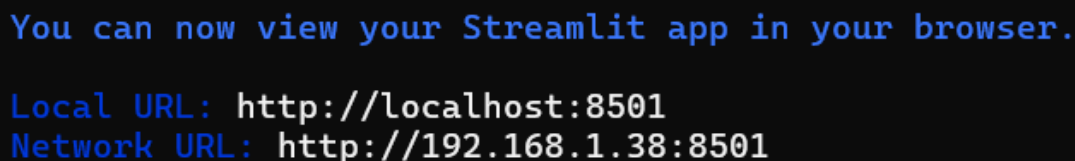
A continuación, se ha simulado el sistema en una versión funcional y operativa, cuyo objetivo es validar el comportamiento, comprobando su capacidad para localizar correctamente los fragmentos relevantes ante una consulta del usuario en un entorno real, así como generar y mostrar una respuesta coherente mediante el modelo LLM en la interfaz conversacional.

Además, se incluyen casos de prueba donde no existe información en los documentos CAP para verificar si el sistema es capaz de reconocer la falta de datos y de evitar alucinaciones. Este análisis permite evaluar no solo la precisión del sistema, sino también su robustez ante consultas reales en entornos de ingeniería.

En primer lugar, para iniciar la simulación en un entorno local de prueba, se debe ejecutar la API introduciendo el comando `python -m uvicorn backend.api.main:app --reload` en la terminal, lo que desplegará el backend en local. En paralelo, en otra terminal se lanza la interfaz web con `python -m streamlit run frontend/chatbot.py`, que abrirá el navegador por

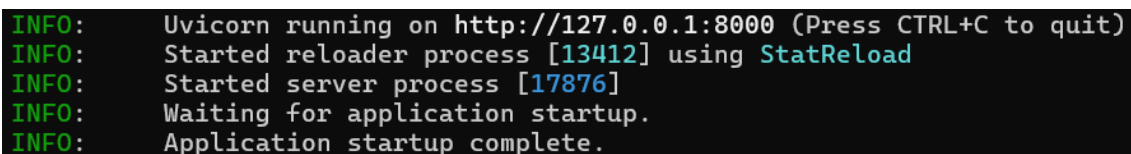


defecto con la aplicación del chatbot. Las Fig. 31 y Fig. 32 muestran los mensajes que confirman la correcta inicialización tanto de la API como del frontend.



```
You can now view your Streamlit app in your browser.  
  
Local URL: http://localhost:8501  
Network URL: http://192.168.1.38:8501
```

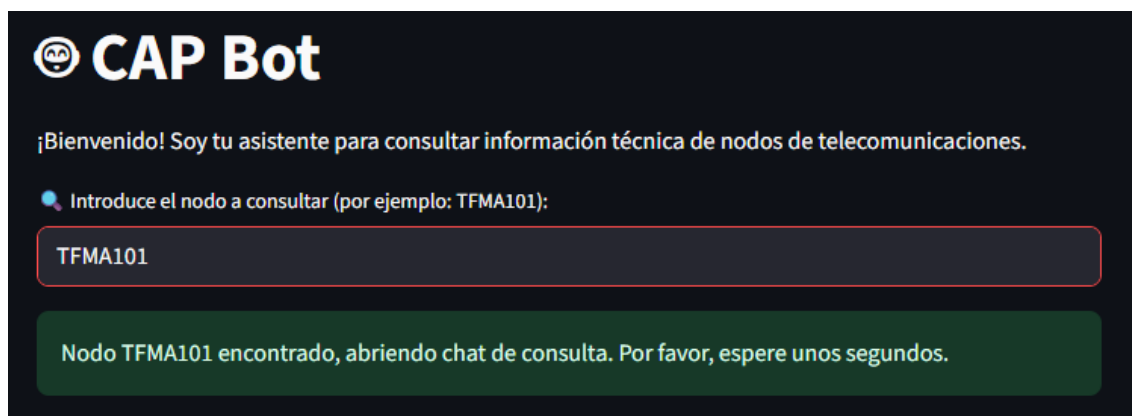
Figura 31. Inicio del entorno web del sistema.



```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO:      Started reloader process [13412] using StatReload  
INFO:      Started server process [17876]  
INFO:      Waiting for application startup.  
INFO:      Application startup complete.
```

Figura 32. Inicio del servidor local de la API mediante Uvicorn.

Una vez el sistema está activo, se introduce el identificador del nodo a consultar. Si dicho nodo está presente en la base de datos, se muestra un mensaje satisfactorio (véase Fig. 33) y se habilita el chat conversacional. Como primera prueba, se introduce la pregunta: ¿Qué modelos de antena y de RRU están instalados en el nodo?



**CAP Bot**

¡Bienvenido! Soy tu asistente para consultar información técnica de nodos de telecomunicaciones.

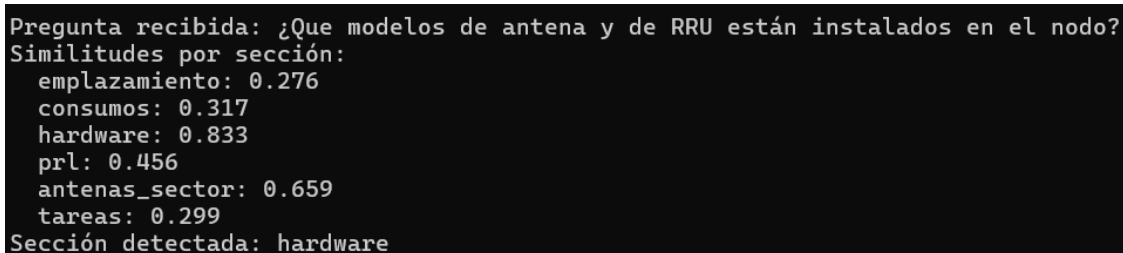
Introduce el nodo a consultar (por ejemplo: TFMA101):

TFMA101

Nodo TFMA101 encontrado, abriendo chat de consulta. Por favor, espere unos segundos.

Figura 33. Interfaz web tras validar el nodo introducido

En la terminal de la API se puede seguir el proceso paso a paso: se calcula la similitud de la pregunta con cada sección del documento, se identifica correctamente la sección “hardware” como la más relevante (véase Fig. 34) y se recuperan los cuatro fragmentos más similares. Estos fragmentos son utilizados como contexto en el *prompt* que se envía al modelo LLM (véase Fig. 35).



```
Pregunta recibida: ¿Que modelos de antena y de RRU están instalados en el nodo?  
Similitudes por sección:  
emplazamiento: 0.276  
consumos: 0.317  
hardware: 0.833  
prl: 0.456  
antenas_sector: 0.659  
tarefas: 0.299  
Sección detectada: hardware
```

Figura 34. Selección automática de la sección más relevante según similitud semántica

```
Fragmentos utilizados en el prompt:  
Fragmento 1: El nodo dispone de 4 antenas (modelos: ASI4518R39v07, AIR 6419 3,5 B78Y 5G, A04220PA01v06), 5 RRUs (...  
Fragmento 2: RRU modelo RRU 2479 B8/B20/B28, tipo de conector: 4.3-10....  
Fragmento 3: RRU modelo RRU 11 B7, tipo de conector: 4.3-10....  
Fragmento 4: RRU modelo RRU 4486 B8/B20/B28, tipo de conector: 4.3-10....  
Prompt construido correctamente. Enviando al modelo...
```

*Figura 35. Fragmentos recuperados del índice FAISS y utilizados como contexto.*

La respuesta generada (véase Fig. 36) demuestra que el sistema extrae correctamente la información relevante, la presenta de forma clara y estructurada, y además reconoce los elementos para los que no se dispone de información, evitando generar contenido ficticio. Esto refuerza la solidez del sistema ante preguntas técnicas complejas. También se observa que el formato de presentación es adecuado para usuarios no especializados, cumpliendo así uno de los objetivos clave del diseño.

La respuesta del modelo podría ser más exacta si ampliamos el número de fragmentos utilizados o compactamos más la información. Sin embargo, se puede concluir que el sistema funciona correctamente, aportando respuestas coherentes al contexto proporcionado.

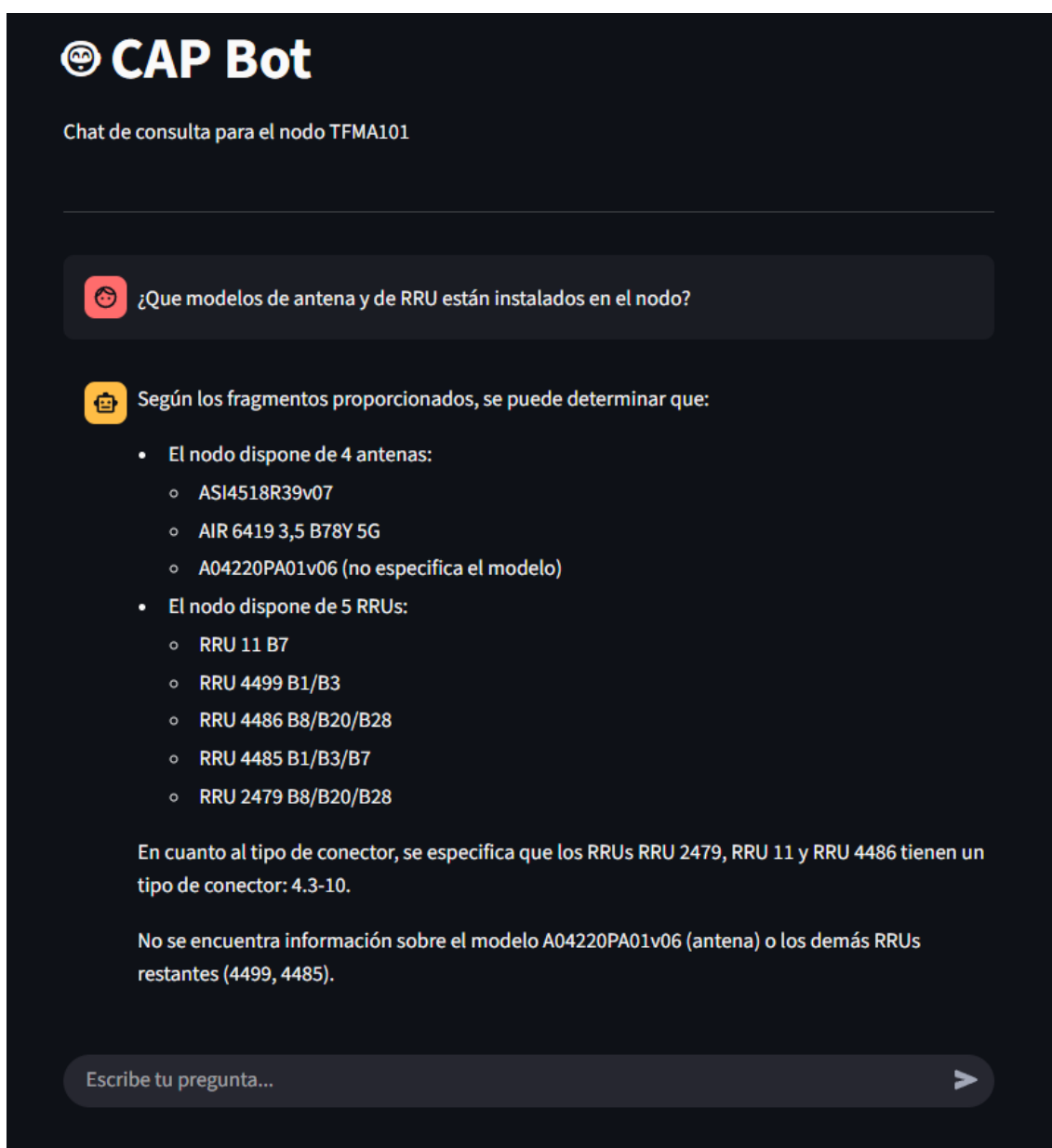


Figura 36. Respuesta generada por el modelo LLM a partir de los fragmentos seleccionados

Posteriormente, se introduce una pregunta para la cual no existe información en los documentos: ¿Cuántos usuarios se conectan al nodo en hora punta? El sistema, en este caso, reconoce la ausencia de datos relevantes en la base documental y devuelve una respuesta informativa indicando dicha carencia, sin inventar valores ni suposiciones técnicas (véase Fig. 37).

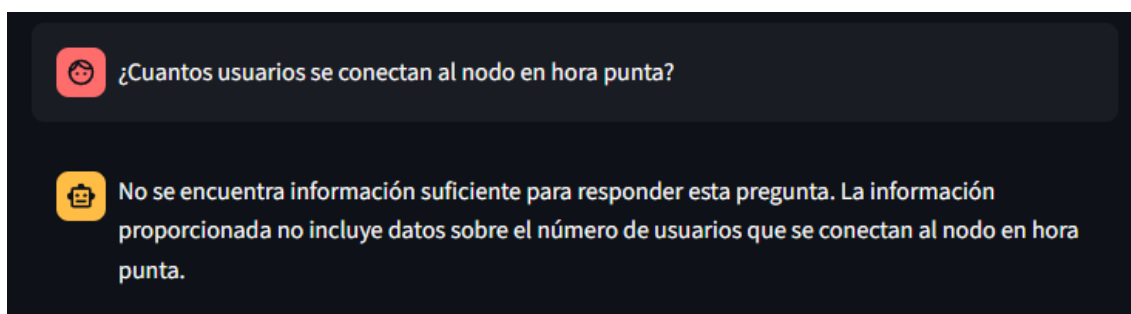


Figura 37. Respuesta del sistema ante una consulta sin información disponible

Por lo que, se puede concluir que la interfaz web, así como el sistema desplegado en un entorno de pruebas de uso real funciona correctamente. Destacando la importancia de la no alucinación del modelo, aportando siempre valores verídicos que se encuentran en los fragmentos proporcionados como contexto.

En el Anexo B se incluye el manual de usuario, el cual proporciona una guía detallada para la ejecución del sistema. Este manual recoge tanto los requisitos del entorno como los pasos necesarios para su despliegue, facilitando así la utilización y escalabilidad del sistema por parte de otros usuarios o desarrolladores.

### 6.3 Limitaciones observadas y fiabilidad del sistema

Durante las pruebas y la simulación final, el sistema ha demostrado un comportamiento sólido, siendo capaz de recuperar información técnica relevante y generar respuestas coherentes. No obstante, se han identificado ciertas limitaciones propias tanto del proceso de recuperación como de la fase de generación, que deben ser tenidas en cuenta para futuras iteraciones o entornos reales de producción.

En cuanto a la recuperación de fragmentos, se ha comprobado que el sistema asocia correctamente la mayoría de las preguntas con la sección correspondiente del documento CAP. Sin embargo, cuando la información necesaria para responder está dispersa entre múltiples fragmentos no contiguos, el índice vectorial puede recuperar solo una parte de ellos, especialmente si se utiliza un valor reducido de  $k$ . Asimismo, si un fragmento es demasiado largo, puede truncarse en el proceso, ya que cada fragmento está limitado a 100 tokens, perdiendo parte del contexto útil. Este se puede ampliar, pero podría introducir más ruido o superar la limitación de tokens del modelo.

Respecto a la generación de respuestas, los modelos LLM empleados han mostrado una buena capacidad de síntesis, claridad y adecuación técnica cuando los fragmentos de entrada son adecuados. No obstante, en situaciones donde los fragmentos contienen información incompleta o formatos poco homogéneos, pueden producirse errores leves o interpretaciones incompletas.

Aunque sí que se pueden visualizar los fragmentos origen como parte de la respuesta en la terminal de la API, podrían mostrarse también al usuario para mejorar la transparencia del sistema, o incluir un modo usuario y un modo desarrollador, por ejemplo.

Por último, en relación con el entorno de ejecución, el sistema funciona de forma satisfactoria en entorno local, tanto a nivel de API como de interfaz web, con latencias aceptables para un entorno de pruebas (1-2 minutos por respuestas). Sin embargo, este rendimiento no puede extrapolarse



directamente a un entorno productivo con múltiples usuarios concurrentes o con volúmenes mayores de documentos.

La carga de memoria y el uso de CPU o GPU local puede ser un cuello de botella si el sistema escala. Como línea de mejora, se plantea su despliegue en un servidor con recursos dedicados, el uso de modelos más potentes adaptados a producción o incluso la integración de una base de datos vectorial más eficiente, como Qdrant o Weaviate.

En conjunto, el sistema ofrece resultados satisfactorios en cuanto a fiabilidad, precisión y claridad, pero existen márgenes de mejora tanto en la fase de recuperación como en la eficiencia de ejecución, que serán clave para su adopción en entornos reales.

## Capítulo 7. Conclusiones y trabajo futuro

### 7.1 Conclusiones

El presente Trabajo Fin de Máster ha demostrado la viabilidad de un sistema de extracción y consulta conversacional de información técnica contenida en documentos CAP, habituales en el diseño de nodos de redes móviles. Partiendo de una necesidad real identificada y teniendo en cuenta los condicionantes del entorno, se ha desarrollado una solución modular, ejecutable en entornos locales, y basada en tecnologías de código abierto.

El sistema propuesto cumple con los objetivos planteados inicialmente:

- Se ha implementado un pipeline de extracción robusto, capaz de extraer automáticamente la información técnica contenida en los documentos CAP mediante técnicas de OCR zonal adaptadas a distintas plantillas. Posteriormente, los datos se estructuran, normalizan y anonimiza, garantizando así la confidencialidad y estandarización necesaria para su posterior consulta.
- Se ha desarrollado un módulo RAG basado en FAISS y *embeddings* semánticos, que permite identificar la sección relevante del documento en función de la consulta del usuario y recuperar los fragmentos más adecuados mediante técnicas de vectorización e indexación contextual.
- Se ha desarrollado una interfaz conversacional web o chatbot utilizando la librería Streamlit. Esta interfaz es sencilla, lo que facilita su uso a cualquier tipo de perfil, y se conecta al sistema a través de una API diseñada en FastAPI, que actúa como puente entre el frontend y el backend.
- Se ha seguido una metodología de evaluación comparativa de distintos modelos LLM de código abierto, ejecutados en entorno local a través de Ollama. Los resultados de esta evaluación se documentan en el Capítulo 6.1.
- Finalmente, el sistema ha sido simulado en un entorno de uso real, donde se ha comprobado su rendimiento en términos de precisión semántica, calidad de respuestas y trazabilidad. Los resultados muestran que el sistema es capaz de ofrecer respuestas precisas y coherentes, basadas en los fragmentos extraídos, sin incurrir en errores de interpretación ni generar alucinaciones.

Durante la fase de validación, el sistema ha sido capaz de resolver consultas reales con un nivel de precisión y coherencia adecuado, incluso operando en entornos con recursos limitados. Además, el diseño modular y reproducible del pipeline permite su adaptación a otros tipos de documentos técnicos o a dominios industriales con necesidades similares.

En definitiva, este trabajo no solo aborda un problema práctico concreto, sino que demuestra cómo técnicas de inteligencia artificial y procesamiento de lenguaje pueden integrarse eficazmente en entornos profesionales, contribuyendo a la digitalización de procesos en el ámbito de la ingeniería de telecomunicaciones.

## 7.2 Trabajo futuro

A pesar de los resultados satisfactorios alcanzados, existen diversas líneas de mejora y expansión del sistema:

1. **Mejora del OCR y detección de plantillas:** aunque el sistema es robusto ante variaciones, podrían integrarse modelos OCR más avanzados para mejorar la extracción estructurada, como el comentado PrepOCR. Además, se podría extender el sistema para detectar más tipos de plantillas u otros documentos.
2. **Optimización del módulo RAG:** aunque el sistema actual ofrece una recuperación semántica funcional, se podría mejorar su precisión mediante técnicas de *re-ranking* con modelos tipo *cross-encoder*, *embeddings* entrenados en lenguaje técnico, y una segmentación más contextual. Estas mejoras permitirían una selección de fragmentos más coherente y alineada con la intención de la consulta, aumentando la calidad de las respuestas generadas por el modelo LLM.
3. **Escalabilidad del sistema:** el sistema actual está orientado a un entorno local y controlado. No obstante, podría evolucionarse hacia una arquitectura distribuida basada en microservicios, que permita procesar múltiples documentos en paralelo y almacenar los resultados en bases de datos NoSQL (como MongoDB). Esta evolución facilitaría su integración en entornos corporativos con volúmenes elevados de documentos y necesidades de consulta simultánea desde diferentes equipos.
4. **Entrenamiento de un modelo de lenguaje propio:** si se dispone de un volumen suficiente de documentación, se podría entrenar o afinar un modelo de lenguaje específico para el dominio técnico de diseño de redes móviles. Este modelo adaptado podría superar a modelos generales en precisión, comprensión del vocabulario técnico, y contextualización de consultas frecuentes, aportando valor estratégico al equipo.
5. **Ampliación de funcionalidades del chatbot o evolución a un agente de IA:** ligado al punto anterior, con un modelo propio entrenado, el sistema podría evolucionar hacia un asistente técnico proactivo, capaz de sugerir configuraciones o detectar inconsistencias en los diseños a partir de los documentos CAP o configuraciones actuales de los nodos, ahorrando tiempo al equipo de diseño y actuando como herramienta soporte para los compañeros de diseño.
6. **Interfaz multilingüe y multiformato:** para entornos internacionales, sería útil incorporar soporte multilingüe y compatibilidad con otros formatos de entrada, como documentos en formato DOCX o HTML.

Con estas líneas de mejora, el sistema podrá evolucionar de una herramienta de consulta puntual a un asistente integral de apoyo al diseño e ingeniería de redes, alineado con los objetivos de automatización y digitalización del sector.



## REFERENCIAS

- [1] Accenture. (2022). *Sistema radiante en redes móviles: conceptos y parámetros técnicos*. [Documento interno no publicado].
- [2] Ministerio de la Presidencia. (2001, 28 de septiembre). *Real Decreto 1066/2001, de 28 de septiembre, por el que se aprueba el Reglamento que establece condiciones de protección del dominio público radioeléctrico, restricciones a las emisiones radioeléctricas y medidas de protección sanitaria frente a emisiones radioeléctricas*. Boletín Oficial del Estado, 234, 39051-39064. <https://www.boe.es/eli/es/rd/2001/09/28/1066/con> boe.es
- [3] Ministerio de Ciencia y Tecnología. (2002, 11 de enero). *Orden CTE/23/2002, de 11 de enero, por la que se establecen condiciones para la presentación de determinados estudios y certificaciones por operadores de servicios de radiocomunicaciones*. Boletín Oficial del Estado, 11, 1613-1624. <https://www.boe.es/eli/es/o/2002/01/11/cte23/con>
- [4] Qaroush, Aziz & Awad, Abdalkarim & Modallal, Mohammad & Ziq, Malik. (2020). *Segmentation-based, omnifont printed Arabic character recognition without font identification*. Journal of King Saud University - Computer and Information Sciences. 34. 10.1016/j.jksuci.2020.10.001.
- [5] Guan, D., Xu, M., Lu, J., & Duan, N. (2025). *PreP-OCR: A Complete Pipeline for Document Image Restoration and Enhanced OCR*. arXiv. <https://doi.org/10.48550/arXiv.2505.20429>
- [6] Sinha, A., & Rekha, D. R. (2025). *Digitization of Document and Information Extraction using OCR*. arXiv. <https://doi.org/10.48550/arXiv.2506.11156>
- [7] Jurafsky, D., & Martin, J. H. (2025). *Speech and Language Processing* (3rd ed.). Draft online manuscript. Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- [8] Shamshiri, S., Karimi, H., Sharma, S., & Mahmoudi, M. R. (2024). *Survey on Latest Advances in Natural Language Processing*. WIREs Data Mining and Knowledge Discovery. <https://doi.org/10.1002/widm.1495>
- [9] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv. <https://arxiv.org/abs/1908.10084>
- [10] Devopedia. (2023). *Word embedding*. Consultado en julio de 2025 del blog Devopedia. <https://devopedia.org/word-embedding>
- [11] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv. <https://arxiv.org/abs/1810.04805>
- [12] Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*. arXiv. <https://arxiv.org/abs/2005.14165>
- [13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention Is All You Need*. arXiv. <https://arxiv.org/abs/1706.03762>
- [14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv. <https://arxiv.org/abs/2305.14314>



- [15] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Fan, A., Bhosale, S., Chaudhary, V., Luo, Y., Stoyanov, V., & Riedel, S. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv. <https://arxiv.org/abs/2005.11401>
- [16] Gao, M., Zhang, J., Wu, H., Xu, B., Xu, J., Zhang, T., Liu, Z., & Liu, L. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv. <https://arxiv.org/abs/2312.10997>
- [17] Zhu, Jinting & Jang-Jaccard, Julian & Welch, Ian & Al-Sahaf, Harith & Camtepe, Seyit. (2022). *A Ransomware Triage Approach using a Task Memory based on Meta-Transfer Learning Framework*. 10.48550/arXiv.2207.10242.
- [18] Gupta, P., Joshi, R., Yadav, R., Shrivastava, M., & Varma, V. (2024). *A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions*. arXiv. <https://arxiv.org/abs/2410.12837>
- [19] Accenture. (2022). *Ejemplos AR y CAP*. [Documento interno no publicado].
- [20] UB Mannheim. (s.f.). *Tesseract OCR for Windows* [Repositorio GitHub]. GitHub. <https://github.com/UB-Mannheim/tesseract/wiki>
- [21] oschwartz10612. (s.f.). *poppler-windows* [Repositorio GitHub]. GitHub. <https://github.com/oschwartz10612/poppler-windows/releases>

## ANEXOS

### Anexo A. Código complementario

Este anexo muestra y documenta brevemente el funcionamiento de los scripts y módulos desarrollados que no fueron descritos en detalle en el Capítulo 5.

#### A.1 Módulos auxiliares de extracción (utils y parsers)

Los módulos auxiliares agrupan funciones de soporte utilizadas en el pipeline de extracción, estructuración y procesamiento de la información contenida en los documentos CAP. Estas funciones no forman parte del núcleo del sistema, pero permiten extraer valores específicos, limpiar datos o preparar estructuras intermedias.

##### A.1.1 Módulo parseadores

Este grupo de scripts se encarga de localizar y extraer valores concretos del texto obtenido por OCR, como identificadores, coordenadas geográficas o bloques técnicos. Se utilizan expresiones regulares y reglas adaptadas al formato habitual de los CAP.

- **parse\_identificadores.py (Fig. 38):** extrae campos clave del nodo como código, dirección, municipio o provincia, a partir de fragmentos textuales. Estas variables son esenciales para identificar de forma unívoca cada documento.
- **parse\_coordenadas.py (Fig. 39):** detecta y normaliza coordenadas geográficas (GMS y UTM), incluyendo la conversión entre sistemas de referencia, para facilitar su uso posterior en mapas o bases de datos.
- **parse\_tablas.py:** implementa funciones específicas para procesar el contenido de tablas detectadas en el documento, como el inventario de hardware, antenas, consumos eléctricos o medidas PRL. Devuelve estructuras limpias y organizadas. Por su extensión, el código no se ha incluido en el presente trabajo.
- **procesar\_trabajos.py:** agrupa las tareas técnicas extraídas por OCR, clasificándolas en acciones como *instalación*, *retirada* o *sustitución*, según la empresa responsable. Permite organizar la información relativa a trabajos de campo de forma estructurada.

```
# parsers/parse_identificadores.py
import re

def parsear_identificadores_A(texto):
    resultado = {}
    lineas = [line.strip() for line in texto.strip().split('\n') if line.strip()]
    for i in range(len(lineas) - 1):
        etiqueta = lineas[i].upper()

        if "DIRECCION" in etiqueta:
            resultado["Direccion"] = lineas[i - 1].strip()

        elif "MUNICIPIO" in etiqueta and "PROVINCIA" in etiqueta:
            partes = lineas[i - 1].split()
            resultado["Provincia"] = partes[-1].strip()
            resultado["Municipio"] = " ".join(partes[:-1]).strip()

    match_nodo = re.search(r'([A-Z]{3}\d{4})[_](\S+)', texto.upper())
    if match_nodo:
        resultado["Nodo"] = match_nodo.group(1)
        resultado["Cluster"] = match_nodo.group(2)

    return resultado

def parsear_identificadores_B(texto):
    resultado = {}
    texto = texto.upper().replace("'", " ").replace('"', " ")
    texto = texto.replace(" ", " ").replace("\n", " ")

    match_nodo = re.search(r'([A-Z]{3}\d{4})[_](\S+)', texto.upper())
    if match_nodo:
        resultado["Nodo"] = match_nodo.group(1)
        resultado["Cluster"] = match_nodo.group(2)

    match_dir = re.search(r'DIRECCION[:\s]+([A-Z0-9\-\.\, ]{5,50})', texto)
    if match_dir:
        resultado['Direccion'] = match_dir.group().strip()

    match_mun = re.search(r'LOCALIDAD[:\s]+([A-Z ]+)', texto)
    if match_mun:
        resultado['Municipio'] = match_mun.group(1).strip()

    match_prov = re.search(r'PROVINCIA[:\s]+([A-Z ]+)', texto)
    if match_prov:
        resultado['Provincia'] = match_prov.group(1).strip()

    return resultado

def parsear_identificadores_emplazamiento(texto):
    resultado = parsear_identificadores_A(texto)
    claves_clave = ['Direccion', 'Municipio', 'Provincia']

    if not all(k in resultado for k in claves_clave):
        resultado = parsear_identificadores_B(texto)

    return resultado
```

Figura 38. Script parse\_identificadores.py

```
# parsers/parse_coordenadas.py
import re
from backend.etl.utils.coord_utils import utm_a_gms
import webbrowser

def extraer_lat_lon(texto):
    matches = re.findall(r'\d{1,3}[^NSEW0nsew\r\n]{1,20}[NSEW00nsew]', texto)
    lat, lon = None, None
    for m in matches:
        m = m.strip().replace(' ', '')
        dir_final = m[-1].upper()

        if dir_final in ['N', 'S'] and lat is None:
            lat = m
        elif dir_final in ['E', 'O', 'W', '0'] and lon is None:
            if dir_final == 'O' or dir_final == '0':
                m = m[:-1] + 'E'
            elif dir_final == 'W':
                m = m[:-1] + 'W'
            lon = m

    return lat, lon

def parsear_coordenadas(texto, abrir_mapa = False):
    texto = texto.replace("\n", " ").replace(" ", " ")
    print(texto)
    resultado = {}

    lat_ocr, lon_ocr = extraer_lat_lon(texto)

    match_x = re.search(r'X[:]=?\s*([\d\.,]+)', texto)
    match_y = re.search(r'Y[:]=?\s*([\d\.,]+)', texto)
    if match_x:
        resultado["UTM_X"] = match_x.group(1).replace(",", ".")
    if match_y:
        resultado["UTM_Y"] = match_y.group(1).replace(",", ".")

    match_huso = re.search(r'HUSO\s*(?:UTM)?[:]=?\s*([0-9]{1,2}[A-Z]?)', texto.upper())
    if match_huso:
        resultado["UTM_HUSO"] = match_huso.group(1).strip()

    match_cota = re.search(r'COTA(?:\s+DE\s+TERRENO)?[:]=?\s*(\d+)', texto, re.IGNORECASE)
    if match_cota:
        resultado["Cota"] = match_cota.group(1) + "m"

    # Verificación cruzada con GMS
    if all(k in resultado for k in ["UTM_X", "UTM_Y", "UTM_HUSO"]):
        lat_gms, lon_gms = utm_a_gms(resultado)
        def direccion(val): return val[-1] if val else None

        # Forzar letra OCR si no coincide
        if lat_gms and lat_ocr and direccion(lat_gms) != direccion(lat_ocr):
            lat_gms = lat_gms[:-1] + direccion(lat_ocr)
            print(lat_gms)
        if lon_gms and lon_ocr and direccion(lon_gms) != direccion(lon_ocr):
            lon_gms = lon_gms[:-1] + direccion(lon_ocr)
            print(lon_gms)

        resultado["Latitud"] = lat_gms
        resultado["Longitud"] = lon_gms

    if abrir_mapa:
        webbrowser.open(f"https://www.google.com/maps/search/?api=1&query={lat_gms},{lon_gms}")

    return resultado
```

Figura 39. Script parse\_coordenadas.py

### A.1.2 Módulo de utilidades

Este grupo de scripts proporciona funciones genéricas que se reutilizan en distintas partes del sistema, desde la lectura del PDF hasta el formateo de coordenadas o conversiones básicas.

- **coord\_utils.py (Fig. 40):** contiene funciones matemáticas para convertir coordenadas UTM a GMS, y viceversa. Incluye validaciones y detección de formatos incorrectos.
- **pdf\_utils.py (Fig. 41):** ofrece utilidades para abrir y procesar documentos PDF, gestionar sus páginas y extraer regiones específicas. Se utiliza como apoyo a la extracción OCR zonal.

```
# utils/coord_utils.py
import re
import utm

def decimal_a_gms(valor, latitud=True):
    grados = int(abs(valor))
    minutos_float = (abs(valor) - grados) * 60
    minutos = int(minutos_float)
    segundos = round((minutos_float - minutos) * 60, 1)
    direccion = "N" if latitud and valor >= 0 else "S" if latitud else "E" if valor >= 0 else "W"
    return f"{grados}°{minutos}'{segundos}\"{direccion}"

def utm_a_gms(coord_dict):
    try:
        x = float(coord_dict["UTM_X"].replace(',', '.'))
        y = float(coord_dict["UTM_Y"].replace(',', '.'))
        huso_raw = coord_dict.get("UTM_HUSO", "31N").strip().upper()

        zone_number = int(re.match(r'\d{1,2}', huso_raw).group())
        zone_letter_match = re.search(r'[C-HJ-NP-X]', huso_raw)
        zone_letter = zone_letter_match.group() if zone_letter_match else 'N'

        lat_decimal, lon_decimal = utm.to_latlon(x, y, zone_number, zone_letter)
        lat_gms = decimal_a_gms(lat_decimal, True)
        lon_gms = decimal_a_gms(lon_decimal, False)

        return lat_gms, lon_gms
    except Exception as e:
        print("Error al convertir coordenadas UTM a GMS:", e)
        return None, None
```

Figura 40. Script coord\_utils.py

```
# utils/pdf_utils.py
import pdfplumber
from backend.etl.ocr.ocr_text import extraer_texto_ocr_por_zona

def encontrar_pagina_por_titulo(doc, rect, titulos_esperados, mostrar_imagen=False):
    """
    Busca en el documento la primera página cuyo título (extraído por OCR) coincida con alguno de los esperados.
    """
    for i, page in enumerate(doc):
        titulo = extraer_texto_ocr_por_zona(page, rect, mostrar_imagen=mostrar_imagen)
        titulo = " ".join(titulo.strip().split())
        print(titulo)
        if any(p in titulo.upper() for p in titulos_esperados):
            return i
    return None

def extraer_tablas_de_pagina(pdf_path, pagina_idx):
    """
    Extrae tablas usando pdfplumber desde la página indicada.
    """
    with pdfplumber.open(pdf_path) as pdf:
        page = pdf.pages[pagina_idx]
        tablas = page.extract_tables()
    return tablas
```

Figura 41. Script pdf\_utils.py

## A.2 Funciones complementarias del script de normalización

En este apartado muestra y explica brevemente las funciones desarrolladas para normalizar los distintos bloques extraídos del documento CAP.

- **normalizar\_consumos**: toma como entrada la tabla de consumos eléctricos procesada y convierte los valores en un formato numérico unificado. Identifica campos como potencia total actual o prevista, y elimina símbolos, unidades o guiones que puedan interferir en el análisis posterior (véase Fig. 42).
- **normalizar\_antenas**: reestructura las antenas detectadas en el documento, organizándolas por sector, y luego incluye los campos tipo, tecnologías, tilt o azimuth (véase Fig. 43).
- **normalizar\_hardware**: procesa los elementos detectados en las tablas de hardware, agrupando modelos, fabricantes, tipos de equipos y cantidades. Establece claves unificadas y corrige posibles errores, permitiendo que la información de hardware sea coherente entre documentos de distintas plantillas (véase Fig. 44).
- **normalizar\_prl**: estandariza la información relativa a prevención de riesgos laborales. Debido a su simplicidad no se ha mostrado su código.



```
def normalizar_consumos(consumos_raw):
    resultado = {
        "actual": {"total": None, "detalle": {}, "tecnologias": []},
        "reformado": {"total": None, "detalle": {}, "tecnologias": []}
    }

    def limpiar_valor(v):
        if isinstance(v, str):
            v = v.replace("W", "").replace("--", "0").strip()
        try:
            return float(v)
        except:
            return 0.0

    def parse_tecnologias(data):
        techs = data.get("TECNOLOGIAS\nEXISTENTES", "")
        techs = techs.replace("\n", " ").replace(",", " ")
        techs = re.split(r"[+~/-]", techs)
        techs = [t.strip().replace("OSP", "").replace("VDF", "") for t in techs if t.strip()]
        return sorted(set(techs))

    # Tipo A1: tiene claves "ACTUAL (W)" y "FINAL (W)"
    if any("ACTUAL (W)" in v for v in consumos_raw.values() if isinstance(v, dict)):
        for k, bloque in consumos_raw.items():
            if not isinstance(bloque, dict):
                continue
            clave_norm = k.lower().replace(" ", "_")
            if clave_norm == "total":
                resultado["actual"]["total"] = limpiar_valor(bloque.get("ACTUAL (W)", 0))
                resultado["reformado"]["total"] = limpiar_valor(bloque.get("FINAL (W)", 0))
            else:
                resultado["actual"]["detalle"][clave_norm] = limpiar_valor(bloque.get("ACTUAL (W)", 0))
                resultado["reformado"]["detalle"][clave_norm] = limpiar_valor(bloque.get("FINAL (W)", 0))

    # Tipo A2: contiene "Actual" y "Reformado" como claves principales
    elif "Actual" in consumos_raw:
        actual = consumos_raw.get("Actual", {})
        reformado = consumos_raw.get("Reformado", {})
        resultado["actual"]["total"] = limpiar_valor(actual.get("CONSUMO", 0))
        resultado["reformado"]["total"] = limpiar_valor(reformado.get("CONSUMO", 0))
        resultado["actual"]["tecnologias"] = parse_tecnologias(actual)
        resultado["reformado"]["tecnologias"] = parse_tecnologias(reformado)

    # Tipo A3: formato simple con clave "ESTADO" y "CONSUMO"
    elif all(isinstance(v, dict) and "CONSUMO" in v for v in consumos_raw.values()):
        for estado, bloque in consumos_raw.items():
            estado_norm = estado.strip().lower()
            if "actual" in estado_norm:
                resultado["actual"]["total"] = limpiar_valor(bloque.get("CONSUMO", 0))
            elif "reformado" in estado_norm:
                resultado["reformado"]["total"] = limpiar_valor(bloque.get("CONSUMO", 0))

    return resultado
```

Figura 42. Función normalizar consumos

```
def normalizar_antenas(antenas_raw):
    def extraer_cantidad(valor):
        if isinstance(valor, str):
            valor = valor.strip()
            if valor.isdigit():
                return int(valor)
            if "x" in valor:
                try:
                    return int(valor.lower().split("x")[0].strip())
                except:
                    return 1
        try:
            return int(valor)
        except:
            return 1

    def procesar_antena(ant):
        def extraer_longitud(valor):
            if isinstance(valor, dict):
                return valor.get("valor")
            if isinstance(valor, str):
                partes = valor.lower().replace("mm", "").replace("m", "").split("x")
                try:
                    return float(partes[-1])
                except:
                    return None
            return None

        modelo = ant.get("MODELO") or ant.get("TIPO_ANTENA")
        tipo = "Activa" if (modelo and "AAU" in modelo) or ant.get("COAX_N") in ["", None, 0] else "Pasiva"
        altura = ant.get("SUELO") or ant.get("ALTURA_TOP_ANT") or ant.get("CUBIERTA")

        return {
            "modelo": modelo,
            "tipo": tipo,
            "orientacion": ant.get("AZIMUT") or ant.get("ORIENTACION"),
            "altura_top": altura,
            "edt": ant.get("EDT"),
            "coax": {
                "cantidad": extraer_cantidad(ant.get("COAX_N")) if "COAX_N" in ant else extraer_cantidad(ant.get("COAX_NTIP0",
                ant.get("TIPO", ""))),
                "tipo": ant.get("COAX_TIP0") or ant.get("TIPO") or ant.get("COAX_NTIP0", "").replace("\\", ""),
                "longitud": extraer_longitud(ant.get("COAX_LONG") or ant.get("LONG"))
            },
            "fo": {
                "cantidad": extraer_cantidad(ant.get("FO_N")) if "FO_N" in ant else extraer_cantidad(ant.get("FO_LONG")),
                "longitud": extraer_longitud(ant.get("FO_LONG") or ant.get("F.O. LONG"))
            },
            "vcc": {
                "cantidad": extraer_cantidad(ant.get("VCC_N")) if "VCC_N" in ant else 1,
                "longitud": extraer_longitud(ant.get("VCC_LONG") or ant.get("CC. LONG"))
            }
        }

    resultado = {}
    for sector, antenas in antenas_raw.items():
        agrupadas = defaultdict(lambda: {"tecnologias": []})

        for ant in antenas if isinstance(antenas, list) else list(antenas.values()):
            clave = (
                ant.get("TIPO_ANTENA") or ant.get("MODELO"),
                ant.get("ORIENTACION") or ant.get("AZIMUT"),
                ant.get("ALTURA_TOP_ANT") or ant.get("SUELO") or ant.get("CUBIERTA"),
                ant.get("EDT"),
                ant.get("COAX_NTIP0") or ant.get("TIPO"),
                ant.get("COAX_LONG"),
                ant.get("FO_LONG"),
                ant.get("VCC_LONG")
            )
            antena_proc = procesar_antena(ant)
            agrupadas[clave].update(antena_proc)
            tecnologia = ant.get("TECNOLOGIA") or ant.get("TECNOLOGIAS")
            if isinstance(tecnologia, str):
                techs = [t.strip().replace("OSP", "").replace("VDF", "") for t in re.split(r"[+,/]", tecnologia) if len(t.strip()) > 1]
            elif isinstance(tecnologia, list):
                techs = [t.strip().replace("OSP", "").replace("VDF", "") for t in tecnologia]
            else:
                techs = []
            agrupadas[clave]["tecnologias"].extend([t for t in techs if t])

        for k in agrupadas:
            agrupadas[k]["tecnologias"] = sorted(set(agrupadas[k]["tecnologias"]))

        resultado[sector.upper()] = list(agrupadas.values())

    return resultado
```

Figura 43. Función normalizar antenas

```
def normalizar_hardware(hardware_raw):  
    hw = {  
        "antenas": [],  
        "rru": [],  
        "bbu": None,  
        "conectividad": hardware_raw.get("conectividad", []),  
        "infraestructura": hardware_raw.get("infraestructura", {})  
    }  
  
    # Unificar clave 'antena' y 'antenas'  
    antenas = hardware_raw.get("antenas") or hardware_raw.get("antena") or []  
    for ant in antenas:  
        hw["antenas"].append({  
            "modelo": ant.get("MODELO") or ant.get("modelo"),  
            "tipo_conector": ant.get("TIPOCONECTOR", "Desconocido"),  
            "cantidad": ant.get("cantidad", 1)  
        })  
  
    # Normalizar RRU  
    for r in hardware_raw.get("rru", []):  
        modelo = r.get("MODELO") or r.get("modelo")  
        if not modelo.startswith("RRU"):  
            modelo = f"RRU {modelo}"  
        hw["rru"].append({  
            "modelo": modelo,  
            "tipo_conector": r.get("TIPOCONECTOR", "Desconocido"),  
            "tecnologias": r.get("tecnologias", [])  
        })  
  
    # BBU  
    if hardware_raw.get("bbu"):  
        hw["bbu"] = hardware_raw["bbu"]  
    return hw
```

Figura 44. Función normalizar hardware

### A.3 Lógica adicional del módulo de vectorización

Las funciones que contiene el script de vectorización permiten resumir bloques técnicos, generar textos narrativos y etiquetar cada fragmento con su sección correspondiente.

- **resumen\_antenas\_sector**: genera un resumen textual de la configuración de antenas para un sector específico. Describe el tipo de antena, las tecnologías asociadas, el azimut o el tilt.
- **resumen\_antenas\_global**: recorre todos los sectores del nodo y aplica `resumen_antenas_sector` a cada uno. Devuelve un resumen global con la distribución completa de antenas, ordenado por sectores. Facilita la comprensión del despliegue técnico de cada nodo.



- **resumen\_prl**: extrae la información clave del bloque PRL y la convierte en frases concisas sobre accesos, elementos de seguridad, altura de la torre o tipo de estructura.
- **resumen\_narrativo\_tareas**: recorre las tareas detectadas para cada empresa y genera frases ordenadas por tipo de acción: instalación, retirada, ampliación, etc. Este resumen permite que el sistema responda a preguntas como “¿Qué tareas debe realizar el cliente?” de forma directa.
- **json\_a\_docs**: es el núcleo del proceso de vectorización. Su objetivo es transformar el contenido del JSON en una lista de fragmentos textuales, cada uno etiquetado con su sección. Esta lista es la que posteriormente se indexa en el sistema de búsqueda. Debido a la complejidad y tamaño de la función, no se ha incluido el código completo.

Las funciones auxiliares del script de vectorización, a excepción de la función `json_a_docs`, se muestran en la Fig. 45.

```
def resumen_antenas_sector(sector, lista_antenas):
    modelos = set()
    tecnologias = set()
    orientaciones = set()
    alturas = set()
    for ant in lista_antenas:
        if ant.get("modelo"):
            modelos.add(ant["modelo"])
        for t in ant.get("tecnologias", []):
            tecnologias.add(t.strip())
        if ant.get("orientacion"):
            orientaciones.add(ant["orientacion"])
        if ant.get("altura_top"):
            alturas.add(ant["altura_top"])
    resumen = (
        f"En el {sector} hay {len(lista_antenas)} antenas instaladas. "
        f"Modelos: {'', '.join(modelos)}. "
        f"Tecnologías soportadas: {'', '.join(tecnologias)}. "
        f"Orientaciones: {'', '.join(orientaciones)}. "
        f"Alturas top: {'', '.join(alturas)}."
    )
    return resumen

def resumen_antenas_global(antenas_por_sector):
    total_antenas = 0
    sectores = []
    modelos = set()
    tecnologias = set()
    orientaciones = set()
    alturas = set()
    for sector, lista_antenas in antenas_por_sector.items():
        sectores.append(sector)
        total_antenas += len(lista_antenas)
        for ant in lista_antenas:
            if ant.get("modelo"):
                modelos.add(ant["modelo"])
            for t in ant.get("tecnologias", []):
                tecnologias.add(t.strip())
            if ant.get("orientacion"):
                orientaciones.add(ant["orientacion"])
            if ant.get("altura_top"):
                alturas.add(ant["altura_top"])
    resumen = (
        f"El nodo cuenta con {total_antenas} antenas distribuidas en {len(sectores)} sectores: {'', '.join(sectores)}. "
        f"Modelos instalados: {'', '.join(modelos)}. "
        f"Tecnologías soportadas: {'', '.join(tecnologias)}. "
        f"Orientaciones presentes: {'', '.join(orientaciones)}. "
        f"Alturas top: {'', '.join(alturas)}."
    )
    return resumen

def resumen_prl(prl_dict):
    def frase(tipo, campos):
        partes = []
        if "ubicacion" in campos and campos["ubicacion"]:
            ubic = ", ".join(campos["ubicacion"])
            partes.append(f"están ubicadas en {ubic.lower()}")
        if "acceso" in campos and campos["acceso"]:
            acc = ", ".join(campos["acceso"])
            partes.append(f"se accede por {acc.lower()}")
        if "seguridad" in campos and campos["seguridad"]:
            seg = ", ".join(campos["seguridad"])
            partes.append(f"y cuentan con {seg.lower()} como medidas de seguridad")
        if partes:
            return f"Las {tipo} {'', '.join(partes)}."
        else:
            return ""
    resmenes = []
    for tipo in ["antenas", "equipos"]:
        if tipo in prl_dict:
            resumen = frase(tipo, prl_dict[tipo])
            if resumen:
                resmenes.append(resumen)
    return " ".join(resmenes)

def resumen_narrativo_tareas(actor, resumen_dict):
    frases = []
    for categoria, lista in resumen_dict.items():
        if lista:
            categoria_txt = categoria.replace("_", " ").capitalize()
            for tarea in lista:
                frases.append(f"{categoria_txt}: {tarea.strip()}")
    if frases:
        joined = " ".join(frases)
        return f"Resumen de las tareas principales que debe realizar el {actor}: {joined}"
    return None
```

Figura 45. Funciones auxiliares del script de vectorización

## Anexo B. Manual de usuario

### B.1 Introducción

El presente manual tiene como objetivo guiar al usuario en el uso del sistema desarrollado para la extracción automatizada y consulta conversacional de información técnica contenida en documentos CAP.

Este sistema ha sido diseñado para facilitar el acceso a información técnica crítica, especialmente en entornos donde los documentos presentan variabilidad en formato, lenguaje técnico y estructura visual.

Está dirigido a perfiles tanto técnicos como no técnicos, incluyendo: equipo de diseño radio, técnicos de operación y mantenimiento o supervisores o perfiles no especializados.

El sistema se basa en una arquitectura local ligera que combina técnicas de OCR, procesamiento de lenguaje natural y modelos de lenguaje ejecutados en local (LLM), todo ello integrado en una interfaz web sencilla y accesible. El usuario puede cargar documentos, lanzar consultas técnicas en lenguaje natural y obtener respuestas con contexto trazable extraído directamente de los documentos originales.

### B.2 Preparación del entorno

Para ejecutar el sistema localmente, es necesario contar con un entorno Python correctamente configurado, así como con los modelos y herramientas necesarias para el procesamiento de documentos, extracción de texto, indexación semántica y generación de respuestas.

El sistema debe contar con los siguientes requisitos:

- **Sistema operativo:** Windows 10+, Ubuntu 20.04+ o equivalente.
- **Python:** versión 3.10 o superior.
- **Memoria RAM:** mínimo 8 GB (recomendado 16 GB).
- **Espacio en disco:** al menos 5 GB libres.
- **GPU (opcional):** para acelerar la ejecución de modelos LLM en Ollama.

El entorno requiere instalar las siguientes librerías de Python:

```
1  Pillow
2  opencv-python
3  faiss-cpu
4  fastapi
5  PyMuPDF
6  numpy
7  pandas
8  pdf2image
9  pdfplumber
10 pydantic
11 pytesseract
12 requests
13 sentence-transformers
14 streamlit
```

Las cuales se encuentran indicadas en el fichero `requirements.txt`.

Para las librerías Pytesseract y pdf2image es necesario instalar los siguientes paquetes de los repositorios de GitHub indicados a continuación:

1. Tesseract: <https://github.com/UB-Mannheim/tesseract/wiki>
2. Poppler: <https://github.com/oschwartz10612/poppler-windows/releases>

Además, se recomienda crear un entorno virtual para ejecutar el proyecto e instalar las librerías a partir del fichero `requirements.txt`:

```
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate    # Windows
pip install -r requirements.txt
```

Por otro lado, para ejecutar los modelos LLM en local, es necesaria la instalación de la herramienta Ollama. Para instalarla:

1. Acceder a <https://ollama.com> y descargar el instalador correspondiente.
2. Una vez instalado, abrir la terminal del sistema y ejecutar el siguiente comando para descargar en local los modelos requeridos.

```
ollama run llama3 # Descargar y ejecutar en la terminal
ollama pull llama3 # Solo descargar en local
```

### B.3 Descripción general del sistema

El sistema permite consultar información técnica contenida en documentos CAP mediante lenguaje natural. Aunque internamente el sistema realiza varias etapas complejas, el usuario solo necesita interactuar con la interfaz web una vez que los documentos han sido previamente preparados. A continuación, se explican las fases principales, diferenciando aquellas que requieren preparación previa de las que forman parte de la experiencia de uso.

Antes de realizar consultas, es necesario que el documento CAP esté indexado correctamente. Esto implica ejecutar una serie de scripts automáticos, proporcionados con el sistema, que procesan el documento PDF y lo preparan para su consulta:

1. Extracción de texto mediante OCR: El sistema analiza visualmente el documento y extrae el contenido relevante.
2. Normalización y estructuración de datos: El texto extraído se transforma en un formato uniforme (JSON), anonimizando datos sensibles y organizando la información por bloques técnicos.
3. Generación de fragmentos y creación del índice: A partir de ese JSON, el sistema genera pequeños textos etiquetados por cada bloque, que se almacenan en una base de datos semántica (FAISS). Esto es lo que permite luego hacer consultas contextuales.



Una vez completadas estas fases, el documento está listo para ser consultado por cualquier usuario desde la interfaz web:

4. Selección del documento y formulación de la consulta: El usuario elige un nodo CAP disponible e introduce una pregunta en lenguaje natural relacionada con su contenido técnico.
5. Procesamiento y generación de respuesta: El sistema busca los fragmentos más relevantes que responden a la consulta y utiliza un modelo de lenguaje (LLM) para redactar una respuesta precisa y contextualizada, únicamente a partir de esos fragmentos.
6. Visualización del resultado: La respuesta generada es entregada al usuario a través de la API y la interfaz web.

Para un mayor rendimiento del sistema se recomienda lanzar las consultas bloque a bloque, ya que el sistema se organiza por bloques de información extraídos del CAP, si se mezclan dos bloques de contenido el sistema podría dar respuestas imprecisas.

## B.4 Instrucciones de uso

La interfaz ha sido diseñada para facilitar la consulta de información técnica a partir de documentos CAP, sin necesidad de conocimientos avanzados en programación. A continuación, se detallan los pasos necesarios para utilizar el sistema desde la interfaz web.

### B.4.1 Inicializar el sistema

Para inicializar el sistema se tienen que ejecutar los siguientes comandos desde la carpeta raíz del proyecto y desde dos terminales diferentes:

1. Arrancar el servidor API: `python -m uvicorn backend.api.main:app --reload`

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [13412] using StatReload
INFO:      Started server process [17876]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

2. Iniciar la aplicación web o chatbot: `python -m streamlit run frontend/chatbot.py`

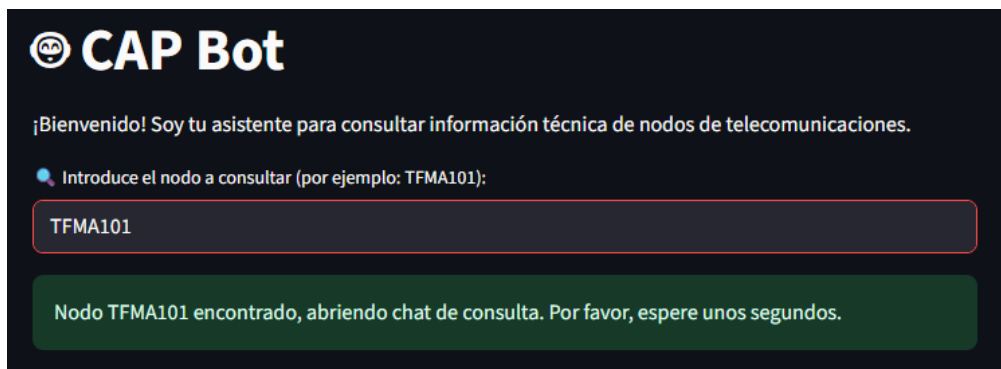
```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.38:8501
```

### B.4.2 Realizar la consulta

Tras cargar la interfaz web:

3. Seleccionar el nodo a consultar: Indicar en el cuadro de texto el nodo a consultar



4. Escribir la consulta: En el campo de entrada, introducir una pregunta técnica en lenguaje natural.



5. Enviar la consulta y visualización de la respuesta: Pulsar el botón de enviar o presionar *Enter*. El sistema llamará a la API y recuperará los fragmentos relevantes, generando una respuesta final que será mostrada en el chat.



## CAP Bot

Chat de consulta para el nodo TFMA101



¿Que modelos de antena y de RRU están instalados en el nodo?



Según los fragmentos proporcionados, se puede determinar que:

- El nodo dispone de 4 antenas:
  - ASI4518R39v07
  - AIR 6419 3,5 B78Y 5G
  - A04220PA01v06 (no especifica el modelo)
- El nodo dispone de 5 RRUs:
  - RRU 11 B7
  - RRU 4499 B1/B3
  - RRU 4486 B8/B20/B28
  - RRU 4485 B1/B3/B7
  - RRU 2479 B8/B20/B28

En cuanto al tipo de conector, se especifica que los RRUs RRU 2479, RRU 11 y RRU 4486 tienen un tipo de conector: 4.3-10.

No se encuentra información sobre el modelo A04220PA01v06 (antena) o los demás RRUs restantes (4499, 4485).

Escribe tu pregunta...

